

Class: CL_JVA_MIG_CORRECT_ACDOCA

Status: Active

Implemented interfaces

IF_JVA_MIG_CORRECT_ACDOCA (Interface for the correction of ACDOCA based on JVA ledger)

Attributes

Description: Correction of ACDOCA values based on JVA ledger data

Instantiation: Private

Final

Not released

Fixed Point Arithmetic

Category: General Object Type

Package: GJVA_MIG

Original lang.: EN

Created by: SAP

Created On: 05.08.2021

Last changed on: 21.09.2026

Attributes

Public attributes

Attrib.	Cat	Description	Ref. Type	Init. value
GC_JVDOCTYPE	Stat. (read only)		STAT.	
GC_JVLINETYPE	Stat. (read only)		STAT.	
GC_JVACTIVITY	Stat. (read only)		STAT.	
GC_ISSUE_TYPE	Stat. (read only)		STAT.	
GC_CORR_TYPE	Stat. (read only)		STAT.	
GC_JVPOSTTYPE	Stat. (read only)		STAT.	
GC_ERROR_TYPE	Stat. (read only)		STAT.	

Protected attribute

Attrib.	Cat	Description	Ref. Type	Init. value
GO_INSTANCE	Stat.		TYPE REF TO IF_JVA_MIG_CORRECT_ACDOCA	

Private attribute

Attrib.	Cat	Description	Init. value
Ref. Type			
MO_OUTPUT	Inst		
TYPE REF TO IF_JVA_OUTPUT			
MS_PROGPARS	Inst		
TYPE IF_JVA_MIG_CORRECT_ACDOCA~GTY_PROGRAM_PARAMETERS			
MT_TASKDATA	Inst		
TYPE GTY_T_TASKDATA			
MO_SETTINGS	Inst		
TYPE REF TO IF_JVA_SETTINGS			
MS_CURR_INFO	Inst		
TYPE FINSS_CMP_CURR_INFO			

Attrib. Ref. Type	Cat	Description	Init. value
MT_LEDGERS TYPE GTY_T_LEDGER_INFO	Inst		
MR_LEDGERS INST	Inst		
MV_JVA_INTEGRATED TYPE ABAP_BOOL	Inst		
MV_HAS_MIGRATED_DOCS TYPE ABAP_BOOL	Inst		
MV_HAS_MIGRATED_JV_DOCS TYPE ABAP_BOOL	Inst		
MV_TIMESTAMP_START TYPE TIMESTAMPL	Inst		
MS_CLASSIC_CURRENCIES TYPE JVA_AND_FI_CURRENCIES	Inst		
MT_FIELDS_JVS01 TYPE RSDD_T_COMPNAME	Inst		
MT_GROUP_JVS01 TYPE RSDD_T_COMPNAME	Inst		
MT_FIELDS_JVS02 TYPE RSDD_T_COMPNAME	Inst		
MT_GROUP_JVS02 TYPE RSDD_T_COMPNAME	Inst		
MT_FIELDS_ACDOCA TYPE RSDD_T_COMPNAME	Inst		
MT_GROUP_ACDOCA TYPE RSDD_T_COMPNAME	Inst		
MS_RESULTS_GLOBAL TYPE CL_JVA_MIG_ALV_OUTPUT=>IF_JVA_MIG_ALV_OUTPUT~GTY_S_RESULTS	Inst		
MS_RESULTS TYPE CL_JVA_MIG_ALV_OUTPUT=>IF_JVA_MIG_ALV_OUTPUT~GTY_S_RESULTS	Inst		
MT_RESULTS TYPE CL_JVA_MIG_ALV_OUTPUT=>IF_JVA_MIG_ALV_OUTPUT~GTY_T_RESULTS	Inst		
MT_JVAACDOCACORRLOG INST	Inst		
MV_MESSAGE_TEXT TYPE STRING	Inst		
MV_DELETE_ZERO_AMOUNT_ITEMS TYPE ABAP_BOOL	Inst		

Intern. types

Private types

Cat	Cat	Ref.	Type	Description
GTY_S_DOCS_PER_PERIOD	TYPE			
GTY_T_DOCS_PER_PERIOD	TYPE			
GTY_S_TASKDATA	TYPE			
GTY_T_TASKDATA	TYPE			
GTY_T_ACTIV	TYPE			
GTY_T_JVACTIVITY	TYPE			
GTY_S_LOG	TYPE			
GTY_T_LOG	TYPE			
GTY_S_DOC	TYPE			
GTY_T_DOC	TYPE			
GTY_S_REF	TYPE			
GTY_T_REF	TYPE			
GTY_S_CO_SPLIT_CRITERIA	TYPE			
GTY_S_ACD_COMPR_INFO	TYPE			
GTY_T_ACD_COMPR_INFO	TYPE			
GTY_S_JV_LINE_INFO	TYPE			
GTY_T_JV_LINE_INFO	TYPE			
GTY_S_JV_COMPR_INFO	TYPE			
GTY_T_JV_COMPR_INFO	TYPE			
GTY_S_JV_LINE_ASSIGNMENTS	TYPE			
GTY_T_JV_LINE_ASSIGNMENTS	TYPE			
GTY_S_ACD_LINE_ASSIGNMENTS	TYPE			
GTY_T_ACD_LINE_ASSIGNMENTS	TYPE			
GTY_S_LINE_ASSIGNMENTS	TYPE			
GTY_S_BALANCES_JVSO	TYPE			
GTY_T_BALANCES_JVSO	TYPE			
GTY_S_BALANCES_ACDOCA	TYPE			
GTY_T_BALANCES_ACDOCA	TYPE			
GTY_S_BALANCES	TYPE			
GTY_T_CORR_LOG	TYPE			
GTY_T_WHERE_CONDITION	TYPE			
GTY_S_LEDGER_INFO	TYPE			
GTY_T_LEDGER_INFO	TYPE			

Methods

Public methods

GET_INSTANCE

Description:

Static method

Returning parameter

```
VALUE(RO_INSTANCE) TYPE REF TO IF_JVA_MIG_CORRECT_ACDOCA

method get_instance.
  if go_instance is not bound.
    go_instance ?= new cl_jva_mig_correct_acdoca( ).
  endif.
  ro_instance = go_instance.
endmethod.
```

SET_INSTANCE

Description:
Static method

Importing parameter

IO_INSTANCE TYPE REF TO IF_JVA_MIG_CORRECT_ACDOCA OPTIONAL

```
method set_instance.  
  if io_instance is bound.  
    go_instance = io_instance.  
  else.  
    go_instance ?= new cl_jva_mig_correct_acdoca( ).  
  endif.  
endmethod.
```

Private methods

CONSTRUCTOR

Description:
Instance mthd

```
method constructor.  
  "Output handler for commands that are forbidden in unit tests  
  mo_output = new cl_jva_output( ).  
endmethod.
```

ADD_BAL_MESSAGE_TO_LOG

Description:
Instance mthd

Importing parameter

IS_JVSO_BAL TYPE GTY_S_BALANCES_JVSO OPTIONAL
IS_ACDOCA_BAL TYPE GTY_S_BALANCES_ACDOCA OPTIONAL
WITH_CO_DATA TYPE ABAP_BOOL OPTIONAL

Changing parameter

CS_DOC TYPE CL_JVA_MIG_CORRECT_ACDOCA=>GTY_S_DOC

```
method add_bal_message_to_log.  
  data: lv_acc_data_info type string,  
        lv_prodper      type string,  
        lv_message_count type i.  
  if is_jvso_bal is not initial.  
    lv_acc_data_info = is_jvso_bal-rjvnam && `/` && is_jvso_bal-regrou  && `/` &&  
                      is_jvso_bal-rpartn && `/` && is_jvso_bal-rrecin  && `/` &&  
                      is_jvso_bal-bilid.  
    if mo_settings->ms_t8jz-pm_act = abap_true.  
      lv_acc_data_info &&= `/` && is_jvso_bal-prodper.  
    endif.  
    if with_co_data  = abap_true.  
      lv_acc_data_info &&= `/` &&  
                      is_jvso_bal-rcntr  && `/` && is_jvso_bal-rprojk  && `/`  
&&  
                      is_jvso_bal-rordnr && `/` && is_jvso_bal-nplnr.  
    endif.
```

```

elseif is_acdoca_bal is not initial.
  lv_acc_data_info = is_acdoca_bal-vname && '/' && is_acdoca_bal-egrup && '/'
&&
  is_acdoca_bal-vptnr && '/' && is_acdoca_bal-recid && '/'
&&
  is_acdoca_bal-btype.
  if mo_settings->ms_t8jz-pm_act = abap_true.
    lv_acc_data_info &&= '/' && is_acdoca_bal-prodper.
  endif.
  if with_co_data = abap_true.
    lv_acc_data_info &&= '/' &&
      is_acdoca_bal-rcntr && '/' && is_acdoca_bal-ps_psp_pnr
&& '/' &&
      is_acdoca_bal-aufnr && '/' && is_acdoca_bal-nplnr.
  endif.
endif.
if is_jvso_bal is not initial and
  is_acdoca_bal is not initial.
  add_message_to_log(
    exporting
      iv_msg_text      = 'Different amounts found for'(dfa) && ` ` &&
lv_acc_data_info
      iv_msg_type      = space
      iv_indent_level  = 1
    changing
      cs_log           = cs_doc-balances_log ).
elseif is_jvso_bal is not initial.
  add_message_to_log(
    exporting
      iv_msg_text      = 'No balance found in GL doc for JV data'(nbg) && ` ` &&
lv_acc_data_info
      iv_msg_type      = space
      iv_indent_level  = 1
    changing
      cs_log           = cs_doc-balances_log ).
elseif is_acdoca_bal is not initial.
  add_message_to_log(
    exporting
      iv_msg_text      = 'No balance found in JV doc for GL data'(nbj) && ` ` &&
lv_acc_data_info
      iv_msg_type      = space
      iv_indent_level  = 1
    changing
      cs_log           = cs_doc-balances_log ).
endif.
if is_jvso_bal-trl <> is_acdoca_bal-wsl.
  add_message_to_log(
    exporting
      iv_msg_text      = 'bal. in trans. curr.'(btc) && `[` && is_jvso_bal-trl &&
`<>` && is_acdoca_bal-wsl && `]`
      iv_msg_type      = space
      iv_indent_level  = 2
    changing
      cs_log           = cs_doc-balances_log ).

```

```

        lv_message_count += 1.
    endif.
    if is_jvso_bal-hsl <> is_acdoca_bal-hsl.
        add_message_to_log(
            exporting
                iv_msg_text      = 'bal. in local curr. '(b1c) && ` [ ` && is_jvso_bal-hsl &&
`<>` && is_acdoca_bal-hsl && ` ] `
                iv_msg_type      = space
                iv_indent_level   = 2
            changing
                cs_log            = cs_doc-balances_log ).
        lv_message_count += 1.
    endif.
    if is_jvso_bal-ksl <> is_acdoca_bal-ksl.
        add_message_to_log(
            exporting
                iv_msg_text      = 'bal. in 2nd JV curr. '(b2c) && ` [ ` && is_jvso_bal-ksl &&
`<>` && is_acdoca_bal-ksl && ` ] `
                iv_msg_type      = space
                iv_indent_level   = 2
            changing
                cs_log            = cs_doc-balances_log ).
        lv_message_count += 1.
    endif.
    if is_jvso_bal-hsl_4c <> is_acdoca_bal-hsl_4c and
        lv_message_count < 3.
        add_message_to_log(
            exporting
                iv_msg_text      = 'bal. in 3rd JV curr. '(b3c) && ` [ ` && is_jvso_bal-hsl_4c
&& `<>` && is_acdoca_bal-hsl_4c && ` ] `
                iv_msg_type      = space
                iv_indent_level   = 2
            changing
                cs_log            = cs_doc-balances_log ).
        lv_message_count += 1.
    endif.
    if is_jvso_bal-ksl_4c <> is_acdoca_bal-ksl_4c and
        lv_message_count < 3.
        add_message_to_log(
            exporting
                iv_msg_text      = 'bal. in 4th JV curr. '(b4c) && ` [ ` && is_jvso_bal-ksl_4c
&& `<>` && is_acdoca_bal-ksl_4c && ` ] `
                iv_msg_type      = space
                iv_indent_level   = 2
            changing
                cs_log            = cs_doc-balances_log ).
        lv_message_count += 1.
    endif.
endmethod.
```

ADD_COMPRESSION_MESSAGE

Description:
Instance mthd

Importing parameter

IS_JVSO TYPE JVSO

Exporting parameter

EV_ALL_AMNTS_ZERO TYPE ABAP_BOOL

Changing parameter

CS_LOG TYPE GTY_S_LOG

CT_JV_COMPR_INFO TYPE GTY_T_JV_COMPR_INFO

```
method add_compression_message.
  data: lv_omitted_objects type string,
        lv_tabix_aggr      type sy-tabix,
        lv_line_string     type string,
        lv_message_text    type string.
  clear ev_all_amnts_zero.
  lv_tabix_aggr = is_jvso-tabix_aggr.
  read table ct_jv_compr_info
    assigning field-symbol(<ls_jv_compr_info>)
    with key tabix_aggr = lv_tabix_aggr.
  if sy-subrc <> 0 or
    <ls_jv_compr_info> is not assigned.
    return.
  endif.
  if <ls_jv_compr_info>-message_added = abap_true.
    return.
  endif.
  if <ls_jv_compr_info>-co_obj_omitted = abap_true.
    lv_omitted_objects = 'cost objects'(cob).
  endif.
  if <ls_jv_compr_info>-pc_omitted = abap_true.
    if lv_omitted_objects is not initial.
      lv_omitted_objects &&= `, `.
    endif.
    lv_omitted_objects = 'profit center'(prc).
  endif.
  if <ls_jv_compr_info>-orecid_omitted = abap_true.
    if lv_omitted_objects is not initial.
      lv_omitted_objects &&= `, `.
    endif.
    lv_omitted_objects = 'orig. RI'(ori).
  endif.
  if <ls_jv_compr_info>-tax_id_omitted = abap_true.
    if lv_omitted_objects is not initial.
      lv_omitted_objects &&= `, `.
    endif.
    lv_omitted_objects = 'tax ID'(tid).
  endif.
  if <ls_jv_compr_info>-po_omitted = abap_true.
    if lv_omitted_objects is not initial.
      lv_omitted_objects &&= `, `.
    endif.
    lv_omitted_objects = 'Purch. Order'(puo).
  endif.
  if lv_omitted_objects is initial.
    lv_omitted_objects = 'technical fields'(tcf).
```

```
endif.
<ls_jv_compr_info>-message_added = abap_true.
data(lv_line_count) = lines( <ls_jv_compr_info>-t_docln_original ).
if lv_line_count > 20.
  lv_line_string = `{` && lv_line_count && ` ` && 'lines'(lns) && `}`.
else.
  loop at <ls_jv_compr_info>-t_docln_original into data(ls_line_info).
    if lv_line_string is not initial.
      lv_line_string &&= `, `.
    endif.
    lv_line_string &&= condense( |{ ls_line_info-docln alpha = out }| ).
  endloop.
endif.
lv_message_text = 'JV doc was compressed by these lines:'(jdc) && ` ` &&
lv_line_string.
if is_jvso-tsl      = 0 and
  is_jvso-hsl      = 0 and
  is_jvso-ksl      = 0 and
  is_jvso-hsl_4c   = 0 and
  is_jvso-ksl_4c   = 0.
  ev_all_amnts_zero = abap_true.
  lv_message_text &&= `[` && 'All amounts sum up to zero'(aaz) && `]`.
endif.
add_message_to_log(
  exporting
    iv_msg_text = lv_message_text
    iv_msg_type = gc_error_type-info
  changing
    cs_log = cs_log ).
endmethod.
```

ADD_ISSUE_INFO

Description:

Instance mthd

Importing parameter

```
IS_LOG TYPE GTY_S_LOG
IS_ACDOCA TYPE ACDOCA_JV
IV_OBJECT_NAME TYPE CHAR100
IV_ISSUE_TYPE TYPE CHAR1
IV_JVSO_VALUE TYPE ANY
IV_ACDOCA_VALUE TYPE ANY
IV_ERROR_TYPE TYPE CHAR1
IV_IS_CO_LINE TYPE ABAP_BOOL OPTIONAL
```

Changing parameter

```
CT_LOG TYPE GTY_T_LOG
```

```
method add_issue_info.
  data(ls_log) = is_log.
  clear: ls_log-comment,
         ls_log-messages[].
  ls_log-issues &&= iv_issue_type.
  ls_log-corr_type = determine_correction_type( iv_corr_type = ls_log-corr_type
                                              iv_issue_type = iv_issue_type
```

```

iv_is_co_line = iv_is_co_line ).

add_message_to_log(
  exporting
    iv_msg_text = |{ iv_object_name } [{ iv_jvso_value }<>{ iv_acdoca_value }]|
    iv_msg_type = iv_error_type
  changing
    cs_log      = ls_log ).
append ls_log to ct_log.
endmethod.

```

ADD_MESSAGE_TO_LOG

Description:

Instance mthd

Importing parameter

```

IS_MESSAGE TYPE BAPIRET2  OPTIONAL
IV_MSG_TEXT TYPE STRING   OPTIONAL
IV_MSG_TYPE TYPE CHAR1    OPTIONAL
IV_INDENT_LEVEL TYPE I    OPTIONAL
IV_IS_UPD_MSG TYPE ABAP_BOOL  OPTIONAL
IV_POSITION TYPE SY-TABIX  OPTIONAL

```

Changing parameter

```

CS_LOG TYPE GTY_S_LOG

```

```

method add_message_to_log.
  data: ls_message      type bapiret2,
        lv_message_type like iv_msg_type,
        lv_message_id   type string,
        lv_message_text type string.
  if is_message is not initial.
    ls_message = is_message.
    lv_message_type = ls_message-type.
    if ls_message-id <> 'GJ' and
       ls_message-number <> '000'.
      lv_message_id = ls_message-id && `-' && ls_message-number.
    endif.
    if ls_message-message is not initial.
      if lv_message_id is initial.
        lv_message_text = ls_message-message.
      else.
        lv_message_text = lv_message_id && `: ` && ls_message-message.
      endif.
    elseif ls_message-id is not initial and
           ls_message-type is not initial and
           ls_message-number is not initial.
      message id ls_message-id type ls_message-type number ls_message-number
        with ls_message-message_v1 ls_message-message_v2
             ls_message-message_v3 ls_message-message_v4
        into lv_message_text.
      lv_message_text = lv_message_id && `: ` && lv_message_text.
    else.
      lv_message_text = lv_message_id.
    endif.
    if cs_log-comment is not initial.

```

```
        cs_log-comment &&= ` - ` .
    endif.
    cs_log-comment &&= lv_message_text.
else.
    lv_message_text = iv_msg_text.
    lv_message_type = iv_msg_type.
    if iv_is_upd_msg = abap_true.
        if cs_log-comment is initial.
            loop at cs_log-messages into ls_message
                where message is not initial.
                    if ls_message-message(4) = ` - ` .
                        continue.
                    endif.
                    if ls_message-message(2) = ` - ` .
                        ls_message-message = ls_message-message+2.
                    endif.
                    if cs_log-comment is initial.
                        cs_log-comment = ls_message-message.
                    else.
                        cs_log-comment &&= ` ` && ls_message-message.
                    endif.
                endloop.
            endif.
            if lv_message_text is not initial.
                if cs_log-comment is not initial.
                    cs_log-comment = lv_message_text && ` - ` && cs_log-comment.
                else.
                    cs_log-comment = lv_message_text.
                endif.
            endif.
        else.
            if cs_log-comment is not initial.
                cs_log-comment &&= ` - ` .
            endif.
            cs_log-comment &&= lv_message_text.
        endif.
    endif.
    if lv_message_text is not initial and
       ms_progpars-do_not_store_issue_details = abap_false and
       sy-batch is initial.
        case iv_indent_level.
            when 1.
                lv_message_text = ` - ` && lv_message_text.
            when 2.
                lv_message_text = ` - ` && lv_message_text.
        endcase.
        if ls_message-type is initial and
           lv_message_type is not initial.
            ls_message-type = lv_message_type.
        endif.
        ls_message-message = lv_message_text.
        if iv_position > 0.
            insert ls_message into cs_log-messages index iv_position.
        else.
```

```
        append ls_message to cs_log-messages.
    endif.
endif.
cs_log-error = set_error_type( iv_old_error = cs_log-error
                              iv_new_error = lv_message_type ).

endmethod.
```

ADD_UP_ALL_RESULTS_STATS

Description:

Instance mthd

Importing parameter

IS_RESULTS_TO_ADD TYPE CL_JVA_MIG_ALV_OUTPUT=>IF_JVA_MIG_ALV_OUTPUT~GTY_S_RESULTS

Changing parameter

CS_RESULTS TYPE CL_JVA_MIG_ALV_OUTPUT=>IF_JVA_MIG_ALV_OUTPUT~GTY_S_RESULTS

```
method add_up_all_results_stats.
    cs_results-doc_count += is_results_to_add-doc_count.
    cs_results-line_count += is_results_to_add-line_count.
    add_up_results_stats(
        exporting
            is_stats_to_add = is_results_to_add-fi_docs
        changing
            cs_stats        = cs_results-fi_docs ).
    add_up_results_stats(
        exporting
            is_stats_to_add = is_results_to_add-co_docs
        changing
            cs_stats        = cs_results-co_docs ).
    add_up_results_stats(
        exporting
            is_stats_to_add = is_results_to_add-equity_change
        changing
            cs_stats        = cs_results-equity_change ).
    add_up_results_stats(
        exporting
            is_stats_to_add = is_results_to_add-equity_adjust
        changing
            cs_stats        = cs_results-equity_adjust ).
    add_up_results_stats(
        exporting
            is_stats_to_add = is_results_to_add-farm_in_out
        changing
            cs_stats        = cs_results-farm_in_out ).
    add_up_results_stats(
        exporting
            is_stats_to_add = is_results_to_add-suspense
        changing
            cs_stats        = cs_results-suspense ).
    add_up_results_stats(
        exporting
            is_stats_to_add = is_results_to_add-cash_call
        changing
            cs_stats        = cs_results-cash_call ).
```

```
add_up_results_stats(  
  exporting  
    is_stats_to_add = is_results_to_add-cutback  
  changing  
    cs_stats      = cs_results-cutback ).  
endmethod.
```

ADD_UP_RESULTS_STATS_TOTALS

Description:
Instance mthd

Changing parameter

```
CS_RESULTS TYPE CL_JVA_MIG_ALV_OUTPUT=>IF_JVA_MIG_ALV_OUTPUT~GTY_S_RESULTS
```

```
method add_up_results_stats_totals.  
  add_up_results_stats(  
    exporting  
      is_stats_to_add = cs_results-fi_docs  
    changing  
      cs_stats      = cs_results-total ).  
  add_up_results_stats(  
    exporting  
      is_stats_to_add = cs_results-co_docs  
    changing  
      cs_stats      = cs_results-total ).  
  add_up_results_stats(  
    exporting  
      is_stats_to_add = cs_results-equity_change  
    changing  
      cs_stats      = cs_results-total ).  
  add_up_results_stats(  
    exporting  
      is_stats_to_add = cs_results-equity_adjust  
    changing  
      cs_stats      = cs_results-total ).  
  add_up_results_stats(  
    exporting  
      is_stats_to_add = cs_results-farm_in_out  
    changing  
      cs_stats      = cs_results-total ).  
  add_up_results_stats(  
    exporting  
      is_stats_to_add = cs_results-suspense  
    changing  
      cs_stats      = cs_results-total ).  
  add_up_results_stats(  
    exporting  
      is_stats_to_add = cs_results-cash_call  
    changing  
      cs_stats      = cs_results-total ).  
  add_up_results_stats(  
    exporting  
      is_stats_to_add = cs_results-cutback  
    changing
```

```
        cs_stats          = cs_results-total ).
endmethod.
```

ADD_UP_RESULTS_STATS

Description:
Instance mthd

Importing parameter

IS_STATS_TO_ADD TYPE
CL_JVA_MIG_ALV_OUTPUT=>IF_JVA_MIG_ALV_OUTPUT~GTY_S_ITEM_STATISTICS

Changing parameter

CS_STATS TYPE CL_JVA_MIG_ALV_OUTPUT=>IF_JVA_MIG_ALV_OUTPUT~GTY_S_ITEM_STATISTICS

```
method add_up_results_stats.
  cs_stats-lines_processed += is_stats_to_add-lines_processed.
  cs_stats-lines_ok += is_stats_to_add-lines_ok.
  cs_stats-lines_with_issues += is_stats_to_add-lines_with_issues.
  cs_stats-lines_updated += is_stats_to_add-lines_updated.
  cs_stats-lines_warnings += is_stats_to_add-lines_warnings.
  cs_stats-lines_alignm_failures += is_stats_to_add-lines_alignm_failures.
  cs_stats-lines_update_failures += is_stats_to_add-lines_update_failures.
  cs_stats-docs_processed += is_stats_to_add-docs_processed.
  cs_stats-docs_ok += is_stats_to_add-docs_ok.
  cs_stats-docs_with_issues += is_stats_to_add-docs_with_issues.
  cs_stats-docs_updated += is_stats_to_add-docs_updated.
  cs_stats-docs_warnings += is_stats_to_add-docs_warnings.
  cs_stats-docs_alignm_failures += is_stats_to_add-docs_alignm_failures.
  cs_stats-docs_update_failures += is_stats_to_add-docs_update_failures.
endmethod.
```

ADD_SY_MESSAGE_TO_RESULTS

Description:
Instance mthd

Changing parameter

CS_RESULTS TYPE CL_JVA_MIG_ALV_OUTPUT=>IF_JVA_MIG_ALV_OUTPUT~GTY_S_RESULTS

```
method add_sy_message_to_results.
  data: ls_message type bapiret2.
  if sy-msgid is initial.
    return.
  endif.
  ls_message = value #( id          = sy-msgid
                        number      = sy-msgno
                        type        = sy-msgty
                        message_v1  = sy-msgv1
                        message_v2  = sy-msgv2
                        message_v3  = sy-msgv3
                        message_v4  = sy-msgv4
                        message     = mv_message_text ).
  append ls_message to cs_results-t_messages.
endmethod.
```

ADD_ITEM_INFO_FOR_LINE

Description:

Instance mthd

Importing parameter

IV_UPDATE TYPE ABAP_BOOL

IS_LOG TYPE GTY_S_LOG

```
method add_item_info_for_line.
  data: ls_color      type lvc_s_scol,
        ls_col       type lvc_s_colo,
        ls_item_info type
cl_jva_mig_alv_output=>if_jva_mig_alv_output-gty_s_item_info.
  move-corresponding is_log to ls_item_info.
  ls_item_info-type = is_log-jvdoctype.
  if is_critical_error( is_log-error ).
    if is_log-corr_type = gc_corr_type-correction_not_possible.
      ls_col-col = col_negative.
      ls_col-int = 1.
    else.
      ls_col-col = col_negative.
      ls_col-int = 0.
    endif.
  elseif is_log-corr_type >= gc_corr_type-post_new_doc and
    is_log-updated = abap_false.
    ls_col-col = col_negative.
    ls_col-int = 1.
  elseif is_log-updated = abap_true or
    is_log-error = gc_error_type-warning.
    if ms_progpars-analysis_only = abap_true or
      ms_progpars-testrun = abap_true.
      ls_col-col = col_total.
    else.
      ls_col-col = col_positive.
    endif.
  endif.
  if ls_col-col <> 0.
    ls_color = value #( fname = 'COMMENT'
                       color = ls_col ).
    append ls_color to ls_item_info-t_color.
  endif.
  if ms_progpars-show_all_lines = abap_true.
    append ls_item_info to ms_results-t_item_info.
  elseif ms_progpars-show_updates_and_errors = abap_true.
    if iv_update = abap_true      or
      is_log-balances_correct = abap_true or
      ls_col-col <> 0.
      append ls_item_info to ms_results-t_item_info.
    endif.
  elseif ms_progpars-show_errors_only = abap_true.
    if ls_col-col = col_negative.
      append ls_item_info to ms_results-t_item_info.
    endif.
```

```
endif.  
endmethod.
```

ADD_ITEM_INFO_FOR_DOC

Description:
Instance mthd

Importing parameter

```
IV_UPDATE TYPE ABAP_BOOL  
IS_DOC TYPE GTY_S_DOC  
IT_DOC_COMB TYPE GTY_T_DOC  
IT_LOG TYPE GTY_T_LOG  
IT_ACDOCA_DOC TYPE ACDOCA_JV_T  
IV_MSG_TEXT TYPE STRING  
IV_MSG_TYPE TYPE CHAR1  
IV_UPDATED TYPE ABAP_BOOL OPTIONAL  
IV_BELNR TYPE BELNR_D OPTIONAL  
IV_GJAHR TYPE GJAHR OPTIONAL  
IO_MESSAGES TYPE REF TO IF_JVA_MESSAGE_COLLECTION OPTIONAL
```

```
method add_item_info_for_doc.  
  data: ls_message      type bapiret2,  
        lv_message_text type string,  
        lv_docnr_string type string,  
        lt_log_doc      type gty_t_log.  
  lv_message_text = iv_msg_text.  
  if ms_progpars-do_not_store_issue_details = abap_false and  
    sy_batch is initial.  
    if is_critical_error( iv_msg_type ).  
      lv_message_text &&= ` - ` && 'Double click here for the details!'(dch).  
    endif.  
  endif.  
  if lines( it_doc_comb ) > 1.  
    loop at it_doc_comb assigning field-symbol(<ls_doc>).  
      if lv_docnr_string is not initial.  
        lv_docnr_string &&= `, `.  
      endif.  
      lv_docnr_string &&= condense( |{ <ls_doc>-docnr alpha = out }| ).  
    endloop.  
  endif.  
  loop at it_doc_comb into data(ls_doc).  
    clear lt_log_doc.  
    read table it_log transporting no fields  
              with key rbukrs = ls_doc-rbukrs  
                    ryear  = ls_doc-ryear  
                    docnr   = ls_doc-docnr.  
  
    if sy-subrc = 0.  
      loop at it_log into data(ls_log)  
        from sy-tabix.  
        if ls_log-rbukrs <> ls_doc-rbukrs or  
          ls_log-ryear <> ls_doc-ryear or  
          ls_log-docnr <> ls_doc-docnr.  
          exit.  
        endif.  
      endloop.  
    endif.  
  endloop.  
endmethod.
```

```

        append ls_log to lt_log_doc.
    endloop.
else.
    continue.
endif.
loop at lt_log_doc into data(ls_log_doc)
    where activ <> gc_jvlinetype-balancing_line.

    exit.
endloop.
clear: ls_log_doc-docln,
        ls_log_doc-docln_acd,
        ls_log_doc-comment,
        ls_log_doc-messages.
ls_log_doc-corr_type = is_doc-corr_type.
ls_log_doc-balances_correct = is_doc-balances_correct.
if lv_docnr_string is not initial.
    add_message_to_log(
        exporting
lv_docnr_string      iv_msg_text      = 'Combined analysis for JV docs:'(cba) && ` ` &&
        iv_msg_type    = gc_error_type-info
        changing
        cs_log          = ls_log_doc ).
endif.
if is_doc-gl_no_of_lines = 0.
    if lines( it_acdoca_doc[] ) = 0 and
        iv_msg_type = gc_error_type-error.
        add_message_to_log(
            exporting
            iv_msg_text      = conv string( 'ACDOCA document not found'(adn) )
            iv_msg_type      = gc_error_type-info
            changing
            cs_log            = ls_log_doc ).
        endif.
    else.
        if is_doc-jv_no_of_lines <> is_doc-gl_no_of_lines.
            add_message_to_log(
                exporting
lines'(dnl) && | ( { is_doc-jv_no_of_lines } <> { i
                iv_msg_type      = gc_error_type-info
                changing
                cs_log            = ls_log_doc ).
            endif.
        endif.
        if is_doc-corr_type = gc_corr_type-post_new_doc.
            if lv_message_text is initial.
                lv_message_text = 'ACDOCA doc missing for JV doc'(adf) && ` ` && condense(
|{ ls_doc-docnr alpha = out }| ) && ` - ` && 'D
            endif.
        endif.
    loop at lt_log_doc into ls_log
        where error is not initial
            or corr_type >= gc_corr_type-update_line.

```

```
if iv_update = abap_true.
  update_line_statistics_updates( ls_log ).
endif.
if is_doc-corr_type = gc_corr_type-post_new_doc.
  if lines( ls_log-messages ) <= 20.
    "Add issue messages to main doc item
    loop at ls_log-messages into ls_message
      where type is not initial
      or id is not initial.
      append ls_message to ls_log_doc-messages.
    endloop.
  endif.
else.
  "Add issue messages to main doc item
  loop at ls_log-messages into ls_message
    where type is not initial
    or id is not initial
    or message(1) = '-'. "Infos about value diffs
    append ls_message to ls_log_doc-messages.
  endloop.
endif.
endloop.
"Add information about balances to main doc item
loop at is_doc-balances_log-messages into ls_message.
  append ls_message to ls_log_doc-messages.
endloop.
if io_messages is bound.
  update_log_with_posting_result(
    exporting
      iv_updated   = iv_updated
      iv_belnr     = iv_belnr
      iv_gjahr     = iv_gjahr
      iv_docln     = space
      iv_comment   = iv_msg_text
      io_messages  = io_messages
    changing
      cs_log       = ls_log_doc ).
elseif iv_msg_text is not initial.
  ls_message = value #( id       = 'GJ'
                       number    = '000'
                       type      = iv_msg_type
                       message_v1 = iv_msg_text
                       message   = iv_msg_text ).
  append ls_message to ls_log_doc-messages.
endif.
if lv_message_text is initial.
  lv_message_text = 'Issues found'(isf).
  if ms_progpars-do_not_store_issue_details = abap_false and
    sy-batch is initial.
    lv_message_text &&= ` - ` && 'Double click here for the details!'(dch).
  endif.
endif.
ls_log_doc-comment = lv_message_text.
if iv_msg_type is not initial.
```

```

        ls_log_doc-error = set_error_type( iv_old_error = ls_log_doc-error
                                           iv_new_error = iv_msg_type ).
    endif.
    if ms_progpars-testrun      <> abap_true and
       ms_progpars-analysis_only <> abap_true and
       ls_log_doc-updated = abap_true.
        update_log_db_table_prep( changing cs_log = ls_log_doc ).
    endif.
    add_item_info_for_line( iv_update = abap_false
                           is_log     = ls_log_doc ).

    if iv_update = abap_true.
        update_doc_statistics_updates( ls_log_doc ).
    endif.
endloop.
endmethod.
```

APPEND_LOG_ITEM_FOR_JV_LINE

Description:
Instance mthd

Importing parameter

- IS_LOG TYPE GTY_S_LOG
- IS_JVSO TYPE JVS0
- IT_JV_COMPR_INFO TYPE GTY_T_JV_COMPR_INFO

Changing parameter

- CT_LOG_DOC TYPE GTY_T_LOG

```

method append_log_item_for_jv_line.
    if is_jvso-docln is not initial.
        append is_log to ct_log_doc.
    else.
        read table it_jv_compr_info assigning field-symbol(<ls_jv_compr_info>)
                                   with key tabix_aggr = is_jvso-tabix_aggr.

        if sy-subrc = 0 and <ls_jv_compr_info> is assigned.
            data(ls_log) = is_log.
            loop at <ls_jv_compr_info>-t_docln_original into data(ls_line_info).
                ls_log-docnr = ls_line_info-docnr.
                ls_log-docln = ls_line_info-docln.
                append ls_log to ct_log_doc.
            endloop.
        endif.
    endif.
endmethod.
```

CALC_RUNTIME

Description:
Instance mthd

Importing parameter

- IV_TIMESTAMP_START TYPE TIMESTAMPL
- IV_TIMESTAMP_END TYPE TIMESTAMPL

Returning parameter

- VALUE(RV_RUNTIME) TYPE TZNTSTMPL

```
method calc_runtime.  
  try.  
    rv_runtime = cl_abap_tstmp=>subtract( tstmp1 = iv_timestamp_end  
                                         tstmp2 = iv_timestamp_start ).  
    catch cx_root.  
      rv_runtime = 0.  
      "#EC NEED_CX_ROOT  
  endtry.  
endmethod.
```

CALCULATE_GLOBAL_RUNTIMES

Description:
Instance mthd

```
method calculate_global_runtimes.  
  ms_results_global-runtime_main = calc_runtime( iv_timestamp_start =  
mv_timestamp_start  
                                              iv_timestamp_end   =  
get_timestamp( ) ).  
  if ms_progpars-parallel_processing = abap_true.  
    ms_results_global-runtime_total = ms_results_global-runtime_main  
                                     + ms_results_global-runtime_tasks  
                                     + ms_results_global-runtime_tasks_admin.  
  else.  
    ms_results_global-runtime_total = ms_results_global-runtime_main.  
  endif.  
endmethod.
```

CHECK_COCODES_AND_AUTHORITIES

Description:
Instance mthd

Returning parameter

VALUE(RV_ERROR) TYPE ABAP_BOOL

```
method check_cocodes_and_authorities.  
  data: lv_process_code   type jv_proca value 'CUTBACK2',  
        lv_activity       type activ_auth,  
        lv_auth_miss_count type i.  
  "Get the JV company codes  
  select bukrs from t8jz  
         where bukrs in @ms_progpars-r_bukrs  
         into table @ms_progpars-t_bukrs.  
  data(lv_jv_cocodes_count) = lines( ms_progpars-t_bukrs ).  
  if lv_jv_cocodes_count = 0.  
    rv_error = abap_true.  
    if sy-batch is initial.  
      message i802(g5).  
    else.  
      message e802(g5) into mv_message_text.  
      add_sy_message_to_results( changing cs_results = ms_results_global ).  
    endif.  
    return.  
  endif.  
  if ms_progpars-testrun = abap_false and
```

```
    ms_progpars-analysis_only = abap_false.
    lv_activity = '16'.
else.
    lv_activity = '48'.
endif.
"Check if the user has the required authorities
loop at ms_progpars-t_bukrs into data(ls_bukrs).
    data(lv_tabix) = sy-tabix.
    call function 'JV_AUTHORITY_CHECK_PROCESS'
        exporting
            bukrs      = ls_bukrs-bukrs
            process_code = lv_process_code
            activity    = lv_activity
        exceptions
            no_authority = 1.
    if sy-subrc <> 0.
        lv_auth_miss_count += 1.
        delete ms_progpars-t_bukrs index lv_tabix.
    endif.
endloop.
if lv_auth_miss_count > 0.
    if lv_auth_miss_count = lv_jv_cocodes_count.
        rv_error = abap_true.
        if sy-batch is initial.
            message i292(g4) with ls_bukrs-bukrs lv_process_code lv_activity.
        else.
            message e292(g4) with ls_bukrs-bukrs lv_process_code lv_activity into
mv_message_text.
            add_sy_message_to_results( changing cs_results = ms_results_global ).
        endif.
    endif.
endif.
* Get the GL ledger
loop at ms_progpars-t_bukrs into ls_bukrs.
    call function 'JV_GET_LEDGER'
        exporting
            iv_bukrs      = ls_bukrs-bukrs
        importing
            ev_rldnr      = ms_progpars-jv_ledger
        exceptions
            not_found     = 1
            more_than_one = 2
            jva_not_active = 3
            others         = 4.
    if sy-subrc = 0 and
        ms_progpars-jv_ledger is not initial.
        exit.
    endif.
endloop.
endmethod.
```

CHECK_FOR_REPOSTINGS_ACDOCA

Description:
Instance mthd

Changing parameter
CT_ACDOCA TYPE ACDOCA_JV_T

Returning parameter
VALUE(RV_REPOSTED) TYPE ABAP_BOOL

```
method check_for_repostings_acdoca.
  data: lv_timestamp_latest type acdoca-timestamp.
  data(lv_original_line_count) = lines( ct_acdoca ).
  "Check if the document was reposted by RGJVREC8
  "If yes, delete the irrelevant items
  loop at ct_acdoca assigning field-symbol(<ls_acdoca>).
    "where sgtxt+26(6) = 'REPOST'
    "  and sgtxt(8)    = 'RGJVREC8'.
    if lv_timestamp_latest is initial or
      <ls_acdoca>-timestamp > lv_timestamp_latest.
      lv_timestamp_latest = <ls_acdoca>-timestamp.
    endif.
  endloop.
  if lv_timestamp_latest is not initial.
    "Keep only the lines with the latest timestamp
    delete ct_acdoca where timestamp <> lv_timestamp_latest.
    "Check if there is a reposting
    read table ct_acdoca transporting no fields
              with key sgtxt(8)      = 'RGJVREC8'
                    sgtxt+26(6) = 'REPOST'.

    if sy-subrc <= 2.
      "Delete reversal lines
      delete ct_acdoca where sgtxt(8)      = 'RGJVREC8'
                    and sgtxt+26(6) <> 'REPOST'.

    endif.
  endif.
  if lv_original_line_count <> lines( ct_acdoca ).
    rv_reposted = abap_true.
  endif.
endmethod.
```

CHECK_FOR_REPOSTINGS_JV

Description:
Instance mthd

Changing parameter
CT_JVSO TYPE JVSO_T

Returning parameter
VALUE(RV_REPOSTED) TYPE ABAP_BOOL

```
method check_for_repostings_jv.
  data: lv_latest_cpudt type cpudt,
        lv_latest_cputm type cputm.
  data(lv_original_line_count) = lines( ct_jvso ).
```

```
"Check if the document was reposted by RGJVREC8
"If yes, delete the irrelevant items
loop at ct_jvso assigning field-symbol(<ls_jvso>)
    where rldnr = '4A'.
    "and sgtxt+26(6) = 'REPOST'
    "and sgtxt(8) = 'RGJVREC8'.
    if lv_latest_cpudt is initial or
       lv_latest_cputm is initial or
       <ls_jvso>-cpudt > lv_latest_cpudt or
       ( <ls_jvso>-cpudt = lv_latest_cpudt and
         <ls_jvso>-cputm > lv_latest_cputm ).
       lv_latest_cpudt = <ls_jvso>-cpudt.
       lv_latest_cputm = <ls_jvso>-cputm.
    endif.
endloop.
if lv_latest_cpudt is not initial.
    "Keep only the lines with the latest CPU date and time
    delete ct_jvso where cpudt <> lv_latest_cpudt
                        or cputm <> lv_latest_cputm.
    "Check if there is a reposting
    read table ct_jvso transporting no fields
                        with key sgtxt(8) = 'RGJVREC8'
                        sgtxt+26(6) = 'REPOST'.

    if sy-subrc <= 2.
        "Delete reversal lines
        delete ct_jvso where sgtxt(8) = 'RGJVREC8'
                            and sgtxt+26(6) <> 'REPOST'.
    endif.
endif.
if lv_original_line_count <> lines( ct_jvso ).
    rv_reposted = abap_true.
endif.
endmethod.
```

COMBINE_AND_ANALYZE_JV_LINES

Description:

Instance mthd

Exporting parameter

ET_JVSO_COMBINED TYPE JVSO_T

ET_REF TYPE GTY_T_REF

Changing parameter

CT_JVSO TYPE JVSO_T

CT_DOC TYPE GTY_T_DOC

```
method combine_and_analyze_jv_lines.
    data: ls_jvso_comb type jvso,
          lv_docnr_last type jvso,
          ls_ref        type gty_s_ref,
          ls_ref_last   type gty_s_ref,
          lv_tabix_new  type sy-tabix.
    field-symbols: <ls_doc> type gty_s_doc.
    clear: et_jvso_combined,
           et_ref.
```

```
loop at ct_jvso assigning field-symbol(<ls_jvso>).
  if <ls_jvso>-rldnr = '4A'.
    clear ls_ref.
    move-corresponding <ls_jvso> to ls_ref.
    if ls_ref <> ls_ref_last.
      ls_ref_last = ls_ref.
      collect ls_ref into et_ref.
    endif.
    if <ls_jvso>-docnr <> lv_docnr_last.
      lv_docnr_last = <ls_jvso>-docnr.
      unassign <ls_doc>.
      read table ct_doc assigning <ls_doc>
        with key ryear = <ls_jvso>-ryear
              docnr = <ls_jvso>-docnr
              binary search.
    endif.
    if <ls_doc> is assigned.
      if <ls_doc>-rkokrs is initial.
        if <ls_jvso>-rkokrs is not initial.
          <ls_doc>-rkokrs = <ls_jvso>-rkokrs.
        endif.
      endif.
      if <ls_jvso>-activ <> gc_jvlinetype-balancing_line.
        if <ls_doc>-activ is initial.
          if <ls_jvso>-activ is not initial.
            <ls_doc>-activ = <ls_jvso>-activ.
          endif.
        endif.
        if <ls_jvso>-reffidoc is not initial.
          if <ls_jvso>-refactiv = gc_jvdoctype-co_doc.
            clear <ls_jvso>-reffidoc.
          else.
            if <ls_doc>-reffidoc is initial.
              <ls_doc>-reffidoc = <ls_jvso>-reffidoc.
            elseif <ls_doc>-reffidoc <> <ls_jvso>-reffidoc.
              clear <ls_doc>-reffidoc.
            endif.
          endif.
        endif.
        if <ls_jvso>-refdocnr is not initial.
          if <ls_doc>-refdocnr is initial.
            <ls_doc>-refdocnr = <ls_jvso>-refdocnr.
          elseif <ls_doc>-refdocnr <> <ls_jvso>-refdocnr.
            clear <ls_doc>-refdocnr.
          endif.
        endif.
      endif.
    endif.
    if <ls_jvso>-rldnr = '4A' or
       <ls_jvso>-rldnr = '4B'.
      clear ls_jvso_comb.
      move-corresponding <ls_jvso> to ls_jvso_comb.
      if <ls_jvso>-rldnr = '4B'.
```

```
    ls_jvso_comb-bilid = <ls_jvso>-rbilid.
    clear: ls_jvso_comb-refdocnr,
           ls_jvso_comb-refdocln.
endif.
ls_jvso_comb-curt1 = ms_classic_currencies-curt1_jv.
ls_jvso_comb-hwaer = ms_classic_currencies-hwaer_jv.
if ms_classic_currencies-curt2_jv is initial.
    clear ls_jvso_comb-ksl.
else.
    ls_jvso_comb-curt2 = ms_classic_currencies-curt2_jv.
    ls_jvso_comb-hwae2 = ms_classic_currencies-hwae2_jv.
endif.
append ls_jvso_comb to et_jvso_combined.
lv_tabix_new = sy-tabix.
else.
if <ls_jvso>-docnr = ls_jvso_comb-docnr and
   <ls_jvso>-docln = ls_jvso_comb-docln.
    if ms_classic_currencies-curt3_jv is not initial.
        ls_jvso_comb-hsl_4c = <ls_jvso>-hsl.
        ls_jvso_comb-curt3 = ms_classic_currencies-curt3_jv.
        ls_jvso_comb-hwae3 = ms_classic_currencies-hwae3_jv.
    endif.
    if ms_classic_currencies-curt4_jv is not initial.
        ls_jvso_comb-ksl_4c = <ls_jvso>-ksl.
        ls_jvso_comb-curt4 = ms_classic_currencies-curt4_jv.
        ls_jvso_comb-hwae4 = ms_classic_currencies-hwae4_jv.
    endif.
    modify et_jvso_combined from ls_jvso_comb index lv_tabix_new.
else.
    clear ls_jvso_comb.
    move-corresponding <ls_jvso> to ls_jvso_comb.
    if <ls_jvso>-rldnr = '4D'.
        ls_jvso_comb-bilid = <ls_jvso>-rbilid.
    endif.
    clear: ls_jvso_comb-hsl, ls_jvso_comb-ksl.
    if ms_classic_currencies-curt3_jv is not initial.
        ls_jvso_comb-hsl_4c = <ls_jvso>-hsl.
        ls_jvso_comb-curt3 = ms_classic_currencies-curt3_jv.
        ls_jvso_comb-hwae3 = ms_classic_currencies-hwae3_jv.
    endif.
    if ms_classic_currencies-curt4_jv is not initial.
        ls_jvso_comb-ksl_4c = <ls_jvso>-ksl.
        ls_jvso_comb-curt4 = ms_classic_currencies-curt4_jv.
        ls_jvso_comb-hwae4 = ms_classic_currencies-hwae4_jv.
    endif.
    append ls_jvso_comb to et_jvso_combined.
    lv_tabix_new = sy-tabix.
endif.
endif.
endloop.
endmethod.
```

COMPARE_AND_CORRECT_DOCUMENTS

Description:

Instance mthd

Importing parameter

IT_JVSO TYPE JVSO_T
IT_ACDOCA TYPE ACDOCA_JV_T
IT_ACDOCA_INT TYPE ACDOCA_JV_T
IT_ACDOCA_MIG TYPE ACDOCA_JV_T
IT_CORR_LOG TYPE GTY_T_CORR_LOG

Changing parameter

CT_DOC_JV TYPE GTY_T_DOC

```
method compare_and_correct_documents.  
  data: ls_doc          type gty_s_doc,  
        ls_doc_main     type gty_s_doc,  
        lt_doc_comb      type gty_t_doc,  
        ls_doc_jv_next   type gty_s_doc,  
        lt_jvso_doc      type jvso_t,  
        lt_acdoca_doc     type acdoca_jv_t,  
        lt_log_doc       type gty_t_log,  
        lv_has_reversal   type abap_bool,  
        lv_has_reposting type abap_bool,  
        lv_posting_mode   type char1,  
        lv_corr_type      type char1,  
        ls_line_assignments type gty_s_line_assignments,  
        ls_balances       type gty_s_balances,  
        lt_ledgers        type rldnr_t.  
  loop at ct_doc_jv into data(ls_doc_jv).  
    clear ls_doc_jv_next.  
    read table ct_doc_jv into ls_doc_jv_next index sy-tabix + 1.  
    append ls_doc_jv to lt_doc_comb.  
    if ls_doc_jv-is_fi_doc = abap_true or  
       ls_doc_jv-is_co_doc = abap_true.  
      if ls_doc_jv-rbukrs = ls_doc_jv_next-rbukrs and  
         ls_doc_jv-ryear = ls_doc_jv_next-ryear and  
         ls_doc_jv-activ = ls_doc_jv_next-activ and  
         ls_doc_jv-reffidoc = ls_doc_jv_next-reffidoc and  
         ls_doc_jv-refdocnr = ls_doc_jv_next-refdocnr.  
        continue.  
      endif.  
    endif.  
    clear: lt_jvso_doc,  
           ls_doc_main,  
           ls_balances,  
           lv_has_reversal,  
           lv_has_reposting,  
           lv_posting_mode.  
    loop at lt_doc_comb into data(ls_doc_comb).  
      read table it_jvso transporting no fields  
                with key rbukrs = ls_doc_comb-rbukrs  
                      ryear = ls_doc_comb-ryear  
                      docnr = ls_doc_comb-docnr
```

```
        binary search.
if sy-subrc <> 0.
    continue.
endif.
if ls_doc_main is initial.
    ls_doc_main = ls_doc_comb.
endif.
loop at it_jvso into data(ls_jvso)
    from sy-tabix.
    if ls_jvso-rbukrs <> ls_doc_comb-rbukrs or
       ls_jvso-ryear <> ls_doc_comb-ryear or
       ls_jvso-docnr <> ls_doc_comb-docnr.
        exit.
    endif.
    if mv_delete_zero_amount_items = abap_true.
        "Lines with zero amounts are not relevant, so ignore them
        if ls_jvso-tsl = 0 and
           ls_jvso-hsl = 0 and
           ls_jvso-ksl = 0 and
           ls_jvso-hsl_4c = 0 and
           ls_jvso-ksl_4c = 0.
            continue.
        endif.
    endif.
    if ms_progpars-select_fi_documents = abap_true or
       ms_progpars-select_co_documents = abap_true.
        "Ignore the balancing lines when the corresponding option is active
        if ms_progpars-exclude_balancing_lines = abap_true.
            if ls_jvso-activ = gc_jvlinetype-balancing_line or
               ls_jvso-refactiv = gc_jvlinetype-balancing_line.
                ls_doc_main-jv_no_of_lines_excluded += 1.
                continue.
            endif.
        endif.
        "Do not process line without FI or CO doc reference
        if ( ls_jvso-activ = gc_jvlinetype-fi_doc or
            ls_jvso-activ = gc_jvlinetype-fi_doc_mm ) and
           ls_jvso-reffidoc is initial.
            continue.
        endif.
        if ls_jvso-activ = gc_jvlinetype-co_doc and
           ls_jvso-refdocnr is initial.
            continue.
        endif.
    else.
        if ls_doc_main-is_fi_doc = abap_false and
           ls_doc_main-is_co_doc = abap_false.
            "In case of internal documents ignore the balancing lines always
            if ls_jvso-activ = gc_jvlinetype-balancing_line or
               ls_jvso-refactiv = gc_jvlinetype-balancing_line.
                ls_doc_main-jv_no_of_lines_excluded += 1.
                continue.
            endif.
            "Equity Adjustment and Suspense postings can have offset lines
```

```
"with the corporate recovery indicator; they need to be deleted
"to not cause issues during line checks
if ls_doc_main-activ = gc_jvlinetype-eq_adjust_reversal or
   ls_doc_main-activ = gc_jvlinetype-eq_adjust_reposting or
   ls_doc_main-activ = gc_jvlinetype-suspense or
   ls_doc_main-activ = gc_jvlinetype-unsuspense.
   if ls_jvso-rrecin = mo_settings->ms_t8jz-corprecind.
     ls_doc_main-jv_no_of_lines_excluded += 1.
     continue.
   endif.
endif.
endif.
endif.
append ls_jvso to lt_jvso_doc.
if ls_doc_main-is_fi_doc = abap_true          or
   ls_doc_main-is_co_doc = abap_true          or
   ls_doc_main-jvdoctype = gc_jvdoctype-cutback or
   ls_doc_main-jvdoctype = gc_jvdoctype-cash_call.
   prepare_jvso_balances( exporting
                          is_jvso = ls_jvso
                          changing
                          cs_balances = ls_balances ).
endif.
determine_jv_doc_type( exporting
                      iv_activ      = ls_jvso-activ
                      changing
                      cs_doc        = ls_doc_main
                      cv_has_reversal = lv_has_reversal
                      cv_has_reposting = lv_has_reposting ).

endloop.
endloop.
if lt_jvso_doc[] is initial.
  clear lt_doc_comb.
  continue.
endif.
if ms_progpars-select_fi_documents = abap_true or
   ms_progpars-select_co_documents = abap_true.
  if ls_doc_main-jvdoctype <> gc_jvdoctype-fi_doc and
     ls_doc_main-jvdoctype <> gc_jvdoctype-co_doc.
    clear lt_doc_comb.
    continue.
  endif.
else.
  "Determine the jv posting mode
  if lv_has_reversal = abap_true and
     lv_has_reposting = abap_true.
    if lines( lt_jvso_doc ) = 2.
      lv_posting_mode = gc_jvposttype-combined.
    else.
      lv_posting_mode = gc_jvposttype-unspecific.
    endif.
  elseif lv_has_reversal = abap_true.
    lv_posting_mode = gc_jvposttype-reversal.
  elseif lv_has_reposting = abap_true.
```

```
lv_posting_mode = gc_jvposttype-reposting.
endif.
ls_doc-posting_mode = lv_posting_mode.
endif.
loop at mt_ledgers into data(ls_ledger).
  clear: lt_log_doc,
         lt_acdoca_doc,
         ls_balances-t_acdoca_jv,
         ls_balances-t_acdoca_jv_co.
  ls_doc = ls_doc_main.
  ls_doc-ledger = ls_ledger-rldnr.
  "Get the complete ACDOCA document, if possible
  get_acdoca_doc_lines(
    exporting
      it_acdoca_all = it_acdoca
      it_acdoca_mig = it_acdoca_mig
      it_corr_log   = it_corr_log
    importing
      et_acdoca     = lt_acdoca_doc
    changing
      cs_doc        = ls_doc ).
  if ls_ledger-rldnr <> ms_progpars-jv_ledger and
     lt_acdoca_doc[] is initial.
    continue.
  endif.
  ms_curr_info = ls_ledger-s_curr_info.
  prepare_acdoca_document(
    changing
      cs_doc      = ls_doc
      ct_acdoca   = lt_acdoca_doc
      cs_balances = ls_balances ).
  if ms_progpars-exclude_balancing_lines = abap_true.
    "If balancing lines were excluded from a migrated ACDOCA doc
    "but not from the JV document, there is a issue with the
    "values in the ACTIV and REFACTIV fields. In this case
    "delete the lines from the JV doc where REFFIDLN is initial
    if ls_doc-is-fi-doc = abap_true and
       ls_doc-mig_source = 'J' and
       ls_doc-jv_no_of_lines_excluded = 0 and
       ls_doc-gl_no_of_lines_excluded > 0.
      data(lv_line_count) = lines( lt_jvso_doc ).
      delete lt_jvso_doc where reffidln is initial.
      if sy-subrc = 0.
        ls_doc-jv_no_of_lines_excluded = lv_line_count - lines( lt_jvso_doc ).
      endif.
    endif.
  endif.
  ls_doc-gl_no_of_lines = lines( lt_acdoca_doc ).
  if ls_ledger-rldnr = ms_progpars-jv_ledger.
    if ls_doc-jv-is-fi-doc = abap_true and
       ls_doc-rldnr = '4A' and
       lines( lt_doc_comb[] ) > 1 and
       ( ls_doc-mig_source <> 'J' or
         lines( lt_jvso_doc ) <> lines( lt_acdoca_doc ) ).
```

```

        data(lv_reposted) = check_for_repostings_jv( changing ct_jvso =
lt_jvso_doc ).
        if lv_reposted = abap_true.
            if ls_doc-mig_source = 'J'.
                check_for_repostings_acdoca( changing ct_acdoca = lt_acdoca_doc ).
            endif.
        endif.
        sort lt_jvso_doc by docnr docln.
    endif.
    ls_doc-jv_no_of_lines = lines( lt_jvso_doc ).
    update_doc_statistic_processed( ls_doc-jvdoctype ).
    "Compare the JV doc lines with the corresponding ACDOCA lines
    "The method is recursive when heuristic line assignment is active!
    compare_lines(
        exporting
            is_doc           = ls_doc
            it_jvso_doc       = lt_jvso_doc
            it_acdoca_all     = it_acdoca
            it_acdoca_int     = it_acdoca_int
            it_acdoca_mig     = it_acdoca_mig
            it_corr_log       = it_corr_log
        importing
            es_line_assignments = ls_line_assignments
            ev_corr_type       = lv_corr_type
            et_log_doc         = lt_log_doc
        changing
            ct_acdoca_doc      = lt_acdoca_doc ).
    ls_doc-corr_type = lv_corr_type.
    if ls_doc-corr_type > gc_corr_type-update_line      and
        ls_balances is not initial.
        compare_balances( changing
            cs_balances = ls_balances
            cs_doc      = ls_doc ).
    endif.
    loop at lt_doc_comb assigning field-symbol(<ls_doc>).
        <ls_doc>-corr_type = ls_doc-corr_type.
    endloop.
    data(lv_doc_stats_updated) = abap_false.
    loop at lt_log_doc assigning field-symbol(<ls_log>).
        if sy-tabix = 1.
            "Update doc statistics
            if ls_doc-corr_type = gc_corr_type-correction_not_possible.
                update_doc_stats_alignm_fails( <ls_log> ).
            endif.
        endif.
        "Update line statistics
        if <ls_log>-corr_type = gc_corr_type-correction_not_possible or
            <ls_log>-error = gc_error_type-error.
            update_line_stats_alignm_fails( <ls_log> ).
        elseif <ls_log>-issues is not initial.
            update_line_statistics_issues( <ls_log> ).
        if lv_doc_stats_updated = abap_false.
            lv_doc_stats_updated = abap_true.

```

```
        update_doc_statistics_issues( <ls_log> ).
    endif.
endif.
endloop.
sort lt_log_doc by rbukrs ryear docnr docln.
case ls_doc-corr_type.
    when gc_corr_type-update_line or
         gc_corr_type-rev_and_repost_line or
         gc_corr_type-post_missing_line.
        correct_acdoca_lines(
            exporting
                it_jvso_doc      = lt_jvso_doc
                it_acdoca_doc    = lt_acdoca_doc
                is_line_assignments = ls_line_assignments
                iv_ledger        = ls_ledger-rldnr
                iv_jvdoctype     = ls_doc-jvdoctype
                iv_posting_mode   = lv_posting_mode
            changing
                ct_log_doc      = lt_log_doc ).
    when gc_corr_type-post_new_doc.
        post_missing_document(
            exporting
                is_doc          = ls_doc
                it_doc_comb     = lt_doc_comb
                it_jvso_doc     = lt_jvso_doc
                iv_ledger       = ls_ledger-rldnr
                iv_posting_mode = lv_posting_mode
            changing
                ct_log_doc     = lt_log_doc ).
    when gc_corr_type-rev_and_repost_doc.
        rev_and_rep_acdoca_doc(
            exporting
                is_doc          = ls_doc
                it_doc_comb     = lt_doc_comb
                it_jvso_doc     = lt_jvso_doc
                it_acdoca_doc   = lt_acdoca_doc
                iv_ledger       = ls_ledger-rldnr
                iv_posting_mode = lv_posting_mode
            changing
                ct_log_doc     = lt_log_doc ).
    when gc_corr_type-do_not_correct.
        if ms_progpars-analysis_only <> abap_true and
           ms_progpars-show_all_lines = abap_true.
            loop at lt_log_doc into data(ls_log).
                ls_log-belnr_new = 'NOT POSTED'  ##NO_TEXT.
                add_message_to_log(
                    exporting
                        iv_msg_text = conv string( 'NOT POSTED due to settings'(ntp) )
                        iv_msg_type = gc_error_type-statement
                    changing
                        cs_log = ls_log ).
                add_item_info_for_line( iv_update = abap_false
                                       is_log     = ls_log ).
            endloop.
        else.
            loop at lt_log_doc into data(ls_log).
                ls_log-belnr_new = 'NOT POSTED'  ##NO_TEXT.
                add_message_to_log(
                    exporting
                        iv_msg_text = conv string( 'NOT POSTED due to settings'(ntp) )
                        iv_msg_type = gc_error_type-statement
                    changing
                        cs_log = ls_log ).
                add_item_info_for_line( iv_update = abap_false
                                       is_log     = ls_log ).
            endloop.
        endif.
    endcase.
```

```
endif.
when gc_corr_type-correction_not_possible.
  add_item_info_for_doc(
    iv_update      = abap_false
    is_doc         = ls_doc
    it_doc_comb    = lt_doc_comb
    it_log         = lt_log_doc
    it_acdoca_doc  = lt_acdoca_doc
    iv_msg_text    = conv string( 'The detected issues cannot be fixed
through this program'(icf) )
    iv_msg_type    = gc_error_type-error ).
when gc_corr_type-correction_not_required.
  if ls_doc-balances_correct = abap_true.
    data(lv_message_text) = conv string( 'Document OK - Line assignment
failures occurred, but the balances match'(blm) ).
  else.
    lv_message_text = conv string( 'Document OK - No issues detected'(dok)
).
endif.
if ms_progpars-show_all_lines = abap_true or
   ls_doc-balances_correct = abap_true.
  add_item_info_for_doc(
    iv_update      = abap_false
    is_doc         = ls_doc
    it_doc_comb    = lt_doc_comb
    it_log         = lt_log_doc
    it_acdoca_doc  = lt_acdoca_doc
    iv_msg_text    = lv_message_text
    iv_msg_type    = gc_error_type-info ).
else.
  loop at lt_log_doc transporting no fields
      where error = gc_error_type-error
      or error = gc_error_type-abend
      or error = gc_error_type-shortdump.

      exit.
  endloop.
  if sy-subrc = 0.
    loop at lt_log_doc into ls_log.
      add_item_info_for_line( iv_update = abap_false
                             is_log    = ls_log ).
    endloop.
  endif.
endif.
endcase.
endloop.
clear lt_doc_comb.
endloop.
endmethod.
```

COMPARE_BALANCES

Description:

Instance mthd

Changing parameter

CS_BALANCES TYPE CL_JVA_MIG_CORRECT_ACDOCA=>GTY_S_BALANCES

CS_DOC TYPE CL_JVA_MIG_CORRECT_ACDOCA=>GTY_S_DOC

method compare_balances.

data: lv_balance_error type abap_bool.

field-symbols: <ls_jvso> type gty_s_balances_jvso,
 <ls_acdoca> type gty_s_balances_acdoca.

if ((cs_doc-activ = gc_jvlinetype-fi_doc or
 cs_doc-activ = gc_jvlinetype-fi_doc_mm) and
 ms_progpars-check_co_in_bs_lines = abap_true) or
 (cs_doc-activ = gc_jvlinetype-co_doc and
 ms_progpars-check_co_in_co_lines = abap_true).

if cs_balances-t_acdoca_jv_co is initial or
 cs_balances-t_jvso_jv_co is initial.

return.

endif.

sort: cs_balances-t_acdoca_jv_co,
 cs_balances-t_jvso_jv_co.

"Check if the JV balances can be found in the GL balances

loop at cs_balances-t_jvso_jv_co assigning <ls_jvso>.

unassign <ls_acdoca>.

read table cs_balances-t_acdoca_jv_co assigning <ls_acdoca>

with key racct	= <ls_jvso>-racct
vname	= <ls_jvso>-rjvnam
egrup	= <ls_jvso>-regrou
vptnr	= <ls_jvso>-rpartn
recid	= <ls_jvso>-rrecin
btype	= <ls_jvso>-bilid
prodper	= <ls_jvso>-prodper
rcntr	= <ls_jvso>-rcntr
aufnr	= <ls_jvso>-rordnr
ps_psp_pnr	= <ls_jvso>-rprojk
nplnr	= <ls_jvso>-nplnr

binary search.

if <ls_acdoca> is not assigned.

 lv_balance_error = abap_true.

else.

 <ls_acdoca>-matched = abap_true.

 <ls_jvso>-matched = abap_true.

 if <ls_jvso>-tsl = <ls_acdoca>-wsl and
 <ls_jvso>-hsl = <ls_acdoca>-hsl and
 <ls_jvso>-ksl = <ls_acdoca>-ksl and
 <ls_jvso>-hsl_4c = <ls_acdoca>-hsl_4c and
 <ls_jvso>-ksl_4c = <ls_acdoca>-ksl_4c.

 <ls_acdoca>-identical = abap_true.

 <ls_jvso>-identical = abap_true.

else.

 lv_balance_error = abap_true.

 add_bal_message_to_log(exporting

 is_jvso_bal = <ls_jvso>

 is_acdoca_bal = <ls_acdoca>

 with_co_data = abap_true

 changing

```

                                cs_doc      = cs_doc ).

endif.
endif.
endloop.
"Check if the remaining GL balances can be found in the JV balances
loop at cs_balances-t_acdoca_jv_co assigning <ls_acdoca>
                                where matched = abap_false.

unassign <ls_jvso>.
read table cs_balances-t_jvso_jv_co assigning <ls_jvso>
    with key racct   = <ls_acdoca>-racct
          rjvnam    = <ls_acdoca>-vname
          regrou    = <ls_acdoca>-egrup
          rpartn    = <ls_acdoca>-vptnr
          rrecin    = <ls_acdoca>-recid
          bilid     = <ls_acdoca>-btype
          prodper   = <ls_acdoca>-prodper
          rcntr     = <ls_acdoca>-rcntr
          rordnr    = <ls_acdoca>-aufnr
          rprojk    = <ls_acdoca>-ps_psp_pnr
          nplnr     = <ls_acdoca>-nplnr
          binary search.
if <ls_jvso> is not assigned.
    lv_balance_error = abap_true.
else.
    <ls_acdoca>-matched = abap_true.
    <ls_jvso>-matched = abap_true.
    if <ls_acdoca>-wsl   = <ls_jvso>-tsl   and
       <ls_acdoca>-hsl   = <ls_jvso>-hsl   and
       <ls_acdoca>-ksl   = <ls_jvso>-ksl   and
       <ls_acdoca>-hsl_4c = <ls_jvso>-hsl_4c and
       <ls_acdoca>-ksl_4c = <ls_jvso>-ksl_4c.
        <ls_acdoca>-identical = abap_true.
        <ls_jvso>-identical = abap_true.
    else.
        lv_balance_error = abap_true.
        add_bal_message_to_log( exporting
                                is_jvso_bal   = <ls_jvso>
                                is_acdoca_bal = <ls_acdoca>
                                with_co_data  = abap_true
                                changing
                                cs_doc        = cs_doc ).

endif.
endif.
endloop.
if lv_balance_error = abap_true.
    loop at cs_balances-t_jvso_jv_co assigning <ls_jvso>
                                where matched = abap_false.

        add_bal_message_to_log( exporting
                                is_jvso_bal   = <ls_jvso>
                                with_co_data  = abap_true
                                changing
                                cs_doc        = cs_doc ).

endloop.
loop at cs_balances-t_acdoca_jv_co assigning <ls_acdoca>

```

```
                                where matched = abap_false.
add_bal_message_to_log( exporting
                        is_acdoca_bal = <ls_acdoca>
                        with_co_data = abap_true
                        changing
                        cs_doc         = cs_doc ).

endloop.
endif.
else.
  if cs_balances-t_acdoca_jv is initial or
     cs_balances-t_jvso_jv   is initial.
    return.
  endif.
  sort: cs_balances-t_acdoca_jv,
        cs_balances-t_jvso_jv.
  "Check if the JV balances can be found in the GL balances
  loop at cs_balances-t_jvso_jv assigning <ls_jvso>.
    unassign <ls_acdoca>.
    read table cs_balances-t_acdoca_jv assigning <ls_acdoca>
              with key racct      = <ls_jvso>-racct
                      vname      = <ls_jvso>-rjvnam
                      egrou      = <ls_jvso>-regrou
                      vptnr      = <ls_jvso>-rpartn
                      recid      = <ls_jvso>-rrecin
                      btype      = <ls_jvso>-bilid
                      prodper    = <ls_jvso>-prodper
              binary search.
    if <ls_acdoca> is not assigned.
      lv_balance_error = abap_true.
    else.
      <ls_acdoca>-matched = abap_true.
      <ls_jvso>-matched = abap_true.
      if <ls_jvso>-tsl    = <ls_acdoca>-wsl    and
         <ls_jvso>-hsl    = <ls_acdoca>-hsl    and
         <ls_jvso>-ksl    = <ls_acdoca>-ksl    and
         <ls_jvso>-hsl_4c = <ls_acdoca>-hsl_4c and
         <ls_jvso>-ksl_4c = <ls_acdoca>-ksl_4c.
        <ls_acdoca>-identical = abap_true.
        <ls_jvso>-identical = abap_true.
      else.
        lv_balance_error = abap_true.
        add_bal_message_to_log( exporting
                                is_jvso_bal   = <ls_jvso>
                                is_acdoca_bal = <ls_acdoca>
                                changing
                                cs_doc         = cs_doc ).
      endif.
    endif.
  endloop.
  "Check if the remaining GL balances can be found in the JV balances
  loop at cs_balances-t_acdoca_jv assigning <ls_acdoca>
                                where matched = abap_false.

    unassign <ls_jvso>.
    read table cs_balances-t_jvso_jv assigning <ls_jvso>
```

```

        with key racct   = <ls_acdoca>-racct
            rjvnam      = <ls_acdoca>-vname
            regrou      = <ls_acdoca>-egrup
            rpartn      = <ls_acdoca>-vptnr
            rrecin      = <ls_acdoca>-recid
            bilid       = <ls_acdoca>-btype
            prodper     = <ls_acdoca>-prodper
        binary search.
    if <ls_jvso> is not assigned.
        lv_balance_error = abap_true.
    else.
        <ls_acdoca>-matched = abap_true.
        <ls_jvso>-matched = abap_true.
        if <ls_acdoca>-wsl   = <ls_jvso>-tsl   and
            <ls_acdoca>-hsl   = <ls_jvso>-hsl   and
            <ls_acdoca>-ksl   = <ls_jvso>-ksl   and
            <ls_acdoca>-hsl_4c = <ls_jvso>-hsl_4c and
            <ls_acdoca>-ksl_4c = <ls_jvso>-ksl_4c.
            <ls_acdoca>-identical = abap_true.
            <ls_jvso>-identical = abap_true.
        else.
            lv_balance_error = abap_true.
            add_bal_message_to_log( exporting
                                    is_jvso_bal   = <ls_jvso>
                                    is_acdoca_bal = <ls_acdoca>
                                    changing
                                    cs_doc        = cs_doc ).
        endif.
    endif.
endloop.
if lv_balance_error = abap_true.
    loop at cs_balances-t_jvso_jv assigning <ls_jvso>
        where matched = abap_false.
        add_bal_message_to_log( exporting
                                is_jvso_bal   = <ls_jvso>
                                changing
                                cs_doc        = cs_doc ).
    endloop.
    loop at cs_balances-t_acdoca_jv assigning <ls_acdoca>
        where matched = abap_false.
        add_bal_message_to_log( exporting
                                is_acdoca_bal = <ls_acdoca>
                                changing
                                cs_doc        = cs_doc ).
    endloop.
endif.
if lv_balance_error = abap_true.
    add_message_to_log(
        exporting
        iv_msg_text      = conv string( 'Detected balances between JV and GL
document:'(bdt) )
        iv_msg_type      = 'E'
        iv_position      = 1

```

```
        changing
        cs_log          = cs_doc-balances_log ).
elseif ms_progpars-balance_check_over_line_check = abap_true.
    cs_doc-corr_type = gc_corr_type-correction_not_required.
    cs_doc-balances_correct = abap_true.
endif.
endmethod.
```

COMPARE_FIELD_VALUES

Description:

Instance mthd

Importing parameter

```
IS_DOC TYPE GTY_S_DOC
IS_JVSO TYPE JVSO
IS_ACDOCA TYPE ACDOCA_JV
IV_TABIX_ACD TYPE SY-TABIX
IV_JVDOCTYPE TYPE CHAR2
IT_JV_COMPR_INFO TYPE GTY_T_JV_COMPR_INFO
IT_ACD_COMPR_INFO TYPE GTY_T_ACD_COMPR_INFO
```

Changing parameter

```
CS_LOG TYPE GTY_S_LOG
```

```
method compare_field_values.
data: lv_wsl      type acdoca-wsl,
      lv_hsl      type acdoca-hsl,
      lv_ksl      type acdoca-ksl,
      lv_hsl_4c   type acdoca-hsl,
      lv_ksl_4c   type acdoca-ksl,
      lv_wsl_rev  type acdoca-wsl,
      lv_hsl_rev  type acdoca-hsl,
      lv_ksl_rev  type acdoca-ksl,
      lv_hsl_4c_rev type acdoca-hsl,
      lv_ksl_4c_rev type acdoca-ksl,
      lv_issue_type type c,
      lv_object_name type char100,
      lv_is_co_line type abap_bool,
      lt_log       type gty_t_log,
      ls_message   type bapiret2,
      lv_corr_type type c,
      lv_error_type type c,
      lv_first_day type sy-datum,
      lv_last_day  type sy-datum,
      ls_cokey     type cokey_is.
field-symbols: <lv_acdoca_value> type any,
               <lv_jvso_value>  type any.
if is_acdoca-co_buzei is not initial or
( is_acdoca-mig_source is not initial and
  is_acdoca-glaccount_type = 'P' ).
    lv_is_co_line = abap_true.
endif.
"Check if the amounts match
if cs_log-heuristic_hit = abap_false.
    lv_wsl      = is_jvso-trl.
```

```

lv_hsl      = is_jvso-hsl.
lv_ksl      = is_jvso-ksl.
lv_hsl_4c   = is_jvso-hsl_4c.
lv_ksl_4c   = is_jvso-ksl_4c.
lv_wsl_rev  = - is_jvso-tsl.
lv_hsl_rev  = - is_jvso-hsl.
lv_ksl_rev  = - is_jvso-ksl.
lv_hsl_4c_rev = - is_jvso-hsl_4c.
lv_ksl_4c_rev = - is_jvso-ksl_4c.
if ( is_acdoca-wsl = lv_wsl_rev and
    is_acdoca-wsl <> 0 ) or
    ( is_acdoca-hsl = lv_hsl_rev and
    is_acdoca-hsl <> 0 ) or
    ( is_acdoca-ksl = lv_ksl_rev and
    is_acdoca-ksl <> 0 ) or
    ( is_acdoca-hsl_4c = lv_hsl_4c_rev and
    is_acdoca-hsl_4c <> 0 ) or
    ( is_acdoca-ksl_4c = lv_ksl_4c_rev and
    is_acdoca-ksl_4c <> 0 ).
cs_log-issues &&= gc_issue_type-wrong_amount_sign.
if iv_jvdoctype = gc_jvdoctype-fi_doc or
    iv_jvdoctype = gc_jvdoctype-co_doc or
    iv_jvdoctype = gc_jvdoctype-cash_call or
    iv_jvdoctype = gc_jvdoctype-cutback.
cs_log-corr_type = gc_corr_type-correction_not_possible.
lv_error_type = gc_error_type-error.
else.
cs_log-corr_type = gc_corr_type-rev_and_repost_doc.
lv_error_type = gc_error_type-warning.
endif.
if is_acdoca-wsl <> 0 and
    is_acdoca-wsl = lv_wsl_rev.
add_issue_info(
    exporting
        is_log      = cs_log
        is_acdoca    = is_acdoca
        iv_object_name = 'trans. curr. sign'(trs)
        iv_issue_type = lv_issue_type
        iv_jvso_value = is_jvso-tsl
        iv_acdoca_value = is_acdoca-wsl
        iv_error_type = lv_error_type
    changing
        ct_log      = lt_log ).
endif.
if is_acdoca-hsl <> 0 and
    is_acdoca-hsl = lv_hsl_rev.
add_issue_info(
    exporting
        is_log      = cs_log
        is_acdoca    = is_acdoca
        iv_object_name = 'local curr. sign'(lcs)
        iv_issue_type = lv_issue_type
        iv_jvso_value = is_jvso-hsl
        iv_acdoca_value = is_acdoca-hsl

```

```
        iv_error_type = lv_error_type
    changing
        ct_log = lt_log ).
endif.
if is_acdoca-ksl <> 0 and
    is_acdoca-ksl = lv_ksl_rev.
    add_issue_info(
        exporting
            is_log = cs_log
            is_acdoca = is_acdoca
            iv_object_name = '2nd local curr. sign'(2cs)
            iv_issue_type = lv_issue_type
            iv_jvso_value = is_jvso-ksl
            iv_acdoca_value = is_acdoca-ksl
            iv_error_type = lv_error_type
        changing
            ct_log = lt_log ).
endif.
if is_acdoca-hsl_4c <> 0 and
    is_acdoca-hsl_4c = lv_hsl_4c_rev.
    add_issue_info(
        exporting
            is_log = cs_log
            is_acdoca = is_acdoca
            iv_object_name = '3rd local curr. sign'(3cs)
            iv_issue_type = lv_issue_type
            iv_jvso_value = is_jvso-hsl_4c
            iv_acdoca_value = is_acdoca-hsl_4c
            iv_error_type = lv_error_type
        changing
            ct_log = lt_log ).
endif.
if is_acdoca-ksl_4c <> 0 and
    is_acdoca-ksl_4c = lv_ksl_4c_rev.
    add_issue_info(
        exporting
            is_log = cs_log
            is_acdoca = is_acdoca
            iv_object_name = '4th local curr. sign'(4cs)
            iv_issue_type = lv_issue_type
            iv_jvso_value = is_jvso-ksl_4c
            iv_acdoca_value = is_acdoca-ksl_4c
            iv_error_type = lv_error_type
        changing
            ct_log = lt_log ).
endif.
elseif is_acdoca-wsl <> lv_wsl or
    is_acdoca-hsl <> lv_hsl or
    is_acdoca-ksl <> lv_ksl or
    is_acdoca-hsl_4c <> lv_hsl_4c or
    is_acdoca-ksl_4c <> lv_ksl_4c.
cs_log-issues &&= gc_issue_type-wrong_amount.
if iv_jvdoctype = gc_jvdoctype-fi_doc or
    iv_jvdoctype = gc_jvdoctype-co_doc or
```

```
    iv_jvdoctype = gc_jvdoctype-cash_call or
    iv_jvdoctype = gc_jvdoctype-cutback.
    cs_log-corr_type = gc_corr_type-correction_not_possible.
    lv_error_type = gc_error_type-error.
else.
    cs_log-corr_type = gc_corr_type-rev_and_repost_doc.
    lv_error_type = gc_error_type-warning.
endif.
if is_acdoca-wsl <> lv_wsl.
    add_issue_info(
        exporting
            is_log          = cs_log
            is_acdoca       = is_acdoca
            iv_object_name  = 'trans. curr. amnt.'(tra)
            iv_issue_type   = lv_issue_type
            iv_jvso_value   = is_jvso-tsl
            iv_acdoca_value = is_acdoca-wsl
            iv_error_type   = lv_error_type
        changing
            ct_log          = lt_log ).
endif.
if is_acdoca-hsl <> lv_hsl.
    add_issue_info(
        exporting
            is_log          = cs_log
            is_acdoca       = is_acdoca
            iv_object_name  = 'local curr. amnt.'(lca)
            iv_issue_type   = lv_issue_type
            iv_jvso_value   = is_jvso-hsl
            iv_acdoca_value = is_acdoca-hsl
            iv_error_type   = lv_error_type
        changing
            ct_log          = lt_log ).
endif.
if is_acdoca-ksl <> lv_ksl.
    add_issue_info(
        exporting
            is_log          = cs_log
            is_acdoca       = is_acdoca
            iv_object_name  = '2nd local curr. amnt.'(2ca)
            iv_issue_type   = lv_issue_type
            iv_jvso_value   = is_jvso-ksl
            iv_acdoca_value = is_acdoca-ksl
            iv_error_type   = lv_error_type
        changing
            ct_log          = lt_log ).
endif.
if is_acdoca-hsl_4c <> lv_hsl_4c.
    add_issue_info(
        exporting
            is_log          = cs_log
            is_acdoca       = is_acdoca
            iv_object_name  = '3rd local curr. amnt.'(3ca)
            iv_issue_type   = lv_issue_type
```

```
        iv_jvso_value   = is_jvso-hsl_4c
        iv_acdoca_value = is_acdoca-hsl_4c
        iv_error_type   = lv_error_type
    changing
        ct_log          = lt_log ).
endif.
if is_acdoca-ksl_4c <> lv_ksl_4c.
    add_issue_info(
        exporting
            is_log       = cs_log
            is_acdoca     = is_acdoca
            iv_object_name = '4th local curr. amnt.'(4ca)
            iv_issue_type = lv_issue_type
            iv_jvso_value = is_jvso-ksl_4c
            iv_acdoca_value = is_acdoca-ksl_4c
            iv_error_type = lv_error_type
    changing
        ct_log          = lt_log ).
endif.
endif.
endif.
if ms_progpars-check_venture = abap_true or
ms_progpars-check_jva_data = abap_true.
    "Check if the ventures match
    if is_acdoca-vname <> is_jvso-rjvnam.
        add_issue_info(
            exporting
                is_log       = cs_log
                is_acdoca     = is_acdoca
                iv_object_name = 'venture'(ven)
                iv_issue_type = gc_issue_type-wrong_venture
                iv_jvso_value = is_jvso-rjvnam
                iv_acdoca_value = is_acdoca-vname
                iv_error_type = gc_error_type-warning
            changing
                ct_log       = lt_log ).
        endif.
    endif.
endif.
if ms_progpars-check_equity_group = abap_true or
ms_progpars-check_jva_data = abap_true.
    "Check if the equity groups match
    if is_acdoca-egrup <> is_jvso-regrou.
        add_issue_info(
            exporting
                is_log       = cs_log
                is_acdoca     = is_acdoca
                iv_object_name = 'equity group'(eqg)
                iv_issue_type = gc_issue_type-wrong_equity_group
                iv_jvso_value = is_jvso-regrou
                iv_acdoca_value = is_acdoca-egrup
                iv_error_type = gc_error_type-warning
            changing
                ct_log       = lt_log ).
        endif.
    endif.
endif.
```

```
endif.
if ms_progpars-check_billing_indicator = abap_true or
ms_progpars-check_jva_data = abap_true.
  "Check if the billing indicators match
  if is_acdoca-btype <> is_jvso-bilid.
    add_issue_info(
      exporting
        is_log          = cs_log
        is_acdoca       = is_acdoca
        iv_object_name  = 'billing indicator'(bil)
        iv_issue_type   = gc_issue_type-wrong_billing_indicator
        iv_jvso_value   = is_jvso-bilid
        iv_acdoca_value = is_acdoca-btype
        iv_error_type   = gc_error_type-warning
      changing
        ct_log          = lt_log ).
  endif.
endif.
"The following fields are available in ledger 4A only
if is_jvso-rldnr = '4A'.
  if ms_progpars-check_venture_partner = abap_true or
  ms_progpars-check_jva_data = abap_true.
    "Check if the venture partners match
    if is_acdoca-vptnr <> is_jvso-rpartn.
      add_issue_info(
        exporting
          is_log          = cs_log
          is_acdoca       = is_acdoca
          iv_object_name  = 'venture partner'(vpt)
          iv_issue_type   = gc_issue_type-wrong_partner
          iv_jvso_value   = is_jvso-rpartn
          iv_acdoca_value = is_acdoca-vptnr
          iv_error_type   = gc_error_type-warning
        changing
          ct_log          = lt_log ).
    endif.
  endif.
if ms_progpars-check_recovery_indicator = abap_true or
ms_progpars-check_jva_data = abap_true.
  "Check if the recovery indicators match
  if is_acdoca-recid <> is_jvso-rrecin.
    add_issue_info(
      exporting
        is_log          = cs_log
        is_acdoca       = is_acdoca
        iv_object_name  = 'recovery indicator'(rec)
        iv_issue_type   = gc_issue_type-wrong_recovery_indicator
        iv_jvso_value   = is_jvso-rrecin
        iv_acdoca_value = is_acdoca-recid
        iv_error_type   = gc_error_type-warning
      changing
        ct_log          = lt_log ).
  elseif is_acdoca-co_buzei is not initial and
    is_acdoca-hrkft is not initial.
```

```
"Check if the C0 subkey needs to be updated
clear ls_cokey.
call function 'K_SUBKEY_INTERPRET'
  exporting
    co_hrkft = is_acdoca-hrkft
  importing
    merkmale = ls_cokey
  exceptions
    not_found = 2
    others    = 4.
if sy-subrc = 0 and
  ls_cokey is not initial.
  if is_acdoca-recid <> ls_cokey-recind.
    add_issue_info(
      exporting
        is_log          = cs_log
        is_acdoca       = is_acdoca
        iv_object_name  = 'rec. ind. in C0 subkey'(rik)
        iv_issue_type   = gc_issue_type-wrong_recovery_indicator
        iv_jvso_value   = is_jvso-rrecind
        iv_acdoca_value = ls_cokey-recind
        iv_error_type   = gc_error_type-warning
      changing
        ct_log          = lt_log ).
    endif.
  endif.
endif.
if mo_settings->ms_t8jz-pm_act = abap_true and
  ( ms_progpars-check_production_period = abap_true or
    ms_progpars-check_jva_data = abap_true ).
  "Check if the production periods are identical
  if is_jvso-prodper is initial and
    is_acdoca-prodper is not initial.
    "Do not regard as an error when PM is active and
    "the ACDOCA line has a PRODPER value while the JV line has none
    "In this case the ACDOCA value is correct while the JV value is not
  elseif ms_progpars-prodper_as_period = abap_true.
    if is_acdoca-prodper(6) <> is_jvso-prodper(6).
      data(lv_prodper_different) = abap_true.
    endif.
  elseif ms_progpars-prodper_last_eq_first = abap_true.
    if is_acdoca-prodper <> is_jvso-prodper.
      if is_acdoca-prodper(6) <> is_jvso-prodper(6).
        lv_prodper_different = abap_true.
      else.
        lv_first_day = is_jvso-prodper(6) && '01'.
        call function 'LAST_DAY_OF_MONTHS'
          exporting
            day_in          = is_jvso-prodper
          importing
            last_day_of_month = lv_last_day
          exceptions
            day_in_no_date   = 1
        endif.
      endif.
    endif.
  endif.
endif.
```

```
        others                = 2.
    if sy-subrc = 0.
        if not ( ( is_jvso-prodper = lv_first_day and
                    is_acdoca-prodper = lv_last_day ) or
                 ( is_jvso-prodper = lv_last_day and
                    is_acdoca-prodper = lv_first_day ) ).
            lv_prodper_different = abap_true.
        endif.
    endif.
endif.
else.
    if is_acdoca-prodper <> is_jvso-prodper.
        lv_prodper_different = abap_true.
    endif.
endif.
if lv_prodper_different = abap_true.
    add_issue_info(
        exporting
            is_log           = cs_log
            is_acdoca        = is_acdoca
            iv_object_name   = 'production period'(prp)
            iv_issue_type    = gc_issue_type-wrong_prodper
            iv_jvso_value    = is_jvso-prodper
            iv_acdoca_value  = is_acdoca-prodper
            iv_error_type    = gc_error_type-warning
        changing
            ct_log           = lt_log ).
endif.
endif.
if ( ms_progpars-check_co_in_co_lines = abap_true and
    lv_is_co_line = abap_true ) or
    ( ms_progpars-check_co_in_bs_lines = abap_true and
    lv_is_co_line = abap_false ).
    "Check if the cost objects match
    if ( is_jvso-rcntr is not initial or
        is_doc-is-fi_doc = abap_false and is_doc-is_co_doc = abap_false ) and
        is_acdoca-rcntr <> is_jvso-rcntr.
        add_issue_info(
            exporting
                is_log           = cs_log
                is_acdoca        = is_acdoca
                iv_object_name   = 'cost center'(cct)
                iv_issue_type    = gc_issue_type-wrong_cost_center
                iv_jvso_value    = is_jvso-rcntr
                iv_acdoca_value  = is_acdoca-rcntr
                iv_error_type    = gc_error_type-warning
                iv_is_co_line    = lv_is_co_line
            changing
                ct_log           = lt_log ).
    endif.
    if ( is_jvso-rordnr is not initial or
        is_doc-is-fi_doc = abap_false and is_doc-is_co_doc = abap_false ) and
        is_acdoca-aufnr <> is_jvso-rordnr.
```

```
add_issue_info(
  exporting
    is_log      = cs_log
    is_acdoca   = is_acdoca
    iv_object_name = 'order'(ord)
    iv_issue_type = gc_issue_type-wrong_order
    iv_jvso_value = is_jvso-rordnr
    iv_acdoca_value = is_acdoca-aufnr
    iv_error_type = gc_error_type-warning
    iv_is_co_line = lv_is_co_line
  changing
    ct_log      = lt_log ).
endif.
if is_jvso-rordnr is not initial and
  is_jvso-nplnr is initial and
  is_acdoca-vornr <> is_jvso-vornr.
add_issue_info(
  exporting
    is_log      = cs_log
    is_acdoca   = is_acdoca
    iv_object_name = 'OLC order'(olc)
    iv_issue_type = gc_issue_type-wrong_order
    iv_jvso_value = is_jvso-vornr
    iv_acdoca_value = is_acdoca-vornr
    iv_error_type = gc_error_type-warning
    iv_is_co_line = lv_is_co_line
  changing
    ct_log      = lt_log ).
endif.
if ( is_jvso-rprojk is not initial or
  is_doc-is-fi-doc = abap_false and is_doc-is-co-doc = abap_false ) and
  is_acdoca-ps_psp_pnr <> is_jvso-rprojk.
add_issue_info(
  exporting
    is_log      = cs_log
    is_acdoca   = is_acdoca
    iv_object_name = 'WBS element'(wbs)
    iv_issue_type = gc_issue_type-wrong_wbs_element
    iv_jvso_value = is_jvso-rprojk
    iv_acdoca_value = is_acdoca-ps_psp_pnr
    iv_error_type = gc_error_type-warning
    iv_is_co_line = lv_is_co_line
  changing
    ct_log      = lt_log ).
endif.
if ( is_jvso-nplnr is not initial or
  is_doc-is-fi-doc = abap_false and is_doc-is-co-doc = abap_false ).
if is_acdoca-nplnr <> is_jvso-nplnr.
add_issue_info(
  exporting
    is_log      = cs_log
    is_acdoca   = is_acdoca
    iv_object_name = 'network'(ntw)
    iv_issue_type = gc_issue_type-wrong_network
```

```

        iv_jvso_value      = is_jvso-nplnr
        iv_acdoca_value    = is_acdoca-nplnr
        iv_error_type      = gc_error_type-warning
        iv_is_co_line      = lv_is_co_line
    changing
        ct_log              = lt_log ).
endif.
if is_acdoca-nplnr_vorgn <> is_jvso-vornr.
    add_issue_info(
        exporting
            is_log           = cs_log
            is_acdoca        = is_acdoca
            iv_object_name   = 'network number'(nwn)
            iv_issue_type    = gc_issue_type-wrong_network
            iv_jvso_value    = is_jvso-vornr
            iv_acdoca_value  = is_acdoca-nplnr_vorgn
            iv_error_type    = gc_error_type-warning
            iv_is_co_line    = lv_is_co_line
    changing
        ct_log              = lt_log ).
    endif.
endif.
endif.
endif. "fields in ledger 4A
if cs_log-docln is not initial.
    data(lv_docln_jv_info) = condense( |{ cs_log-docln alpha = out }| ).
else.
    clear lv_docln_jv_info.
    read table it_jv_compr_info assigning field-symbol(<ls_jv_compr_info>)
        with key tabix_aggr = is_jvso-tabix_aggr.
    if sy-subrc = 0 and <ls_jv_compr_info> is assigned.
        data(lv_line_count) = lines( <ls_jv_compr_info>-t_docln_original ).
        if lv_line_count > 10.
            lv_docln_jv_info = `{` && lv_line_count && `` && 'lines'(lns) && `}`.
        else.
            loop at <ls_jv_compr_info>-t_docln_original into data(ls_line_info).
                if lv_docln_jv_info is not initial.
                    lv_docln_jv_info &&= `+`.
                endif.
                lv_docln_jv_info &&= condense( |{ ls_line_info-docln alpha = out }| ).
            endloop.
        endif.
    endif.
endif.
endif.
read table it_acd_compr_info assigning field-symbol(<ls_acd_compr_info>)
    with key tabix_aggr = iv_tabix_acd.
if sy-subrc <> 0 or <ls_acd_compr_info> is not assigned.
    data(lv_docln_acd_info) = condense( |{ cs_log-docln alpha = out }| ).
else.
    lv_line_count = lines( <ls_acd_compr_info>-t_docln_original ).
    if lv_line_count > 10.
        lv_docln_acd_info = `{` && lv_line_count && `` && 'lines'(lns) && `}`.
    else.
        loop at <ls_acd_compr_info>-t_docln_original into data(lv_docln_acdoca).

```

```

        if lv_docln_acd_info is not initial.
            lv_docln_acd_info &&= `+`.
        endif.
        lv_docln_acd_info &&= condense( |{ lv_docln_acdoca alpha = out }| ).
    endloop.
endif.
endif.
data(lv_line_info) = lv_docln_jv_info && ` ` && 'and ACDOCA line'(aal) && ` ` &&
lv_docln_acd_info.
if is_jvso-reffidln is not initial.
    data(lv_refln_info) = 'FI doc line'(fil) && ` ` && condense( |{
is_jvso-reffidln alpha = out }| ).
endif.
if is_jvso-refdocln is not initial and
    is_jvso-refdocln <> is_jvso-reffidln.
    if lv_refln_info is not initial.
        lv_refln_info &&= `, `.
    endif.
    lv_refln_info &&= 'Ref. doc line'(rdl) && ` ` && condense( |{ is_jvso-refdocln
alpha = out }| ).
endif.
if lv_refln_info is not initial.
    lv_line_info &&= `[ ` && lv_refln_info && ` ]`.
endif.
if lt_log[] is not initial.
    loop at lt_log assigning field-symbol(<ls_log>).
        lv_error_type = set_error_type( iv_old_error = lv_error_type
                                         iv_new_error = <ls_log>-error ).
        lv_corr_type = determine_correction_type( iv_corr_type = lv_corr_type
                                                  iv_issue_type = <ls_log>-issues(1)
                                                  iv_is_co_line = lv_is_co_line ).
    endloop.
if lv_corr_type = gc_corr_type-rev_and_repost_doc and
    ( ms_progpars-select_fi_documents = abap_true or
      ms_progpars-select_co_documents = abap_true ).
    lv_error_type = set_error_type( iv_old_error = lv_error_type
                                     iv_new_error = gc_error_type-error ).
endif.
data(lv_msg_text) = 'Issues detected for JV doc line'(isd) && ` ` &&
lv_line_info && `: `.
ls_message = value #( id          = 'GJ'
                      number      = '000'
                      type        = lv_error_type
                      message_v1   = lv_msg_text
                      message     = lv_msg_text ).
add_message_to_log(
    exporting
        is_message      = ls_message
        iv_indent_level = 0
    changing
        cs_log          = cs_log ).
add_message_to_log(
    exporting
        iv_msg_text = conv string( 'Different values'(dif) )

```

```

        iv_msg_type = space
        iv_indent_level = 1
    changing
        cs_log      = cs_log ).
loop at lt_log into data(ls_log).
    cs_log-issues &&= ls_log-issues.
    cs_log-corr_type = determine_correction_type( iv_corr_type =
cs_log-corr_type
                                                    iv_issue_type = conv char1(
ls_log-issues )
                                                    iv_is_co_line = lv_is_co_line
).
    add_message_to_log(
        exporting
            iv_msg_text = ls_log-comment
            iv_msg_type = space
            iv_indent_level = 2
        changing
            cs_log      = cs_log ).
endloop.
else.
    if ms_progpars-show_all_lines = abap_true.
        add_message_to_log(
            exporting
                iv_msg_text      = 'No issues found for JV doc line'(lok) && ` ` &&
lv_line_info
                iv_msg_type      = space
                iv_is_upd_msg    = abap_true
            changing
                cs_log = cs_log ).
    endif.
endif.
endmethod.
```

COMPARE_LINES

Description:
Instance mthd

Importing parameter

IS_DOC TYPE GTY_S_DOC
IT_JVSO_DOC TYPE JVSO_T
IT_ACDOCA_ALL TYPE ACDOCA_JV_T
IT_ACDOCA_INT TYPE ACDOCA_JV_T
IT_ACDOCA_MIG TYPE ACDOCA_JV_T
IT_CORR_LOG TYPE GTY_T_CORR_LOG
IV_RECURSION_LEVEL TYPE I OPTIONAL
VALUE(IS_LINE_ASSIGNMENTS_OLD) TYPE GTY_S_LINE_ASSIGNMENTS OPTIONAL
VALUE(IV_ERROR_COUNT_OLD) TYPE I OPTIONAL
VALUE(IV_CORR_TYPE_OLD) TYPE CHAR1 OPTIONAL
VALUE(IT_LOG_DOC_OLD) TYPE GTY_T_LOG OPTIONAL

Exporting parameter

ES_LINE_ASSIGNMENTS TYPE GTY_S_LINE_ASSIGNMENTS
EV_CORR_TYPE TYPE CHAR1
ET_LOG_DOC TYPE GTY_T_LOG

Changing parameter

```

CT_ACDOCA_DOC TYPE ACDOCA_JV_T

method compare_lines.
  data: ls_log                type gty_s_log,
        lt_acdoca_doc         type acdoca_jv_t,
        lt_acdoca             type acdoca_jv_t,
        ls_line_info          type gty_s_jv_line_info,
        ls_jv_acd_assignments type gty_s_jv_line_assignments,
        lt_acd_jv_assignments type gty_t_acd_line_assignments,
        ls_acd_jv_assignments type gty_s_acd_line_assignments,
        lt_jv_compr_info      type gty_t_jv_compr_info,
        lt_acd_compr_info     type gty_t_acd_compr_info,
        lt_acd_docln          type gty_s_jv_line_assignments-t_acd_docln,
        lt_log                type gty_t_log,
        lv_error_type_jv_line type char1,
        lv_corr_type_jv_line  type char1,
        lv_compressed          type abap_bool,
        lv_line_count          type i,
        lv_lines_missing_count type i,
        lv_lines_without_issues_count type i,
        lv_lines_to_be_corrected_count type i,
        lv_lines_to_be_reversed_count type i,
        lv_error_count        type i.
  data(lt_jvso) = it_jvso_doc.
  if iv_recursion_level > 0.
    if iv_recursion_level >= 2.
      if ms_progpars-check_co_in_bs_lines = abap_false.
        data(lv_compress_by_co) = abap_true.
      else.
        data(lv_compress_by_pc) = abap_true.
      endif.
    endif.
    if iv_recursion_level >= 3.
      lv_compress_by_pc = abap_true.
    endif.
    lv_compressed = compress_jv_doc( exporting
                                     iv_compress_by_co      = lv_compress_by_co
                                     iv_compress_by_pc      = lv_compress_by_pc
                                     it_jvso_original       = it_jvso_doc
                                   importing
                                     et_jvso                = lt_jvso
                                     et_jv_compr_info       = lt_jv_compr_info
                                ).
  endif.
  if iv_recursion_level = 0 or
    lv_compressed = abap_true.
    clear: es_line_assignments,
           ev_corr_type,
           et_log_doc.
    "Analyze the lines doc-wise
    loop at lt_jvso into data(ls_jvso).
      data(lv_tabix) = sy-tabix.
      lv_line_count += 1.

```

```

if iv_recursion_level = 0.
    update_line_stats_processed( ls_jvso ).
endif.
clear: ls_jv_acd_assignments,
       ls_acd_jv_assignments,
       lv_error_type_jv_line,
       lv_corr_type_jv_line,
       lt_acd_jv_assignments,
       lt_acd_docln,
       lt_log.
clear ls_log.
move-corresponding ls_jvso to ls_log.
ls_log-jvdoctype = is_doc-jvdoctype.
ls_log-rldnr_acd = is_doc-ledger.
if lv_compressed = abap_true.
    add_compression_message(
        exporting
            is_jvso          = ls_jvso
        importing
            ev_all_amnts_zero = data(lv_all_amnts_zero)
        changing
            cs_log            = ls_log
            ct_jv_compr_info  = lt_jv_compr_info ).
    if lv_all_amnts_zero = abap_true.
        append_log_item_for_jv_line( exporting
                                       is_log          = ls_log
                                       is_jvso          = ls_jvso
                                       it_jv_compr_info = lt_jv_compr_info
        changing
            ct_log_doc          = et_log_doc ).

        continue.
    endif.
endif.
lt_acdoca = get_acdoca_line_for_jv(
    exporting
        is_doc          = is_doc
        is_jvso          = ls_jvso
        iv_tabix         = lv_tabix
        it_acdoca_doc    = ct_acdoca_doc
        it_acdoca_all    = it_acdoca_all
        it_acdoca_int    = it_acdoca_int
        it_acdoca_mig    = it_acdoca_mig
        it_corr_log      = it_corr_log
        it_jv_compr_info = lt_jv_compr_info
        iv_compress_by_pc = lv_compress_by_pc
        is_line_assignments = es_line_assignments
    importing
        et_acd_compr_info = lt_acd_compr_info
    changing
        cs_log            = ls_log ).
if ls_log-error = gc_error_type-error.
    lv_error_count += 1.
    append_log_item_for_jv_line( exporting
                                is_log          = ls_log

```

```

                is_jvso      = ls_jvso
                it_jv_compr_info = lt_jv_compr_info
            changing
                ct_log_doc      = et_log_doc ).
    if ls_log-corr_type > ev_corr_type.
        ev_corr_type = ls_log-corr_type.
    endif.
elseif lines( lt_acdoca ) = 0.
    lv_lines_missing_count += 1.
    append_log_item_for_jv_line( exporting
                                is_log      = ls_log
                                is_jvso     = ls_jvso
                                it_jv_compr_info = lt_jv_compr_info
                                changing
                                ct_log_doc      = et_log_doc ).

    if ls_log-corr_type > ev_corr_type.
        ev_corr_type = ls_log-corr_type.
    endif.
else.
    loop at lt_acdoca into data(ls_acdoca).
        data(lv_tabix_acd) = sy-tabix.
        if ct_acdoca_doc[] is initial.
            read table lt_acdoca_doc transporting no fields
                                with key docln = ls_acdoca-docln
                                binary search.

            if sy-subrc <> 0.
                insert ls_acdoca into lt_acdoca_doc index sy-tabix.
            endif.
        endif.
        data(ls_log_item) = ls_log.
        clear: ls_log_item-comment,
               ls_log_item-messages[].
        ls_log_item-rldnr_acd = ls_acdoca-rldnr.
        ls_log_item-belnr_acd = ls_acdoca-belnr.
        ls_log_item-docln_acd = ls_acdoca-docln.
        ls_log_item-gjahr_acd = ls_acdoca-gjahr.
        compare_field_values(
            exporting
                is_doc      = is_doc
                is_jvso     = ls_jvso
                is_acdoca   = ls_acdoca
                iv_tabix_acd = lv_tabix_acd
                iv_jvdoctype = is_doc-jvdoctype
                it_jv_compr_info = lt_jv_compr_info
                it_acd_compr_info = lt_acd_compr_info
            changing
                cs_log      = ls_log_item ).
        lv_error_type_jv_line = set_error_type(
                                iv_old_error = lv_error_type_jv_line
                                iv_new_error = ls_log_item-error ).
        if ls_log_item-corr_type > lv_corr_type_jv_line.
            lv_corr_type_jv_line = ls_log_item-corr_type.
        endif.
        if ls_log_item-corr_type > ev_corr_type.

```

```

        ev_corr_type = ls_log_item-corr_type.
    endif.
    append ls_acdoca-docln to lt_acd_docln.
    append ls_log_item to lt_log.
    ls_acd_jv_assignments-docln = ls_acdoca-docln.
    if ls_jvso-docln is not initial.
        ls_line_info-docnr = ls_jvso-docnr.
        ls_line_info-docln = ls_jvso-docln.
        append ls_line_info to ls_acd_jv_assignments-t_jv_docln.
    else.
        read table lt_jv_compr_info assigning field-symbol(<ls_jv_compr_info>)
                                with key tabix_aggr = ls_jvso-tabix_aggr.

        if sy-subrc = 0.
            append lines of <ls_jv_compr_info>-t_docln_original to
ls_acd_jv_assignments-t_jv_docln.
        endif.
    endif.
    append ls_acd_jv_assignments to lt_acd_jv_assignments.
endloop.
case lv_error_type_jv_line.
    when gc_error_type-error or
        gc_error_type-abend or
        gc_error_type-shortdump.
        lv_error_count += 1.
    when gc_error_type-warning.
        case ls_log_item-issues.
            when gc_issue_type-line_missing.
                lv_lines_missing_count += 1.
            when others.
                lv_lines_to_be_corrected_count += 1.
        endcase.
    when others.
        lv_lines_without_issues_count += 1.
endcase.
loop at lt_log into ls_log_item.
    data(lv_add_messages) = abap_false.
    if lines( ls_log_item-messages ) > 0 and
        lines( ls_log-messages ) > 0.
        "Check if the same message was already added to ls_log-messages
        read table ls_log-messages transporting no fields
                                with key message = ls_log_item-messages[ 1
]-message.
        if sy-subrc <> 0.
            lv_add_messages = abap_true.
        endif.
    else.
        lv_add_messages = abap_true.
    endif.
    if lv_add_messages = abap_true.
        append lines of ls_log_item-messages to ls_log-messages.
    endif.
    ls_log-issues = ls_log_item-issues.
    if ls_log_item-corr_type > ls_log-corr_type.
        ls_log-corr_type = ls_log_item-corr_type.

```

```

endif.
if ls_log_item-corr_type = gc_corr_type-correction_not_required.
  if ls_log-comment cs ls_log_item-comment.
    continue.
  endif.
  if ls_log-comment is not initial.
    ls_log-comment &&= ` - `.
  endif.
  ls_log-comment &&= ls_log_item-comment.
endif.
endloop.
append_log_item_for_jv_line( exporting
                             is_log          = ls_log
                             is_jvso         = ls_jvso
                             it_jv_compr_info = lt_jv_compr_info
                             changing
                             ct_log_doc       = et_log_doc ).
"If no critical error occurred during the line check, add the assignments
if lv_corr_type_jv_line < gc_corr_type-rev_and_repost_line.
  ls_jv_acd_assignments-t_acd_docln[] = lt_acd_docln[].
  if ls_jvso-docln is not initial.
    ls_jv_acd_assignments-docnr = ls_jvso-docnr.
    ls_jv_acd_assignments-docln = ls_jvso-docln.
    append ls_jv_acd_assignments to es_line_assignments-jv_acd.
  else.
    read table lt_jv_compr_info assigning <ls_jv_compr_info>
                                with key tabix_aggr = ls_jvso-tabix_aggr.
    if sy-subrc = 0 and <ls_jv_compr_info> is assigned.
      loop at <ls_jv_compr_info>-t_docln_original into data(lv_line_info).
        ls_jv_acd_assignments-docnr = lv_line_info-docnr.
        ls_jv_acd_assignments-docln = lv_line_info-docln.
        append ls_jv_acd_assignments to es_line_assignments-jv_acd.
      endloop.
    endif.
  endif.
  loop at lt_acd_jv_assignments into ls_acd_jv_assignments.
    read table es_line_assignments-acd_jv transporting no fields
                                with key docln =
ls_acd_jv_assignments-docln
                                binary search.
    insert ls_acd_jv_assignments into es_line_assignments-acd_jv
                                index sy-tabix.
  endloop.
endif.
endloop.
if ct_acdoca_doc[] is initial.
  ct_acdoca_doc[] = lt_acdoca_doc[].
endif.
if lines( es_line_assignments-jv_acd ) = lines( lt_jvso ).
  es_line_assignments-all_lines_assigned = abap_true.
endif.
sort es_line_assignments-jv_acd by docnr docln.
if lv_error_count > 0.

```

```

    data(lv_corr_type) = gc_corr_type-correction_not_possible.
elseif lv_lines_missing_count = lv_line_count.
    if is_doc-jvdoctype = gc_jvdoctype-fi_doc or
       is_doc-jvdoctype = gc_jvdoctype-co_doc.
        lv_corr_type = gc_corr_type-correction_not_possible.
    else.
        lv_corr_type = gc_corr_type-post_new_doc.
    endif.
elseif lv_lines_to_be_reversed_count > 0 or
       ( lv_lines_missing_count > 0 and
         lv_lines_missing_count < lv_line_count ).
    if is_doc-jvdoctype = gc_jvdoctype-fi_doc or
       is_doc-jvdoctype = gc_jvdoctype-co_doc.
        lv_corr_type = gc_corr_type-correction_not_possible.
    else.
        if is_doc-posting_mode = gc_jvposttype-combined.
            lv_corr_type = gc_corr_type-rev_and_repost_doc.
        else.
            clear ev_corr_type.
            lv_corr_type = gc_corr_type-update_line.
        endif.
    endif.
endif.
if lv_corr_type > ev_corr_type.
    ev_corr_type = lv_corr_type.
endif.
"Check if the new results is better than the old one. If the old
"one is better, use the old results for further processing
if iv_recursion_level > 0.
    if it_log_doc_old[] is not initial and
       iv_error_count_old < lv_error_count.
        ev_corr_type = iv_corr_type_old.
        et_log_doc[] = it_log_doc_old[].
        lv_error_count = iv_error_count_old.
        es_line_assignments = is_line_assignments_old.
    endif.
endif.
endif.
if ms_progpars-heuristic_line_assignment = abap_false or
   ( is_doc-activ <> gc_jvlinetype-fi_doc and
     is_doc-activ <> gc_jvlinetype-fi_doc_mm ) or
   ev_corr_type <> gc_corr_type-correction_not_possible or
   lv_compress_by_pc = abap_true or
   iv_recursion_level >= 3.
    return.
endif.
compare_lines(
    exporting
        is_doc                = is_doc
        it_jvso_doc           = it_jvso_doc
        it_acdoca_all         = it_acdoca_all
        it_acdoca_int         = it_acdoca_int
        it_acdoca_mig         = it_acdoca_mig
        it_corr_log           = it_corr_log

```

```

        iv_recursion_level      = iv_recursion_level + 1
        iv_error_count_old      = lv_error_count
        is_line_assignments_old = es_line_assignments
        iv_corr_type_old        = ev_corr_type
        it_log_doc_old          = et_log_doc
importing
        es_line_assignments      = es_line_assignments
        ev_corr_type              = ev_corr_type
        et_log_doc                = et_log_doc
changing
        ct_acdoca_doc            = ct_acdoca_doc ).
endmethod.
```

COMPRESS_JV_DOC

Description:
Instance mthd

Importing parameter

```
IT_JVSO_ORIGINAL TYPE JVSO_T
IV_COMPRESS_BY_CO TYPE ABAP_BOOL
IV_COMPRESS_BY_PC TYPE ABAP_BOOL
```

Exporting parameter

```
ET_JVSO TYPE JVSO_T
ET_JV_COMPR_INFO TYPE GTY_T_JV_COMPR_INFO
```

Returning parameter

```
VALUE(RV_COMPRESSED) TYPE ABAP_BOOL
```

```
method compress_jv_doc.
  data: lt_jvso_aggr      type jvso_t,
        ls_jv_compr_info type gty_s_jv_compr_info,
        lv_tabix_hit      type sy-tabix,
        ls_line_info      type gty_s_jv_line_info.
  clear et_jv_compr_info.
  et_jvso[] = it_jvso_original[].
  "Check if the document has different ORECID values, tax IDs, cost objects
  "or profit centers per FI doc line
  sort et_jvso by reffidoc reffidln rjvnam regrou rpartn rrecin bilid.
  loop at et_jvso into data(ls_jvso).
    ls_line_info-docnr = ls_jvso-docnr.
    ls_line_info-docln = ls_jvso-docln.
    clear: ls_jvso-docnr,
           ls_jvso-docln,
           ls_jvso-drcrk,
           ls_jvso-rbusa, "RBusa is not updated reliably in JVS01
           ls_jvso-tabix_aggr,
           ls_jvso-cpudt,
           ls_jvso-cputm,
           ls_jvso-sgtxt.
    if ls_jvso-orecid is not initial.
      data(lv_orecid_omitted) = abap_true.
    endif.
    clear: ls_jvso-orecid.
    if ls_jvso-mwskz is not initial.
      data(lv_tax_id_omitted) = abap_true.
```

```
endif.
clear: ls_jvso-mwskz.
if ls_jvso-ebeln is not initial or
   ls_jvso-ebelp is not initial.
   data(lv_po_omitted) = abap_true.
endif.
clear: ls_jvso-ebeln,
       ls_jvso-ebelp.
if iv_compress_by_co = abap_true.
   if ls_jvso-rcntr is not initial or
      ls_jvso-rordnr is not initial or
      ls_jvso-rprojk is not initial or
      ls_jvso-nplnr is not initial or
      ls_jvso-aufpl is not initial or
      ls_jvso-aplzl is not initial or
      ls_jvso-vornr is not initial.
      data(lv_co_obj_omitted) = abap_true.
   endif.
   clear: ls_jvso-rcntr,
          ls_jvso-rordnr,
          ls_jvso-rprojk,
          ls_jvso-nplnr,
          ls_jvso-aufpl,
          ls_jvso-aplzl,
          ls_jvso-vornr.
endif.
if iv_compress_by_pc = abap_true.
   if ls_jvso-rprctr is not initial.
      data(lv_pc_omitted) = abap_true.
   endif.
   clear: ls_jvso-rprctr.
endif.
collect ls_jvso into lt_jvso_aggr.
data(lv_tabix_aggr) = sy-tabix.
"Save links between the aggregated item and the original items
"First check if entry already exist
read table et_jv_compr_info assigning field-symbol(<ls_jv_compr_info>)
                                with key tabix_aggr = lv_tabix_aggr.

if sy-subrc = 0 and
   <ls_jv_compr_info> is assigned.
   <ls_jv_compr_info>-count += 1.
   append ls_line_info to <ls_jv_compr_info>-t_docln_original.
else.
   clear ls_jv_compr_info.
   ls_jv_compr_info-tabix_aggr = lv_tabix_aggr.
   ls_jv_compr_info-count = 1.
   append ls_line_info to ls_jv_compr_info-t_docln_original.
   ls_jv_compr_info-reffidoc = ls_jvso-reffidoc.
   ls_jv_compr_info-reffidln = ls_jvso-reffidln.
   if lv_orecid_omitted = abap_true.
      ls_jv_compr_info-orecid_omitted = abap_true.
   endif.
   if lv_tax_id_omitted = abap_true.
      ls_jv_compr_info-tax_id_omitted = abap_true.
```

```
endif.
if lv_po_omitted = abap_true.
  ls_jv_compr_info-po_omitted = abap_true.
endif.
if lv_co_obj_omitted = abap_true.
  ls_jv_compr_info-co_obj_omitted = abap_true.
endif.
if lv_pc_omitted = abap_true.
  ls_jv_compr_info-pc_omitted = abap_true.
endif.
append ls_jv_compr_info to et_jv_compr_info.
endif.
endloop.
sort et_jvso by docnr docln.
loop at et_jv_compr_info into ls_jv_compr_info
  where count > 1.
  rv_compressed = abap_true.
  read table lt_jvso_aggr into data(ls_jvso_aggr)
    index ls_jv_compr_info-tabix_aggr.

  if sy-subrc = 0.
    loop at ls_jv_compr_info-t_docln_original into ls_line_info.
      delete et_jvso where docnr = ls_line_info-docnr
        and docln = ls_line_info-docln.
    endloop.
    ls_jvso_aggr-tabix_aggr = ls_jv_compr_info-tabix_aggr.
    append ls_jvso_aggr to et_jvso.
  endif.
endloop.
delete et_jv_compr_info where count <= 1.
"Sort the returned table
sort et_jvso by docnr docln.
endmethod.
```

CORRECT_ACDOCA_LINE

Description:
Instance mthd

Importing parameter

```
IS_JVSO TYPE JVSO
IS_ACDOCA TYPE ACDOCA_JV
IV_LEDGER TYPE RLDNR
IV_IS_REVERSAL_LINE TYPE ABAP_BOOL
```

Changing parameter

```
CS_LOG TYPE GTY_S_LOG
```

```
method correct_acdoca_line.
  data: lv_updated      type abap_bool,
        lv_belnr        type belnr_d,
        lv_gjahr         type gjahr,
        lv_success_text type string,
        lv_error_text    type string,
        lv_message_text  type string,
        lo_messages      type ref to if_jva_message_collection.
  if ms_progpars-testrun = abap_true.
```

```
lv_success_text = 'Line update tested successfully'(lst).
lv_error_text = 'ERROR when testing the line update'(let).
else.
lv_success_text = 'Line updated successfully'(lsu).
lv_error_text = 'ERROR when updating line'(ler).
endif.
data(lo_post_acdoca) = cl_jva_mig_post_acdoca=>get_instance( ).
"The ACDOCA line is missing: Post it
case cs_log-corr_type.
when gc_corr_type-post_missing_line.
lo_post_acdoca->post_missing_acdoca_line(
exporting
is_jvso          = is_jvso
iv_ledger        = iv_ledger
iv_is_reversal_line = iv_is_reversal_line
iv_budat         = ms_progpars-posting_date
iv_test         = ms_progpars-testrun
changing
cv_updated       = lv_updated
cv_belnr         = lv_belnr
cv_gjahr         = lv_gjahr
co_messages      = lo_messages ).
if lo_messages->has_critical_messages( ) = abap_true.
lv_message_text = lv_error_text.
else.
lv_message_text = lv_success_text.
endif.
update_log_with_posting_result(
exporting
iv_updated       = lv_updated
iv_belnr         = lv_belnr
iv_gjahr         = lv_gjahr
iv_docln         = '2'
iv_comment       = lv_message_text
io_messages      = lo_messages
changing
cs_log           = cs_log ).
when gc_corr_type-rev_and_repost_line.
lo_post_acdoca->rev_and_rep_acdoca_line(
exporting
is_jvso          = is_jvso
iv_issues        = cs_log-issues
is_acdoca        = is_acdoca
iv_ledger        = iv_ledger
iv_budat         = ms_progpars-posting_date
iv_test         = ms_progpars-testrun
changing
cv_updated       = lv_updated
cv_belnr         = lv_belnr
cv_gjahr         = lv_gjahr
co_messages      = lo_messages ).
if lv_updated = abap_true.
cs_log-gjahr_rev = lv_gjahr.
cs_log-belnr_rev = lv_belnr.
```

```
        cs_log-docln_rev = 1.
    endif.
    if lo_messages->has_critical_messages( ) = abap_true.
        lv_message_text = lv_error_text.
    else.
        lv_message_text = lv_success_text.
    endif.
    update_log_with_posting_result(
        exporting
            iv_updated   = lv_updated
            iv_belnr     = lv_belnr
            iv_gjahr     = lv_gjahr
            iv_docln     = '2'
            iv_comment   = lv_message_text
            io_messages  = lo_messages
        changing
            cs_log       = cs_log ).
    when gc_corr_type-update_line.
        lo_post_acdoca->update_acdoca_line(
            exporting
                is_jvso   = is_jvso
                iv_issues = cs_log-issues
                is_acdoca = is_acdoca
                iv_ledger = iv_ledger
                iv_test   = ms_progpars-testrun
            changing
                cv_updated = lv_updated
                co_messages = lo_messages ).
    if lo_messages->has_critical_messages( ) = abap_true.
        lv_message_text = lv_error_text.
    else.
        lv_message_text = lv_success_text.
    endif.
    update_log_with_posting_result(
        exporting
            iv_updated   = lv_updated
            iv_belnr     = is_acdoca-belnr
            iv_gjahr     = is_acdoca-gjahr
            iv_docln     = is_acdoca-docln
            iv_comment   = lv_message_text
            io_messages  = lo_messages
        changing
            cs_log       = cs_log ).
endcase.
if ms_progpars-testrun <> abap_true and
   cs_log-updated = abap_true.
    update_log_db_table_prep(
        changing
            cs_log = cs_log ).
endif.
endmethod.
```

CORRECT_ACDOCA_LINES

Description:
Instance mthd

Importing parameter

IT_JVSO_DOC TYPE JVSO_T
IT_ACDOCA_DOC TYPE ACDOCA_JV_T
IS_LINE_ASSIGNMENTS TYPE GTY_S_LINE_ASSIGNMENTS
IV_LEDGER TYPE RLDNR
IV_JVDOCTYPE TYPE CHAR2
IV_POSTING_MODE TYPE CHAR1

Changing parameter

CT_LOG_DOC TYPE GTY_T_LOG

```
method correct_acdoca_lines.  
  data: ls_jvso          type jvso,  
        ls_acdoca        type acdoca_jv,  
        lv_doc_stats_updated type abap_bool,  
        lv_docln_acd      type acdoca-docln,  
        lt_acd_docln      type hashed table of acdoca-docln  
                                with unique key table_line.  
  loop at ct_log_doc assigning field-symbol(<ls_log>).  
    if <ls_log>-issues is initial or  
      <ls_log>-error = gc_error_type-error.  
      add_item_info_for_line( iv_update = abap_false  
                             is_log    = <ls_log> ).  
      continue.  
    endif.  
    if ms_progpars-analysis_only = abap_true.  
      add_message_to_log(  
        exporting  
          iv_msg_text    = space  
          iv_msg_type    = gc_error_type-warning  
          iv_is_upd_msg  = abap_true  
        changing  
          cs_log = <ls_log> ).  
      add_item_info_for_line( iv_update = abap_false  
                             is_log    = <ls_log> ).  
      continue.  
    endif.  
    clear ls_jvso.  
    read table it_jvso_doc into ls_jvso  
                                with key docnr = <ls_log>-docnr  
                                docln = <ls_log>-docln  
                                binary search.  
    if ls_jvso is initial.  
      add_message_to_log(  
        exporting  
          iv_msg_text = conv string( 'Technical error: JVS01 line not found in  
internal table IT_JVSO_DOC'(tej) )  
          iv_msg_type = gc_error_type-error  
        changing  
          cs_log = <ls_log> ).  
    endif.  
  endmethod
```

```
add_item_info_for_line( iv_update = abap_false
                        is_log     = <ls_log> ).

continue.
endif.
"Get the ACDDOCA lines to which the JV doc line was assigned
read table is_line_assignments-jv_acd into data(ls_line_assignments)
                        with key docnr = ls_jvso-docnr
                        docln = ls_jvso-docln
                        binary search.

if sy-subrc = 0.
    data(lt_docln) = ls_line_assignments-t_acd_docln.
else.
    clear lt_docln.
    append <ls_log>-docln to lt_docln.
endif.
loop at lt_docln into data(lv_docln).
    lv_docln_acd = |{ lv_docln alpha = in }|.
    "As multiple assignments are possible, check if the ACDOCA line
    "has already been processed before
    read table lt_acd_docln transporting no fields
                        with key table_line = lv_docln_acd.
    if sy-subrc = 0.
        loop at ct_log_doc into data(ls_log_previous)
                        where docnr      = <ls_log>-docnr
                        and docln       < <ls_log>-docln
                        and docln_acd = <ls_log>-docln_acd.
            ls_log_previous-docln = <ls_log>-docln.
            add_item_info_for_line( iv_update = abap_true
                                    is_log     = ls_log_previous ).
            update_line_statistics_updates( ls_log_previous ).
            exit.
        endloop.
        if sy-subrc <> 0.
            add_item_info_for_line( iv_update = abap_true
                                    is_log     = <ls_log> ).
            update_line_statistics_updates( <ls_log> ).
        endif.
        continue.
    else.
        collect lv_docln_acd into lt_acd_docln.
    endif.
clear ls_acdoca.
read table it_acdoca_doc into ls_acdoca
                        with key rldnr = <ls_log>-rldnr_acd
                        gjahr = <ls_log>-gjahr_acd
                        belnr = <ls_log>-belnr_acd
                        docln = lv_docln_acd.

correct_acdoca_line(
    exporting
        is_jvso      = ls_jvso
        is_acdoca     = ls_acdoca
        iv_ledger     = iv_ledger
        iv_is_reversal_line = ls_acdoca-rev_line
    changing
```

```

        cs_log          = <ls_log> ).
update_line_statistics_updates( <ls_log> ).
if lv_doc_stats_updated = abap_false.
    lv_doc_stats_updated = abap_true.
    update_doc_statistics_updates( <ls_log> ).
endif.
add_item_info_for_line( iv_update = abap_true
                        is_log    = <ls_log> ).

endloop.
endloop.
endmethod.

```

CALCULATE_TASK_PACKAGES

Description:

Instance mthd

Importing parameter

IS_PROGPARS TYPE IF_JVA_MIG_CORRECT_ACDOCA~GTY_PROGRAM_PARAMETERS

Returning parameter

VALUE(RT_PACKAGES) TYPE IF_JVA_MIG_CORRECT_ACDOCA~GTY_T_TASK_PACKAGES

```

method calculate_task_packages.
data: lv_package_size_minimum type i value 10000,
      lt_docs_per_period      type gty_t_docs_per_period,
      ls_package              like line of rt_packages,
      ls_periods              like line of ls_package-t_periods,
      lv_cond                 type char255,
      lv_new_total            type i.
count_jv_documents_by_period(
  exporting
    is_progpar      = is_progpar
  changing
    ct_docs_per_period = lt_docs_per_period ).
if lt_docs_per_period[] is initial.
  return.
endif.
sort lt_docs_per_period by rbukrs ryear poper.
loop at lt_docs_per_period into data(ls_docs_per_period).
  if ls_package-doc_count > 0.
    lv_new_total = ls_package-doc_count + ls_docs_per_period-doc_count.
    if ls_package-doc_count > ms_progpar-number_of_docs_per_task or
      ( ls_package-doc_count >= lv_package_size_minimum and
        lv_new_total > ms_progpar-number_of_docs_per_task ).
      append ls_package to rt_packages.
      clear ls_package.
    endif.
  endif.
  if ls_docs_per_period-doc_count > 0.
    ls_package-doc_count += ls_docs_per_period-doc_count.
    read table ls_package-t_periods assigning field-symbol(<ls_periods>)
      with key ryear = ls_docs_per_period-ryear
      binary search.

    data(lv_tabix_periods) = sy-tabix.
    if sy-subrc = 0 and <ls_periods> is assigned.

```

```
        append ls_docs_per_period-poper to <ls_periods>-t_poper.
    else.
        ls_periods-ryear = ls_docs_per_period-ryear.
        ls_periods-t_poper = value #( ( ls_docs_per_period-poper ) ).
        insert ls_periods into ls_package-t_periods index lv_tabix_periods.
    endif.
endif.
endloop.
if ls_package-doc_count > 0.
    append ls_package to rt_packages.
endif.
endmethod.
```

COUNT_JV_DOCUMENTS_BY_PERIOD

Description:
Instance mthd

Importing parameter

IS_PROGPARS TYPE IF_JVA_MIG_CORRECT_ACDOCA~GTY_PROGRAM_PARAMETERS

Changing parameter

CT_DOCS_PER_PERIOD TYPE GTY_T_DOCS_PER_PERIOD

```
method count_jv_documents_by_period.
    data: ct_docs_per_period_op like ct_docs_per_period.
    if is_progpars-select_fi_documents = abap_true or
       is_progpars-select_co_documents = abap_true.
        data(lt_ref_cond) = make_reference_condition( is_progpars ).
        select rbukrs, ryear, poper, count( distinct docnr ) as doc_count
            from jvso1
            where rldnr      = '4A'
               and rbukrs   = @is_progpars-bukrs
               and ryear    in @is_progpars-r_ryear
               and poper    in @is_progpars-r_poper
               and (lt_ref_cond)
               and budat    in @is_progpars-r_budat
               and bldat    in @is_progpars-r_bldat
               and racct    in @is_progpars-r_racct
               and rjvnam   in @is_progpars-r_rjvnam
               and regrou   in @is_progpars-r_regrou
               and rrecin   in @is_progpars-r_rrecin
               and bilid    in @is_progpars-r_bilid
               and prodper  in @is_progpars-r_prodper
               and rcntr    in @is_progpars-r_rcntr
               and rordnr   in @is_progpars-r_rordnr
               and rprojk   in @is_progpars-r_rprojk
               and nplnr    in @is_progpars-r_nplnr
               and aufpl    in @is_progpars-r_aufpl
               and aplzl    in @is_progpars-r_aplzl
        group by rbukrs, ryear, poper
        order by rbukrs, ryear, poper
        into corresponding fields of table @ct_docs_per_period. "#EC CI_NOFIELD
    else.
        if is_progpars-r_activ_me[] is not initial.
            select rbukrs, ryear, poper, count( distinct docnr ) as doc_count
```

```

        from jvso1
        where rldnr = '4A'
           and rbukrs = @is_progpars-bukrs
           and ryear in @is_progpars-r_ryear
           and poper in @is_progpars-r_poper
           and docnr in @is_progpars-r_docnr
           and activ in @is_progpars-r_activ_me
        group by rbukrs, ryear, poper
        order by rbukrs, ryear, poper
        into corresponding fields of table @ct_docs_per_period. "#EC
CI_NOFIELD
endif.
if is_progpars-r_activ_op[] is not initial.
    select rbukrs, ryear, poper, count( distinct docnr ) as doc_count
    from jvso2
    where rldnr = '4B'
       and rbukrs = @is_progpars-bukrs
       and ryear in @is_progpars-r_ryear
       and poper in @is_progpars-r_poper
       and docnr in @is_progpars-r_docnr
       and activ in @is_progpars-r_activ_op
    group by rbukrs, ryear, poper
    order by rbukrs, ryear, poper
    into corresponding fields of table @ct_docs_per_period_op. "#EC
CI_NOFIELD
    if ct_docs_per_period_op[] is not initial.
        if ct_docs_per_period[] is initial.
            ct_docs_per_period[] = ct_docs_per_period_op[].
        else.
            loop at ct_docs_per_period_op assigning
field-symbol(<ls_docs_per_period_op>).
                read table ct_docs_per_period assigning
field-symbol(<ls_docs_per_period>)
                    with key rbukrs =
<ls_docs_per_period_op>-rbukrs
                                ryear =
<ls_docs_per_period_op>-ryear
                                poper =
<ls_docs_per_period_op>-popper
                                binary search.
                    data(lv_tabix) = sy-tabix.
                    if sy-subrc = 0 and <ls_docs_per_period> is assigned.
                        <ls_docs_per_period>-doc_count += <ls_docs_per_period_op>-doc_count.
                    else.
                        insert <ls_docs_per_period_op> into ct_docs_per_period index
lv_tabix.
                    endif.
                endloop.
            endif.
        endif.
    endif.
endif.
endmethod.
"count_jv_documents_by_period

```

COUNT_JV_DOCUMENTS_TOTAL

Description:
Instance mthd

Importing parameter

IS_PROGPARS TYPE IF_JVA_MIG_CORRECT_ACDOCA~GTY_PROGRAM_PARAMETERS

Returning parameter

VALUE(RV_COUNT) TYPE I

```
method count_jv_documents_total.  
  if is_progpars-select_fi_documents = abap_true or  
    is_progpars-select_co_documents = abap_true.  
    data(lt_ref_cond) = make_reference_condition( is_progpars ).  
    select count( distinct docnr ) as doc_count  
      from jvso1  
      where rldnr      = '4A'  
        and rbukrs    in @is_progpars-r_bukrs  
        and ryear     in @is_progpars-r_ryear  
        and poper     in @is_progpars-r_poper  
        and (lt_ref_cond)  
        and budat     in @is_progpars-r_budat  
        and bldat     in @is_progpars-r_bldat  
        and racct     in @is_progpars-r_racct  
        and rjvnam    in @is_progpars-r_rjvnam  
        and regrou    in @is_progpars-r_regrou  
        and rrecin    in @is_progpars-r_rrecin  
        and bilid     in @is_progpars-r_bilid  
        and prodper   in @is_progpars-r_prodper  
        and rcntr     in @is_progpars-r_rcntr  
        and rordnr    in @is_progpars-r_rordnr  
        and rprojk    in @is_progpars-r_rprojk  
        and nplnr     in @is_progpars-r_nplnr  
        and aufpl     in @is_progpars-r_aufpl  
        and aplzl     in @is_progpars-r_aplzl  
    into @rv_count.                                "#EC CI_NOFIELD  
  else.  
    if is_progpars-r_activ_me[] is not initial.  
      select count( distinct docnr ) as doc_count  
        from jvso1  
        where rldnr = '4A'  
          and rbukrs in @is_progpars-r_bukrs  
          and ryear  in @is_progpars-r_ryear  
          and poper  in @is_progpars-r_poper  
          and docnr  in @is_progpars-r_docnr  
          and activ  in @is_progpars-r_activ_me  
      into @rv_count.                                "#EC CI_NOFIELD  
    endif.  
    if is_progpars-r_activ_op[] is not initial.  
      select count( distinct docnr ) as doc_count  
        from jvso2  
        where rldnr = '4B'  
          and rbukrs in @is_progpars-r_bukrs  
          and ryear  in @is_progpars-r_ryear
```

```
        and poper in @is_progpars-r_poper
        and docnr in @is_progpars-r_docnr
        and activ in @is_progpars-r_activ_op
        into @data(lv_count_op).          "#EC CI_NOFIELD
    rv_count += lv_count_op.
endif.
endif.
endmethod.
```

DETERMINE_CORRECTION_TYPE

Description:

Instance mthd

Importing parameter

```
IV_CORR_TYPE TYPE CHAR1
IV_ISSUE_TYPE TYPE CHAR1
IV_IS_CO_LINE TYPE ABAP_BOOL OPTIONAL
```

Returning parameter

```
VALUE(RV_CORR_TYPE) TYPE CHAR1
```

```
method determine_correction_type.
    data: lv_corr_type type char1.
    case iv_issue_type.
        when gc_issue_type-line_missing.
            if ms_progpars-select-fi_documents = abap_true or
               ms_progpars-select-co_documents = abap_true.
                lv_corr_type = gc_corr_type-correction_not_possible.
            else.
                lv_corr_type = gc_corr_type-post_new_doc.
            endif.
        when gc_issue_type-balancing_line_issue.
            lv_corr_type = gc_corr_type-correction_not_required.
        when gc_issue_type-wrong_amount or
               gc_issue_type-wrong_amount_sign.
            lv_corr_type = gc_corr_type-rev_and_repost_doc.
        when gc_issue_type-wrong_jva_data.
            if ms_progpars-repost_venture = abap_true or
               ms_progpars-repost_equity_group = abap_true or
               ms_progpars-repost_venture_partner = abap_true or
               ms_progpars-repost_recovery_indicator = abap_true or
               ms_progpars-repost_billing_indicator = abap_true or
               ms_progpars-repost_production_period = abap_true.
                lv_corr_type = gc_corr_type-rev_and_repost_line.
            elseif ms_progpars-update_venture = abap_true or
                   ms_progpars-update_equity_group = abap_true or
                   ms_progpars-update_venture_partner = abap_true or
                   ms_progpars-update_recovery_indicator = abap_true or
                   ms_progpars-update_billing_indicator = abap_true or
                   ms_progpars-update_production_period = abap_true or
                   ms_progpars-update_existing_lines = abap_true.
                lv_corr_type = gc_corr_type-update_line.
            endif.
        when gc_issue_type-wrong_venture.
            if ms_progpars-repost_venture = abap_true.
```

```
    lv_corr_type = gc_corr_type-rev_and_repost_line.
elseif ms_progpars-update_venture = abap_true or
       ms_progpars-update_existing_lines = abap_true.
    lv_corr_type = gc_corr_type-update_line.
endif.
when gc_issue_type-wrong_equity_group.
    if ms_progpars-repost_equity_group = abap_true.
        lv_corr_type = gc_corr_type-rev_and_repost_line.
    elseif ms_progpars-update_equity_group = abap_true or
           ms_progpars-update_existing_lines = abap_true.
        lv_corr_type = gc_corr_type-update_line.
    endif.
when gc_issue_type-wrong_partner.
    if ms_progpars-repost_venture_partner = abap_true.
        lv_corr_type = gc_corr_type-rev_and_repost_line.
    elseif ms_progpars-update_venture_partner = abap_true or
           ms_progpars-update_existing_lines = abap_true.
        lv_corr_type = gc_corr_type-update_line.
    endif.
when gc_issue_type-wrong_recovery_indicator.
    if ms_progpars-repost_recovery_indicator = abap_true.
        lv_corr_type = gc_corr_type-rev_and_repost_line.
    elseif ms_progpars-update_recovery_indicator = abap_true or
           ms_progpars-update_existing_lines = abap_true.
        lv_corr_type = gc_corr_type-update_line.
    endif.
when gc_issue_type-wrong_billing_indicator.
    if ms_progpars-repost_billing_indicator = abap_true.
        lv_corr_type = gc_corr_type-rev_and_repost_line.
    elseif ms_progpars-update_billing_indicator = abap_true or
           ms_progpars-update_existing_lines = abap_true.
        lv_corr_type = gc_corr_type-update_line.
    endif.
when gc_issue_type-wrong_prodper.
    if ms_progpars-repost_production_period = abap_true.
        lv_corr_type = gc_corr_type-rev_and_repost_line.
    elseif ms_progpars-update_production_period = abap_true or
           ms_progpars-update_existing_lines = abap_true.
        lv_corr_type = gc_corr_type-update_line.
    endif.
when gc_issue_type-wrong_cost_center or
       gc_issue_type-wrong_order or
       gc_issue_type-wrong_wbs_element or
       gc_issue_type-wrong_network or
       gc_issue_type-wrong_co_data.
    if iv_is_co_line = abap_true.
        if ms_progpars-repost_co_in_co_lines = abap_true.
            lv_corr_type = gc_corr_type-rev_and_repost_line.
        elseif ms_progpars-update_co_in_co_lines = abap_true or
               ms_progpars-update_existing_lines = abap_true.
            lv_corr_type = gc_corr_type-update_line.
        endif.
    else.
        if ms_progpars-repost_co_in_bs_lines = abap_true.
```

```

        lv_corr_type = gc_corr_type-rev_and_repost_line.
    elseif ms_progpars-update_co_in_bs_lines = abap_true or
        ms_progpars-update_existing_lines = abap_true.
        lv_corr_type = gc_corr_type-update_line.
    endif.
endif.
endcase.
if iv_corr_type is initial.
    rv_corr_type = lv_corr_type.
elseif lv_corr_type > iv_corr_type.
    rv_corr_type = lv_corr_type.
else.
    rv_corr_type = iv_corr_type.
endif.
endmethod.
```

DETERMINE_JVACTIVITY

Description:

Instance mthd

Importing parameter

IV_ACTIV TYPE JVS01-ACTIV

Returning parameter

VALUE(RV_JVACTIVITY) TYPE ACDOCA-JVACTIVITY

```

method determine_jvactivity.
    case iv_activ.
        when cl_jva_mig_correct_acdoca=>gc_jvlinetype-equity_change.
            rv_jvactivity = cl_jva_mig_correct_acdoca=>gc_jvactivity-equity_change.
        when cl_jva_mig_correct_acdoca=>gc_jvlinetype-eq_adjust_reversal.
            rv_jvactivity = cl_jva_mig_correct_acdoca=>gc_jvactivity-eq_adjust_reversal.
        when cl_jva_mig_correct_acdoca=>gc_jvlinetype-eq_adjust_reposting.
            rv_jvactivity = cl_jva_mig_correct_acdoca=>gc_jvactivity-eq_adjust_reposting.
        when cl_jva_mig_correct_acdoca=>gc_jvlinetype-farm_io_reversal.
            rv_jvactivity = cl_jva_mig_correct_acdoca=>gc_jvactivity-farm_io_reversal.
        when cl_jva_mig_correct_acdoca=>gc_jvlinetype-farm_io_reposting.
            rv_jvactivity = cl_jva_mig_correct_acdoca=>gc_jvactivity-farm_io_reposting.
        when cl_jva_mig_correct_acdoca=>gc_jvlinetype-suspense.
            rv_jvactivity = cl_jva_mig_correct_acdoca=>gc_jvactivity-suspense.
        when cl_jva_mig_correct_acdoca=>gc_jvlinetype-unsuspense.
            rv_jvactivity = cl_jva_mig_correct_acdoca=>gc_jvactivity-unsuspense.
        when cl_jva_mig_correct_acdoca=>gc_jvlinetype-cutback_operator.
            rv_jvactivity = cl_jva_mig_correct_acdoca=>gc_jvactivity-cutback.
        when cl_jva_mig_correct_acdoca=>gc_jvlinetype-cashcall_operator.
            rv_jvactivity = cl_jva_mig_correct_acdoca=>gc_jvactivity-cashcall.
        when others.
            clear rv_jvactivity.
    endcase.
endmethod.
```

DETERMINE_JV_AMNTS_IN_ACD_LINE

Description:

Instance mthd

Changing parameter

```
CS_ACDOCA TYPE ACDOCA_JV

method determine_jv_amnts_in_acd_line.
  cs_acdoca-ksl_gl = cs_acdoca-ksl.
  cs_acdoca-ksl    = get_acdoca_amount_for_jv( is_acdoca = cs_acdoca
                                              iv_curtp  =
ms_classic_currencies-curt2_jv
                                              iv_waers  =
ms_classic_currencies-hwae2_jv ).
  cs_acdoca-hsl_4c = get_acdoca_amount_for_jv( is_acdoca = cs_acdoca
                                              iv_curtp  =
ms_classic_currencies-curt3_jv
                                              iv_waers  =
ms_classic_currencies-hwae3_jv ).
  cs_acdoca-ksl_4c = get_acdoca_amount_for_jv( is_acdoca = cs_acdoca
                                              iv_curtp  =
ms_classic_currencies-curt4_jv
                                              iv_waers  =
ms_classic_currencies-hwae4_jv ).
endmethod.
```

DETERMINE_JV_DOC_TYPE

Description:
Instance mthd

Importing parameter

```
IV_ACTIV TYPE JVS01-ACTIV
```

Changing parameter

```
CS_DOC TYPE GTY_S_DOC
CV_HAS_REVERSAL TYPE ABAP_BOOL
CV_HAS_REPOSTING TYPE ABAP_BOOL

method determine_jv_doc_type.
  case iv_activ.
    when gc_jvlinetype-fi_doc or
         gc_jvlinetype-fi_doc_mm.
      if cs_doc-jvdoctype is initial.
        cs_doc-jvdoctype = gc_jvdoctype-fi_doc.
      endif.
    when gc_jvlinetype-co_doc.
      if cs_doc-jvdoctype is initial.
        cs_doc-jvdoctype = gc_jvdoctype-co_doc.
      endif.
    when gc_jvlinetype-equity_change.
      cs_doc-jvdoctype = gc_jvdoctype-equity_change.
      cv_has_reversal = abap_true.
      cv_has_reposting = abap_true.
    when gc_jvlinetype-eq_adjust_reversal.
      cs_doc-jvdoctype = gc_jvdoctype-equity_adjustment.
      cv_has_reversal = abap_true.
    when gc_jvlinetype-eq_adjust_reposting.
      cs_doc-jvdoctype = gc_jvdoctype-equity_adjustment.
      cv_has_reposting = abap_true.
```

```
when gc_jvlinetype-suspense.  
  cs_doc-jvdoctype = gc_jvdoctype-suspense.  
  cv_has_reversal = abap_true.  
when gc_jvlinetype-unsuspense.  
  cs_doc-jvdoctype = gc_jvdoctype-suspense.  
  cv_has_reposting = abap_true.  
when gc_jvlinetype-cutback_operator.  
  cs_doc-jvdoctype = gc_jvdoctype-cutback.  
  cv_has_reversal = abap_true.  
  cv_has_reposting = abap_true.  
when gc_jvlinetype-cashcall_operator.  
  cs_doc-jvdoctype = gc_jvdoctype-cash_call.  
  cv_has_reversal = abap_true.  
  cv_has_reposting = abap_true.  
endcase.  
endmethod.
```

DISPLAY_RESULTS

Description:

Instance mthd

```
method display_results.  
  if lines( mt_results ) <= 1 and  
    ms_progpars-print_results = abap_false and  
    ms_results_global-doc_count = 0 and  
    ms_results_global-line_count = 0.  
    message i708(gi).  
    return.  
  endif.  
  if ms_progpars-print_results = abap_true.  
    cl_jva_mig_alv_output=>get_instance( abap_true )->display_spools(  
      it_spool = ms_results_global-t_spool  
      is_results = ms_results_global ).  
  else.  
    if lines( mt_results ) = 0.  
      cl_jva_mig_alv_output=>get_instance( )->display_results( is_results =  
ms_results ).  
    elseif lines( mt_results ) = 1.  
      cl_jva_mig_alv_output=>get_instance( )->display_results( is_results =  
mt_results[ 1 ] ).  
    else.  
      "sort mt_results by bukr.  
      "insert ms_results_global into mt_results index 1.  
      cl_jva_mig_alv_output=>get_instance( )->display_results( it_results =  
mt_resultsis_results =  
ms_results_global ).  
    endif.  
  endif.  
endmethod.
```

GET_ACDOCA_AMOUNT_FOR_JV

Description:

Instance mthd

Importing parameter

IS_ACDOCA TYPE ACDOCA_JV
IV_CURTP TYPE CURTP
IV_WAERS TYPE WAERS

Returning parameter

VALUE(RV_AMOUNT) TYPE ACDOCA-KSL

```
method get_acdoca_amount_for_jv.  
  if iv_curtp is initial.  
    return.  
  endif.  
  case iv_curtp.  
    when ms_curr_info-curtype1.  
      rv_amount = is_acdoca-hsl.  
    when ms_curr_info-curtype2.  
      rv_amount = is_acdoca-ksl.  
    when ms_curr_info-curtype3.  
      rv_amount = is_acdoca-osl.  
    when ms_curr_info-curtype4.  
      rv_amount = is_acdoca-vsl.  
    when ms_curr_info-curtype5.  
      rv_amount = is_acdoca-bsl.  
    when ms_curr_info-curtype6.  
      rv_amount = is_acdoca-csl.  
    when ms_curr_info-curtype7.  
      rv_amount = is_acdoca-dsl.  
    when ms_curr_info-curtype8.  
      rv_amount = is_acdoca-esl.  
    when ms_curr_info-curtype9.  
      rv_amount = is_acdoca-fsl.  
    when ms_curr_info-curtype10.  
      rv_amount = is_acdoca-gsl.  
    when others.  
      if iv_waers = is_acdoca-rwcur.  
        rv_amount = is_acdoca-wsl.  
      endif.  
    endcase.  
  endmethod.
```

GET_ACDOCA_DOC_LINES

Description:
Instance mthd

Importing parameter

IT_ACDOCA_ALL TYPE ACDOCA_JV_T
IT_ACDOCA_MIG TYPE ACDOCA_JV_T
IT_CORR_LOG TYPE GTY_T_CORR_LOG

Exporting parameter

ET_ACDOCA TYPE ACDOCA_JV_T

Changing parameter

CS_DOC TYPE GTY_S_DOC

```
method get_acdoca_doc_lines.
```

```
clear et_acdoca.
if cs_doc-is_fi_doc = abap_true.
  read table it_acdoca_all transporting no fields
                        with key rldnr   = cs_doc-ledger
                        rbukrs  = cs_doc-rbukrs
                        gjahr   = cs_doc-ryear
                        belnr   = cs_doc-reffidoc.

  if sy-subrc = 0.
    loop at it_acdoca_all into data(ls_acdoca)
      from sy-tabix.
      if ls_acdoca-rldnr <> cs_doc-ledger or
         ls_acdoca-rbukrs <> cs_doc-rbukrs or
         ls_acdoca-gjahr <> cs_doc-ryear or
         ls_acdoca-belnr <> cs_doc-reffidoc.
        exit.
      endif.
      append ls_acdoca to et_acdoca.
      if ls_acdoca-mig_source is not initial.
        "If the document was migrated, determine if it was fully migrated
        "with a single migration source
        if cs_doc-mig_source is initial.
          cs_doc-mig_source = ls_acdoca-mig_source.
        elseif cs_doc-mig_source <> ls_acdoca-mig_source.
          clear cs_doc-mig_source.
        endif.
      endif.
    endloop.
  endif.
  if cs_doc-bstat is not initial or
     cs_doc-mig_source = 'J'.
    sort et_acdoca by rbukrs gjahr belnr docln.
  else.
    sort et_acdoca by rbukrs gjahr belnr buzei awitem.
  endif.
elseif cs_doc-is_co_doc = abap_true.
  read table it_acdoca_all transporting no fields
                        with key rldnr   = cs_doc-ledger
                        rbukrs  = cs_doc-rbukrs
                        gjahr   = cs_doc-ryear
                        co_belnr = cs_doc-refdocnr.

  if sy-subrc = 0.
    loop at it_acdoca_all into ls_acdoca
      from sy-tabix.
      if ls_acdoca-rldnr <> cs_doc-ledger or
         ls_acdoca-rbukrs <> cs_doc-rbukrs or
         ls_acdoca-gjahr <> cs_doc-ryear or
         ls_acdoca-co_belnr <> cs_doc-refdocnr.
        exit.
      endif.
      append ls_acdoca to et_acdoca.
    endloop.
  endif.
  sort et_acdoca by rbukrs gjahr co_belnr co_buzei awitem.
elseif cs_doc-jvdoctype = gc_jvdoctype-cutback or
```

```
    cs_doc-jvdoctype = gc_jvdoctype-cash_call.
read table it_acdoca_all transporting no fields
    with key rldnr = cs_doc-ledger
    rbukrs = cs_doc-rbukrs
    gjahr = cs_doc-ryear
    zuonr = cs_doc-zuonr.

if sy-subrc = 0.
    loop at it_acdoca_all into ls_acdoca
        from sy-tabix.
        if ls_acdoca-rldnr <> cs_doc-ledger or
           ls_acdoca-rbukrs <> cs_doc-rbukrs or
           ls_acdoca-gjahr <> cs_doc-ryear or
           ls_acdoca-zuonr <> cs_doc-zuonr.
            exit.
        endif.
        append ls_acdoca to et_acdoca.
    endloop.
endif.
if et_acdoca[] is initial.
    data(ls_corr_log) = get_log_item( is_doc      = cs_doc
                                     it_corr_log = it_corr_log ).

    if ls_corr_log is not initial.
        read table it_acdoca_all transporting no fields
            with key rldnr = cs_doc-ledger
            rbukrs = cs_doc-rbukrs
            gjahr = ls_corr_log-gjahr_new
            belnr = ls_corr_log-belnr_new

            binary search.
        loop at it_acdoca_all into ls_acdoca
            from sy-tabix.
            if ls_acdoca-rldnr <> cs_doc-ledger or
               ls_acdoca-rbukrs <> cs_doc-rbukrs or
               ls_acdoca-gjahr <> ls_corr_log-gjahr_new or
               ls_acdoca-belnr <> ls_corr_log-belnr_new.
                exit.
            endif.
            append ls_acdoca to et_acdoca.
        endloop.
    endif.
endif.
sort et_acdoca by rbukrs gjahr belnr buzei awitem.
endif.
if et_acdoca[] is initial and
   mv_has_migrated_jv_docs = abap_true.
    read table it_acdoca_mig transporting no fields
        with key rldnr = cs_doc-ledger
        rbukrs = cs_doc-rbukrs
        gjahr = cs_doc-ryear
        awref = cs_doc-refdocnr

        binary search.
    loop at it_acdoca_mig into ls_acdoca
        from sy-tabix.
        if ls_acdoca-rldnr <> cs_doc-ledger or
           ls_acdoca-rbukrs <> cs_doc-rbukrs or
```

```

        ls_acdoca-gjahr <> cs_doc-ryear or
        ls_acdoca-awref <> cs_doc-refdocnr.
    exit.
endif.
append ls_acdoca to et_acdoca.
endloop.
if cs_doc-bstat is not initial or
    cs_doc-mig_source = 'J'.
    sort et_acdoca by rbukrs gjahr belnr docln.
else.
    sort et_acdoca by rbukrs gjahr belnr buzei awitem.
endif.
endif.
endmethod.
```

GET_ACDOCA_LINE_BY_ACC_DATA

Description:

Instance mthd

Importing parameter

```

IS_DOC TYPE GTY_S_DOC
IS_JVSO TYPE JVSO
IT_ACDOCA_DOC TYPE ACDOCA_JV_T
IV_IS_REVERSAL_LINE TYPE ABAP_BOOL
IV_WSL TYPE ACDOCA-WSL
IV_HSL TYPE ACDOCA-HSL
```

Changing parameter

```

CS_LOG TYPE GTY_S_LOG
```

Returning parameter

```

VALUE(RT_ACDOCA) TYPE ACDOCA_JV_T
```

```

method get_acdoca_line_by_acc_data.
    data: ls_t8j0f      type t8j0f,
          lt_t8j0g      type t8j0g_t,
          lo_messages    type ref to if_jva_message_collection,
          lv_docln_acd    type acdoca-docln,
          lt_where_clause type gty_t_where_condition,
          lt_jvactivity   type gty_t_jvactivity.
    if is_doc-jvdoctype <> gc_jvdoctype-fi_doc and
        is_doc-jvdoctype <> gc_jvdoctype-co_doc.
        mo_settings->get_posting_rules_by_activity(
            exporting
                iv_vname      = is_jvso-rjvnam
                iv_vptnr      = is_jvso-rpartn
                iv_activ       = is_jvso-activ
                iv_offset      = abap_false
            changing
                cs_t8j0f      = ls_t8j0f
                ct_t8j0g      = lt_t8j0g
                co_messages    = lo_messages ).
    endif.
    if mt_fields_acdoca[] is initial.
        cl_jva_util=>get_selection_fields(
            exporting
```

```

        it_table      = rt_acdoca
        iv_source_table = 'ACDOCA'
        iv_aggr_amnts  = abap_false
        iv_prefix      = space
    changing
        ct_sel_fields  = mt_fields_acdoca
        ct_group_fields = mt_group_acdoca ).
endif.
"Get line based on accounting data and amount
case is_doc-jvdoctype.
    when gc_jvdoctype-cutback or
         gc_jvdoctype-cash_call.
        if is_doc-jvdoctype = gc_jvdoctype-cutback.
            lt_jvactivity = value #( ( 'CI' ) ).
        elseif is_doc-jvdoctype = gc_jvdoctype-cash_call.
            lt_jvactivity = value #( ( 'CC' ) ).
        endif.
        make_where_clause(
            exporting
                is_jvso          = is_jvso
                iv_rldnr         = is_doc-ledger
                iv_is_co_doc      = abap_false
                it_jvactivity     = lt_jvactivity
                iv_search_with_account = abap_false
                iv_search_with_aworg = abap_false
                iv_search_with_acd_only = abap_false
                iv_search_with_postdate = abap_true
                iv_search_with_docdate = abap_false
                iv_search_with_prodper = abap_false
                iv_search_with_recid  = abap_false
                iv_search_with_egrup  = abap_true
                iv_partner_as_customer = abap_true
                iv_search_with_rbusa  = abap_false
                iv_search_prev_periods = abap_false
                iv_search_with_sgtxt  = abap_false
            importing
                et_where_clause     = lt_where_clause ).
        select (mt_fields_acdoca) from acdoca into corresponding fields of table
@rt_acdoca
                                where (lt_where_clause).
    when gc_jvdoctype-equity_change.
        make_where_clause(
            exporting
                is_jvso          = is_jvso
                iv_rldnr         = is_doc-ledger
                iv_is_co_doc      = abap_false
                it_jvactivity     = lt_jvactivity
                iv_search_with_account = abap_true
                iv_search_with_aworg = abap_false
                iv_search_with_acd_only = abap_true
                iv_search_with_postdate = abap_true
                iv_search_with_docdate = abap_false
                iv_search_with_prodper = abap_false
                iv_search_with_recid  = abap_true

```

```

        iv_search_with_egrup      = abap_true
        iv_partner_as_customer    = abap_false
        iv_search_with_rbusa      = abap_false
        iv_search_prev_periods    = abap_false
        iv_search_with_sgtxt      = abap_false
importing
    et_where_clause              = lt_where_clause ).
select (mt_fields_acdoca) from acdoca into corresponding fields of table
@rt_acdoca
                                where (lt_where_clause).
when gc_jvdoctype-equity_adjustment or
    gc_jvdoctype-suspense.
make_where_clause(
    exporting
        is_jvso                  = is_jvso
        iv_rldnr                 = is_doc-ledger
        iv_is_co_doc              = abap_false
        it_jvactivity             = lt_jvactivity
        iv_search_with_account    = abap_true
        iv_search_with_aworg      = abap_false
        iv_search_with_acd_only   = abap_true
        iv_search_with_postdate   = abap_true
        iv_search_with_docdate    = abap_false
        iv_search_with_prodper    = abap_false
        iv_search_with_recid      = abap_true
        iv_search_with_egrup      = abap_true
        iv_partner_as_customer    = abap_false
        iv_search_with_rbusa      = abap_false
        iv_search_prev_periods    = abap_false
        iv_search_with_sgtxt      = abap_false
importing
    et_where_clause              = lt_where_clause ).
select (mt_fields_acdoca) from acdoca into corresponding fields of table
@rt_acdoca
                                where (lt_where_clause).

endcase.
endmethod.
```

GET_ACDOCA_LINE_BY_REF

Description:
Instance mthd

Importing parameter

- IS_DOC TYPE GTY_S_DOC
- IS_JVSO TYPE JVS0
- IV_TABIX TYPE SY-TABIX
- IT_ACDOCA_DOC TYPE ACDOCA_JV_T
- IT_ACDOCA_ALL TYPE ACDOCA_JV_T
- IT_ACDOCA_INT TYPE ACDOCA_JV_T
- IT_ACDOCA_MIG TYPE ACDOCA_JV_T
- IT_CORR_LOG TYPE GTY_T_CORR_LOG
- IV_IS_REV_LINE TYPE ABAP_BOOL

Changing parameter

- CS_LOG TYPE GTY_S_LOG

Returning parameter

VALUE(RT_ACDOCA) TYPE ACDOCA_JV_T

method get_acdoca_line_by_ref.

```
data: lt_acdoca      type acdoca_jv_t,  
      lv_docln_acd   type acdoca-docln,  
      lv_awitem      type acdoca-awitem,  
      lv_buzei       type acdoca-buzei,  
      lv_co_buzei    type acdoca-co_buzei,  
      lv_jvactivity  type acdoca-jvactivity,  
      lv_search_amnt type abap_bool,  
      lv_pos_count   type i,  
      lv_neg_count   type i.
```

lv_awitem = |{ is_jvso-refdocln alpha = in }|.

if is_doc-jvdoctype = gc_jvdoctype-fi_doc.

if is_doc-bstat is not initial.

data(lv_assign_by_docln) = abap_true.

elseif is_doc-mig_source = 'J'.

data(lv_assign_by_tabix) = abap_true.

else.

"First search for BUZEI = REFFIDLN

read table it_acdoca_doc transporting no fields

with key rbukrs = is_jvso-rbukrs

gjahr = is_jvso-ryear

belnr = is_jvso-reffidoc

buzei = is_jvso-reffidln

binary search.

if sy-subrc = 0.

loop at it_acdoca_doc into data(ls_acdoca)

from sy-tabix.

if ls_acdoca-rbukrs <> is_jvso-rbukrs or

ls_acdoca-gjahr <> is_jvso-ryear or

ls_acdoca-belnr <> is_jvso-reffidoc or

ls_acdoca-buzei <> is_jvso-reffidln.

exit.

endif.

append ls_acdoca to rt_acdoca.

endloop.

else.

"Search for AWITEM = REFDOLN

read table it_acdoca_doc transporting no fields

with key rbukrs = is_jvso-rbukrs

gjahr = is_jvso-ryear

belnr = is_jvso-reffidoc

binary search.

if sy-subrc = 0.

loop at it_acdoca_doc into ls_acdoca

from sy-tabix.

if ls_acdoca-rbukrs <> is_jvso-rbukrs or

ls_acdoca-gjahr <> is_jvso-ryear or

ls_acdoca-belnr <> is_jvso-reffidoc.

exit.

endif.

if ls_acdoca-awitem = lv_awitem.

```
        append ls_acdoca to rt_acdoca.
      endif.
    endloop.
  endif.
endif.
if lv_assign_by_docln = abap_true.
  lv_docln_acd = |{ is_jvso-docln alpha = in }|.
  read table it_acdoca_doc into ls_acdoca
    with key rbukrs = is_jvso-rbukrs
         gjahr  = is_jvso-ryear
         belnr  = is_jvso-reffidoc
         docln  = lv_docln_acd
    binary search.

  if sy-subrc = 0.
    append ls_acdoca to rt_acdoca.
  endif.
endif.
if lv_assign_by_tabix = abap_true.
  read table it_acdoca_doc into ls_acdoca
    index iv_tabix.

  if sy-subrc = 0.
    append ls_acdoca to rt_acdoca.
  endif.
endif.
elseif is_doc-jvdoctype = gc_jvdoctype-co_doc.
  lv_co_buzei = |{ is_jvso-refdocln alpha = in }|.
  "First search for CO_BUZEI = REFDOCLN
  read table it_acdoca_doc transporting no fields
    with key rbukrs  = is_jvso-rbukrs
         gjahr      = is_jvso-ryear
         co_belnr   = is_jvso-refdocnr
         co_buzei   = lv_co_buzei
    binary search.

  if sy-subrc = 0.
    loop at it_acdoca_doc into ls_acdoca
      from sy-tabix.
      if ls_acdoca-rbukrs <> is_jvso-rbukrs or
         ls_acdoca-gjahr <> is_jvso-ryear or
         ls_acdoca-co_belnr <> is_jvso-refdocnr or
         ls_acdoca-co_buzei <> lv_co_buzei.
        exit.
      endif.
      append ls_acdoca to rt_acdoca.
    endloop.
  else.
    "Search with AITEM = REFDOCLN
    read table it_acdoca_doc transporting no fields
      with key rbukrs  = is_jvso-rbukrs
           gjahr      = is_jvso-ryear
           co_belnr   = is_jvso-refdocnr
      binary search.

    if sy-subrc = 0.
      loop at it_acdoca_doc into ls_acdoca
```

```

        from sy-tabix.
    if ls_acdoca-rbukrs <> is_jvso-rbukrs or
       ls_acdoca-gjahr <> is_jvso-ryear or
       ls_acdoca-co_belnr <> is_jvso-refdocnr.
        exit.
    endif.
    if ls_acdoca-awitem = lv_awitem.
        append ls_acdoca to rt_acdoca.
    endif.
endloop.
endif.
elseif is_jvso-activ = gc_jvlinetype-cutback_operator or
       is_jvso-activ = gc_jvlinetype-cashcall_operator.
    if is_jvso-activ = gc_jvlinetype-cashcall_operator.
        lv_buzei = |{ is_jvso-docln alpha = in }|.
    elseif is_jvso-activ = gc_jvlinetype-cutback_operator.
        lv_buzei = '001'.
    endif.
    lv_search_amnt = abap_true.
    loop at it_acdoca_doc into ls_acdoca
        where wsl = is_jvso-tsl
          and hsl = is_jvso-hsl
          and ksl = is_jvso-ksl
          and hsl_4c = is_jvso-hsl_4c
          and ksl_4c = is_jvso-ksl_4c
          and racct = is_jvso-racct.

        append ls_acdoca to rt_acdoca.
    endloop.
    if rt_acdoca[] is initial.
        "Check if the document was already treated by checking for the
        "entry in the log table
        data(ls_corr_log) = get_log_item( is_doc      = is_doc
                                          it_corr_log = it_corr_log ).

    if ls_corr_log is not initial.
        "Use the data in the log table to retrieve the relevant doc line
        read table it_acdoca_all into ls_acdoca
            with key rbukrs = is_jvso-rbukrs
              gjahr  = ls_corr_log-gjahr_new
              belnr  = ls_corr_log-belnr_new
              docln  = ls_corr_log-docln_new
            binary search.

        if sy-subrc = 0.
            append ls_acdoca to rt_acdoca.
        else.
            read table it_acdoca_all into ls_acdoca
                with key rbukrs = is_jvso-rbukrs
                  gjahr  = ls_corr_log-gjahr_new
                  belnr  = ls_corr_log-belnr_new
                binary search.

            if sy-subrc = 0.
                loop at it_acdoca_all into ls_acdoca
                    from sy-tabix.
                        if ls_acdoca-rbukrs <> is_jvso-rbukrs or

```

```
        ls_acdoca-gjahr <> ls_corr_log-gjahr_new or
        ls_acdoca-belnr <> ls_corr_log-belnr_new.
    exit.
endif.
append ls_acdoca to rt_acdoca.
endloop.
endif.
endif.
endif.
else.
    if it_acdoca_doc[] is not initial.
        if is_jvso-activ = gc_jvlinetype-fi_doc or
           is_jvso-activ = gc_jvlinetype-fi_doc_mm.
            read table it_acdoca_doc transporting no fields
                                with key rbukrs = is_jvso-rbukrs
                                    gjahr = is_jvso-ryear
                                    belnr = is_jvso-reffidoc
                                    awitem = lv_awitem
                                    buzei = is_jvso-reffidln
                                binary search.
        if sy-subrc = 0.
            loop at it_acdoca_doc into ls_acdoca
                                from sy-tabix.
                if ls_acdoca-rbukrs <> is_jvso-rbukrs or
                   ls_acdoca-gjahr <> is_jvso-ryear or
                   ls_acdoca-belnr <> is_jvso-reffidoc or
                   ls_acdoca-awitem <> lv_awitem or
                   ls_acdoca-buzei <> is_jvso-reffidln.
                    exit.
                endif.
                append ls_acdoca to rt_acdoca.
            endloop.
        else.
            read table it_acdoca_doc transporting no fields
                                with key rbukrs = is_jvso-rbukrs
                                    gjahr = is_jvso-ryear
                                    belnr = is_jvso-reffidoc
                                    awitem = lv_awitem
                                binary search.
        if sy-subrc = 0.
            loop at it_acdoca_doc into ls_acdoca
                                from sy-tabix.
                if ls_acdoca-rbukrs <> is_jvso-rbukrs or
                   ls_acdoca-gjahr <> is_jvso-ryear or
                   ls_acdoca-belnr <> is_jvso-reffidoc or
                   ls_acdoca-awitem <> lv_awitem.
                    exit.
                endif.
                append ls_acdoca to rt_acdoca.
            endloop.
        endif.
    endif.
elseif is_jvso-activ = gc_jvlinetype-co_doc.
```

```
lv_co_buzei = |{ is_jvso-refdocln alpha = in }|.
loop at it_acdoca_doc into ls_acdoca
    where rbukrs    = is_jvso-rbukrs
    and gjahr      = is_jvso-ryear
    and co_belnr   = is_jvso-refdocnr
    and co_buzei   = lv_co_buzei.
    append ls_acdoca to rt_acdoca.
endloop.
else.
loop at it_acdoca_doc into ls_acdoca
    where rbukrs    = is_jvso-rbukrs
    and gjahr      = is_jvso-ryear
    and poper      = is_jvso-poper
    and prec_awtyp = is_jvso-awtyp
    and prec_aworg = is_jvso-aworg
    and prec_awref = is_jvso-refdocnr
    and prec_awitem = lv_awitem.
    append ls_acdoca to rt_acdoca.
endloop.
if rt_acdoca[] is initial and
    is_jvso-reffidln is not initial and
    is_jvso-refdocct = 'W'.
    lv_search_amnt = abap_true.
    loop at it_acdoca_doc into ls_acdoca
        where buzei = is_jvso-reffidln
        and wsl     = is_jvso-tsl
        and hsl     = is_jvso-hsl
        and ksl     = is_jvso-ksl
        and hsl_4c  = is_jvso-hsl_4c
        and ksl_4c  = is_jvso-ksl_4c
        and racct   = is_jvso-racct
        and vname   = is_jvso-rjvnam
        and egrup   = is_jvso-regrou
        and recid   = is_jvso-rrecin.
        append ls_acdoca to rt_acdoca.
    endloop.
    if rt_acdoca[] is initial.
        loop at it_acdoca_doc into ls_acdoca
            where buzei = is_jvso-reffidln
            and wsl     = is_jvso-tsl
            and hsl     = is_jvso-hsl
            and ksl     = is_jvso-ksl
            and hsl_4c  = is_jvso-hsl_4c
            and ksl_4c  = is_jvso-ksl_4c
            and racct   = is_jvso-racct.
            append ls_acdoca to rt_acdoca.
        endloop.
    endif.
endif.
if rt_acdoca[] is initial.
    lv_search_amnt = abap_true.
    loop at it_acdoca_doc into ls_acdoca
        where wsl     = is_jvso-tsl
        and hsl       = is_jvso-hsl
```

```
        and ksl      = is_jvso-ksl
        and hsl_4c   = is_jvso-hsl_4c
        and ksl_4c   = is_jvso-ksl_4c
        and racct    = is_jvso-racct
        and vname    = is_jvso-rjvnam
        and egrupp   = is_jvso-regrou
        and recid    = is_jvso-rrecin.
    append ls_acdoca to rt_acdoca.
endloop.
if rt_acdoca[] is initial.
    loop at it_acdoca_doc into ls_acdoca
        where wsl      = is_jvso-tsl
        and hsl        = is_jvso-hsl
        and ksl        = is_jvso-ksl
        and hsl_4c     = is_jvso-hsl_4c
        and ksl_4c     = is_jvso-ksl_4c
        and racct      = is_jvso-racct.

        append ls_acdoca to rt_acdoca.
    endloop.
endif.
if rt_acdoca[] is initial and
    ( is_jvso-activ    = gc_jvlinetype-balancing_line or
      is_jvso-refactiv = gc_jvlinetype-balancing_line ).
    lv_search_amnt = abap_true.
    loop at it_acdoca_doc into ls_acdoca
        where wsl      = is_jvso-tsl
        and hsl        = is_jvso-hsl
        and ksl        = is_jvso-ksl
        and hsl_4c     = is_jvso-hsl_4c
        and ksl_4c     = is_jvso-ksl_4c
        and vname      = is_jvso-rjvnam
        and egrupp     = is_jvso-regrou
        and recid      = is_jvso-rrecin
        and buzei      is initial.

        append ls_acdoca to rt_acdoca.
    endloop.
    if rt_acdoca[] is initial.
        loop at it_acdoca_doc into ls_acdoca
            where wsl      = is_jvso-tsl
            and hsl        = is_jvso-hsl
            and ksl        = is_jvso-ksl
            and hsl_4c     = is_jvso-hsl_4c
            and ksl_4c     = is_jvso-ksl_4c
            and buzei      is initial.

            append ls_acdoca to rt_acdoca.
        endloop.
    endif.
endif.
endif.
if rt_acdoca[] is initial and
    is_doc-is-fi_doc = abap_false.
    read table it_acdoca_int transporting no fields
```

```

                with key rbukrs      = is_jvso-rbukrs
                  gjahr      = is_jvso-ryear
                  prec_awtyp  = is_jvso-awtyp
                  prec_aworg  = is_jvso-aworg
                  prec_awref  = is_jvso-refdocnr
                  prec_awitem = lv_awitem
                binary search.
loop at it_acdoca_int into ls_acdoca
  from sy-tabix.
  if ls_acdoca-rbukrs <> is_jvso-rbukrs or
    ls_acdoca-gjahr <> is_jvso-ryear or
    ls_acdoca-prec_awtyp <> is_jvso-awtyp or
    ls_acdoca-prec_aworg <> is_jvso-aworg or
    ls_acdoca-prec_awref <> is_jvso-refdocnr or
    ls_acdoca-prec_awitem <> lv_awitem.
    exit.
  endif.
  append ls_acdoca to rt_acdoca.
endloop.
if rt_acdoca[] is initial.
  "In some cases the PREC_AWORG and PREC_AWTYP values can be initial (cause
unknown)
  read table it_acdoca_int transporting no fields
    with key rbukrs      = is_jvso-rbukrs
      gjahr      = is_jvso-ryear
      prec_awtyp  = space
      prec_aworg  = space
      prec_awref  = is_jvso-refdocnr
      prec_awitem = lv_awitem
    binary search.
  loop at it_acdoca_int into ls_acdoca
    from sy-tabix.
    if ls_acdoca-rbukrs <> is_jvso-rbukrs or
      ls_acdoca-gjahr <> is_jvso-ryear or
      ls_acdoca-prec_awtyp <> space or
      ls_acdoca-prec_aworg <> space or
      ls_acdoca-prec_awref <> is_jvso-refdocnr or
      ls_acdoca-prec_awitem <> lv_awitem.
      exit.
    endif.
    append ls_acdoca to rt_acdoca.
  endloop.
endif.
if rt_acdoca[] is initial.
  "Check if the document was already treated by checking for the
  "entry in the log table
  ls_corr_log = get_log_item( is_doc      = is_doc
                             it_corr_log = it_corr_log ).
  if ls_corr_log is not initial.
    read table it_acdoca_all transporting no fields
      with key rbukrs = is_jvso-rbukrs
        gjahr  = ls_corr_log-gjahr_new
        belnr  = ls_corr_log-belnr_new

```

```

                                binary search.
loop at it_acdoca_all into ls_acdoca
                                from sy-tabix.
if ls_acdoca-rbukrs <> is_jvso-rbukrs or
ls_acdoca-gjahr <> ls_corr_log-gjahr_new or
ls_acdoca-belnr <> ls_corr_log-belnr_new.
exit.
endif.
if ls_acdoca-prec_awtyp = is_jvso-awtyp and
ls_acdoca-prec_aworg = is_jvso-aworg and
ls_acdoca-prec_awref = is_jvso-refdocnr and
ls_acdoca-prec_awitem = lv_awitem.
append ls_acdoca to rt_acdoca.
endif.
endloop.
endif.
endif.
if rt_acdoca[] is initial and
it_acdoca_mig[] is not initial.
"Check if the document was posted by S4HANA migration functionality
read table it_acdoca_mig transporting no fields
                                with key rbukrs = is_jvso-rbukrs
                                gjahr = is_jvso-ryear
                                awref = is_jvso-refdocnr
                                awitem = lv_awitem
                                binary search.
if sy-subrc = 0.
loop at it_acdoca_mig into ls_acdoca
                                from sy-tabix.
if ls_acdoca-rbukrs <> is_jvso-rbukrs or
ls_acdoca-gjahr <> is_jvso-ryear or
ls_acdoca-awref <> is_jvso-refdocnr or
ls_acdoca-awitem <> lv_awitem.
exit.
endif.
if ms_progpars-select_fi_documents = abap_true or
ms_progpars-select_co_documents = abap_true.
if ls_acdoca-jvactivity is initial.
append ls_acdoca to rt_acdoca.
endif.
else.
if lv_jvactivity is initial.
lv_jvactivity = determine_jvactivity( is_jvso-activ ).
endif.
if ls_acdoca-bttype = is_jvso-refactiv and
( ls_acdoca-jvactivity = lv_jvactivity or
ls_acdoca-jvactivity = is_jvso-activ+2(2) ).
append ls_acdoca to rt_acdoca.
endif.
endif.
endloop.
endif.
endif.
endif.

```

```
if rt_acdoca[] is not initial or
  lv_search_amnt = abap_false.
  return.
endif.
"If the JV lines contains both positive and negative amounts,
"the search needs to be executed separately for each case
if is_jvso-tsl > 0. add 1 to lv_pos_count. endif.
if is_jvso-hsl > 0. add 1 to lv_pos_count. endif.
if is_jvso-ksl > 0. add 1 to lv_pos_count. endif.
if is_jvso-hsl_4c > 0. add 1 to lv_pos_count. endif.
if is_jvso-ksl_4c > 0. add 1 to lv_pos_count. endif.
if is_jvso-tsl < 0. add 1 to lv_neg_count. endif.
if is_jvso-hsl < 0. add 1 to lv_neg_count. endif.
if is_jvso-ksl < 0. add 1 to lv_neg_count. endif.
if is_jvso-hsl_4c < 0. add 1 to lv_neg_count. endif.
if is_jvso-ksl_4c < 0. add 1 to lv_neg_count. endif.
if lv_pos_count > 0 and lv_neg_count > 0.
  data(ls_jvso_pos) = is_jvso.
  if ls_jvso_pos-tsl < 0. clear ls_jvso_pos-tsl. endif.
  if ls_jvso_pos-hsl < 0. clear ls_jvso_pos-hsl. endif.
  if ls_jvso_pos-ksl < 0. clear ls_jvso_pos-ksl. endif.
  if ls_jvso_pos-hsl_4c < 0. clear ls_jvso_pos-hsl_4c. endif.
  if ls_jvso_pos-ksl_4c < 0. clear ls_jvso_pos-ksl_4c. endif.
  data(ls_jvso_neg) = is_jvso.
  if ls_jvso_neg-tsl > 0. clear ls_jvso_neg-tsl. endif.
  if ls_jvso_neg-hsl > 0. clear ls_jvso_neg-hsl. endif.
  if ls_jvso_neg-ksl > 0. clear ls_jvso_neg-ksl. endif.
  if ls_jvso_neg-hsl_4c > 0. clear ls_jvso_neg-hsl_4c. endif.
  if ls_jvso_neg-ksl_4c > 0. clear ls_jvso_neg-ksl_4c. endif.
  data(lt_acdoca_pos) = get_acdoca_line_by_ref(
    exporting
      is_doc      = is_doc
      is_jvso     = ls_jvso_pos
      iv_tabix    = iv_tabix
      it_acdoca_doc = it_acdoca_doc
      it_acdoca_all = it_acdoca_all
      it_acdoca_int = it_acdoca_int
      it_acdoca_mig = it_acdoca_mig
      it_corr_log  = it_corr_log
      iv_is_rev_line = iv_is_rev_line
    changing
      cs_log      = cs_log ).
  data(lt_acdoca_neg) = get_acdoca_line_by_ref(
    exporting
      is_doc      = is_doc
      is_jvso     = ls_jvso_neg
      iv_tabix    = iv_tabix
      it_acdoca_doc = it_acdoca_doc
      it_acdoca_all = it_acdoca_all
      it_acdoca_int = it_acdoca_int
      it_acdoca_mig = it_acdoca_mig
      it_corr_log  = it_corr_log
      iv_is_rev_line = iv_is_rev_line
    changing
```

```

                                cs_log          = cs_log ).
    append lines of lt_acdoca_pos to rt_acdoca.
    append lines of lt_acdoca_neg to rt_acdoca.
    return.
endif.
endmethod.
```

GET_ACDOCA_LINE_FOR_JV

Description:
Instance mthd

Importing parameter

```
IS_DOC TYPE GTY_S_DOC
IS_JVSO TYPE JVSO
IV_TABIX TYPE SY-TABIX
IT_ACDOCA_DOC TYPE ACDOCA_JV_T
IT_ACDOCA_ALL TYPE ACDOCA_JV_T
IT_ACDOCA_INT TYPE ACDOCA_JV_T
IT_ACDOCA_MIG TYPE ACDOCA_JV_T
IT_CORR_LOG TYPE GTY_T_CORR_LOG
IT_JV_COMPR_INFO TYPE GTY_T_JV_COMPR_INFO
IV_COMPRESS_BY_PC TYPE ABAP_BOOL
IS_LINE_ASSIGNMENTS TYPE GTY_S_LINE_ASSIGNMENTS
```

Exporting parameter

```
ET_ACD_COMPR_INFO TYPE GTY_T_ACD_COMPR_INFO
```

Changing parameter

```
CS_LOG TYPE GTY_S_LOG
```

Returning parameter

```
VALUE(RT_ACDOCA) TYPE ACDOCA_JV_T

method get_acdoca_line_for_jv.
    data: ls_acdoca          type acdoca_jv,
          lv_is_reversal_line type abap_bool,
          ls_message         type bapiret2.
    if is_jvso-refdocct = 'K'.
        data(lv_is_co_line) = abap_true.
    endif.
    case cs_log-activ.
        when gc_jvlinetype-equity_change.
            if is_jvso-docln = '001'.
                lv_is_reversal_line = abap_true.
            endif.
        when gc_jvlinetype-eq_adjust_reversal.
            lv_is_reversal_line = abap_true.
        when gc_jvlinetype-suspense.
            lv_is_reversal_line = abap_true.
    endcase.
    rt_acdoca = get_acdoca_line_by_ref(
        exporting
            is_doc          = is_doc
            is_jvso          = is_jvso
            iv_tabix         = iv_tabix
            it_acdoca_doc    = it_acdoca_doc
```

```

        it_acdoca_all = it_acdoca_all
        it_acdoca_int = it_acdoca_int
        it_acdoca_mig = it_acdoca_mig
        it_corr_log = it_corr_log
        iv_is_rev_line = lv_is_reversal_line
    changing
        cs_log = cs_log ).
if ms_progpars-search_by_accounting_data = abap_true and
    lines( rt_acdoca ) = 0 and
    cs_log-jvdoctype <> gc_jvdoctype-fi_doc and
    cs_log-jvdoctype <> gc_jvdoctype-co_doc.
    rt_acdoca = get_acdoca_line_by_acc_data(
        exporting
            is_doc = is_doc
            is_jvso = is_jvso
            it_acdoca_doc = it_acdoca_doc
            iv_is_reversal_line = lv_is_reversal_line
            iv_wsl = conv fins_vwcur12( is_jvso-ysl )
            iv_hsl = conv fins_vwcur12( is_jvso-hsl )
        changing
            cs_log = cs_log ).
    if lines( rt_acdoca ) > 0.
        add_message_to_log(
            exporting
                iv_msg_text = 'ACDOCA line found by accounting data for JV doc line'(afa)
&& ` ` && condense( |{ is_jvso-docln alpha = 0
                iv_msg_type = gc_error_type-info
            changing
                cs_log = cs_log ).
    endif.
endif.
loop at rt_acdoca assigning field-symbol(<ls_acdoca>)
    where ksl_gl = 0.
    determine_jv_amnts_in_acd_line( changing cs_acdoca = <ls_acdoca> ).
endloop.
if lines( rt_acdoca ) > 1.
    rt_acdoca = multiple_acd_line_processing(
        exporting
            is_doc = is_doc
            is_jvso = is_jvso
            it_acdoca_sel = rt_acdoca
            it_acdoca_original = rt_acdoca
            is_line_assignments = is_line_assignments
            iv_do_heuristic = ms_progpars-heuristic_line_assignment
            iv_compress_by_pc = iv_compress_by_pc
        importing
            et_acd_compr_info = et_acd_compr_info
        changing
            cs_log = cs_log ).
endif.
if lines( rt_acdoca ) <> 1.
    if is_jvso-docln is not initial.
        data(lv_docln_info) = condense( |{ is_jvso-docln alpha = out }| ).
    else.

```

```

read table it_jv_compr_info assigning field-symbol(<ls_jv_compr_info>)
                        with key tabix_aggr = is_jvso-tabix_aggr.
if sy-subrc = 0 and <ls_jv_compr_info> is assigned.
  data(lv_line_count) = lines( <ls_jv_compr_info>-t_docln_original ).
  if lv_line_count > 10.
    lv_docln_info = `{` && lv_line_count && ` ` && 'lines'(lns) && `}`.
  else.
    loop at <ls_jv_compr_info>-t_docln_original into data(ls_line_info).
      if lv_docln_info is not initial.
        lv_docln_info &&= `+`.
      endif.
      lv_docln_info &&= condense( |{ ls_line_info-docln alpha = out }| ).
    endloop.
  endif.
endif.
if <ls_jv_compr_info> is not assigned.
  lv_docln_info = condense( |{ is_jvso-docln alpha = out }| ).
endif.
if is_jvso-reffidln is not initial.
  data(lv_refln_info) = 'FI doc line'(fil) && ` ` && condense( |{
is_jvso-reffidln alpha = out }| ).
endif.
if is_jvso-refdocln is not initial and
  is_jvso-refdocln <> is_jvso-reffidln.
  if lv_refln_info is not initial.
    lv_refln_info &&= `,`.
  endif.
  lv_refln_info &&= 'Ref. doc line'(rdl) && ` ` && condense( |{
is_jvso-refdocln alpha = out }| ).
endif.
if lv_refln_info is not initial.
  lv_docln_info &&= `[` && lv_refln_info && `]`.
endif.
endif.
if lines( rt_acdoca ) = 0 and
  ( is_jvso-tsl <> 0 or
    is_jvso-hsl <> 0 or
    is_jvso-ksl <> 0 or
    is_jvso-hsl_4c <> 0 or
    is_jvso-ksl_4c <> 0 ).
  if is_jvso-activ = gc_jvlinetype-balancing_line or
    is_jvso-refactiv = gc_jvlinetype-balancing_line.
    data(lv_issue_type) = gc_issue_type-balancing_line_issue.
  else.
    lv_issue_type = gc_issue_type-line_missing.
  endif.
  cs_log-issues = lv_issue_type.
  cs_log-corr_type = determine_correction_type( iv_corr_type = space
                                                iv_issue_type = lv_issue_type
                                                iv_is_co_line = lv_is_co_line ).
  if cs_log-corr_type = gc_corr_type-correction_not_possible.
    data(lv_error_type) = gc_error_type-error.
  else.

```

```

        lv_error_type = gc_error_type-warning.
    endif.
    add_message_to_log(
        exporting
            iv_msg_text = 'ACDOCA line not found for JV doc line'(anf) && ` ` &&
lv_docln_info
            iv_msg_type = lv_error_type
        changing
            cs_log = cs_log ).
elseif lines( rt_acdoca ) = 1.
    read table rt_acdoca index 1 into ls_acdoca.        "#EC CI_NOORDER
    if sy-subrc = 0.
        cs_log-rldnr_acd = ls_acdoca-rldnr.
        cs_log-belnr_acd = ls_acdoca-belnr.
        cs_log-docln_acd = ls_acdoca-docln.
        cs_log-gjahr_acd = ls_acdoca-gjahr.
    endif.
elseif lines( rt_acdoca ) > 1.
    if cs_log-heuristic_hit = abap_true.
        add_message_to_log(
            exporting
                iv_msg_text = 'Multiple ACDOCA lines assigned to JV doc line'(mlh) && ` `
&& lv_docln_info
                iv_msg_type = gc_error_type-info
            changing
                cs_log = cs_log ).
        read table rt_acdoca index 1 into ls_acdoca.        "#EC CI_NOORDER
        if sy-subrc = 0.
            cs_log-rldnr_acd = ls_acdoca-rldnr.
            cs_log-belnr_acd = ls_acdoca-belnr.
            cs_log-docln_acd = ls_acdoca-docln.
            cs_log-gjahr_acd = ls_acdoca-gjahr.
        endif.
    else.
        add_message_to_log(
            exporting
                iv_msg_text = 'Multiple ACDOCA lines found for JV doc line'(mlf) && ` `
&& lv_docln_info
                iv_msg_type = gc_error_type-error
            changing
                cs_log = cs_log ).
        clear rt_acdoca.
    endif.
endif.
endmethod.
```

GET_CLASSIC_CURRENCIES

Description:
Instance mthd

Importing parameter

IV_BUKRS TYPE BUKRS

method get_classic_currencies.

```

"Pretend that the classic JVA is active
cl_jva_switch_injector=>inject_test_double( new cl_jva_switch_stub_acdoca_off( )
).
call function 'JV_AND_FI_CURRENCIES'
  exporting
    bukrs      = iv_bukrs
  importing
    hwaer      = ms_classic_currencies-hwaer_jv
    hwaer2     = ms_classic_currencies-hwaer2_jv
    hwaer3     = ms_classic_currencies-hwaer3_jv
    hwaer4     = ms_classic_currencies-hwaer4_jv
    curt1      = ms_classic_currencies-curt1_jv
    curt2      = ms_classic_currencies-curt2_jv
    curt3      = ms_classic_currencies-curt3_jv
    curt4      = ms_classic_currencies-curt4_jv
    ldnrh1     = ms_classic_currencies-ldnrh1_jv
    ldnrh2     = ms_classic_currencies-ldnrh2_jv
    ldnrh3     = ms_classic_currencies-ldnrh3_jv
    ldnrh4     = ms_classic_currencies-ldnrh4_jv
    ldnrhf1    = ms_classic_currencies-ldnrhf1_jv
    ldnrhf2    = ms_classic_currencies-ldnrhf2_jv
    ldnrhf3    = ms_classic_currencies-ldnrhf3_jv
    ldnrhf4    = ms_classic_currencies-ldnrhf4_jv
    hwaer_fi   = ms_classic_currencies-hwaer_fi
    hwaer2_fi  = ms_classic_currencies-hwaer2_fi
    hwaer3_fi  = ms_classic_currencies-hwaer3_fi
    curt1_fi   = ms_classic_currencies-curt1_fi
    curt2_fi   = ms_classic_currencies-curt2_fi
    curt3_fi   = ms_classic_currencies-curt3_fi
    kuty2_fi   = ms_classic_currencies-kuty2_fi
    kuty3_fi   = ms_classic_currencies-kuty3_fi
    basw2_fi   = ms_classic_currencies-basw2_fi
    basw3_fi   = ms_classic_currencies-basw3_fi
    umrd2_fi   = ms_classic_currencies-umrd2_fi
    umrd3_fi   = ms_classic_currencies-umrd3_fi.
"Check if the JV currency is needed:
"(a) If a JV currency does is not maintained for the GL, clear the
"   settings to avoid false error messages about amount mismatches
"(b) If a ledger 4C currency is just a copy of a ledger 4A currency,
"   clear the settings to avoid double processing of the same currency
if ms_classic_currencies-curt2_jv is not initial.
  if jv_curtp_exists_as_gl_curtp( ms_classic_currencies-curt2_jv ) = abap_false.
    clear: ms_classic_currencies-curt2_jv,
           ms_classic_currencies-hwaer2_jv.
  endif.
endif.
if ms_classic_currencies-curt3_jv is not initial.
  if jv_curtp_exists_as_gl_curtp( ms_classic_currencies-curt3_jv ) = abap_false.
    clear: ms_classic_currencies-curt3_jv,
           ms_classic_currencies-hwaer3_jv.
  endif.
  if ms_classic_currencies-curt3_jv = ms_classic_currencies-curt1_jv or
    ms_classic_currencies-curt3_jv = ms_classic_currencies-curt2_jv.
    clear: ms_classic_currencies-curt3_jv,

```

```

        ms_classic_currencies-hwae3_jv.
    endif.
endif.
if ms_classic_currencies-curt4_jv is not initial.
    if jv_curtp_exists_as_gl_curtp( ms_classic_currencies-curt4_jv ) = abap_false.
        clear: ms_classic_currencies-curt4_jv,
               ms_classic_currencies-hwae4_jv.
    endif.
    if ms_classic_currencies-curt4_jv = ms_classic_currencies-curt1_jv or
       ms_classic_currencies-curt4_jv = ms_classic_currencies-curt2_jv.
        clear: ms_classic_currencies-curt4_jv,
               ms_classic_currencies-hwae4_jv.
    endif.
endif.
endif.
endmethod.                "get_classic_currencies

```

GET_COMPANY_CODE_SETTINGS

Description:

Instance mthd

Returning parameter

VALUE(RV_ERROR) TYPE ABAP_BOOL

```

method get_company_code_settings.
    try.
        mo_settings = cl_jva_settings=>get_instance( ms_progpars-bukrs ).
    catch cx_jva_settings.
        rv_error = abap_true.
        message e701(gi) with ms_progpars-bukrs into mv_message_text.
        add_sy_message_to_results( changing cs_results = ms_results ).
        return.
    endtry.
    "Check if the NewGL integration is active
    clear ms_progpars-newgl_integration_active.
    select single *
        from t8jz_fagl
        where bukrs = @ms_progpars-bukrs
        into @data(ls_t8jz_fagl).
    if ls_t8jz_fagl-usefaglsplit = abap_true.
        call function 'FI_SPLIT_ACTIVATION'
            exporting
                id_cocd    = ms_progpars-bukrs
            importing
                ed_active = ms_progpars-newgl_integration_active.
    endif.
    "Get leading ledger
    ms_progpars-jv_ledger = mo_settings->get_ledger( ).
    get_ledgers( ms_progpars-bukrs ).
    get_currencies( ms_progpars-bukrs ).
    get_classic_currencies( ms_progpars-bukrs ).
    "Make sure that the JVA on ACDOCA is always checked as on
    cl_jva_switch_injector=>inject_test_double( new cl_jva_switch_stub( ) ).
    "Check if there are migrated documents in ACDOCA
    select distinct mig_source, count( docln ) as line_count

```

```

        from acdoca
        where rbukrs      = @ms_progpars-bukrs
          and gjahr       in @ms_progpars-r_year
          and poper       in @ms_progpars-r_poper
          and mig_source is not initial
        group by mig_source
        order by mig_source
        into table @data(lt_migsource_count)
        bypassing buffer.
if lines( lt_migsource_count ) > 0.
    mv_has_migrated_docs = abap_true.
    read table lt_migsource_count into data(ls_migsource_count)
                                with key mig_source = 'J'.
    if sy-subrc = 0 and ls_migsource_count-line_count > 0.
        mv_has_migrated_jv_docs = abap_true.
    endif.
endif.
"Check if the S4 conversion was run with JVA being integrated
mv_jva_integrated = cl_jva_util=>get_special_setting(
                                iv_bukrs   = ms_progpars-bukrs
                                iv_name     =
'INTEGRATION_IN_S4HANA_MIGRATION_ACTIVE'
                                iv_default = abap_true ).

endmethod.
```

GET_CURRENCIES

Description:
Instance mthd

Importing parameter

IV_BUKRS TYPE BUKRS

```

method get_currencies.
    loop at mt_ledgers assigning field-symbol(<ls_ledger>).
        <ls_ledger>-s_curr_info = cl_fins_acdoc_util=>get_currency_conversion_info(
                                iv_bukrs = iv_bukrs
                                iv_rldnr = <ls_ledger>-rldnr ).

        if <ls_ledger>-rldnr = ms_progpars-jv_ledger.
            ms_curr_info = <ls_ledger>-s_curr_info.
        endif.
    endloop.
endmethod.
```

GET_GLOBAL_SETTINGS

Description:
Instance mthd

Returning parameter

VALUE(RV_ERROR) TYPE ABAP_BOOL

```

method get_global_settings.
    data: lt_parameter type drf_t_spar.
    "Get time stamp for runtime measuring
    mv_timestamp_start = get_timestamp( ).
    if ms_progpars-parallel_processing = abap_true.
```

```
if ms_progpars-server_group is initial.
    rv_error = abap_true.
    message e002(spta) into mv_message_text.
    add_sy_message_to_results( changing cs_results = ms_results_global ).
    return.
endif.
endif.
rv_error = check_cocodes_and_authorities( ).
if rv_error = abap_true.
    return.
endif.
rv_error = get_split_criteria( ).
if rv_error = abap_true.
    return.
endif.
if ms_progpars-bulk_no_of_doc_headers_in_mem < 100 or
    ms_progpars-bulk_no_of_doc_headers_in_mem > 9999999.
    ms_progpars-bulk_no_of_doc_headers_in_mem = 500000.
endif.
if ms_progpars-bulk_no_of_docs_for_line_proc < 1 or
    ms_progpars-bulk_no_of_docs_for_line_proc > 9999.
    ms_progpars-bulk_no_of_docs_for_line_proc = 500.
endif.
if ms_progpars-log_bulk_no_of_itms_for_db_upd < 1 or
    ms_progpars-log_bulk_no_of_itms_for_db_upd > 9999.
    ms_progpars-log_bulk_no_of_itms_for_db_upd = 1000.
endif.
if ms_progpars-maximum_number_of_tasks < 2 or
    ms_progpars-maximum_number_of_tasks > 99.
    ms_progpars-maximum_number_of_tasks = 20.
endif.
if ms_progpars-number_of_docs_per_task < 100 or
    ms_progpars-number_of_docs_per_task > 9999999.
    ms_progpars-number_of_docs_per_task = 1000000.
endif.
"Get time stamp for the log updates
ms_progpars-timestamp_db_updates = floor( get_timestamp( ) ).
prepare_period_range( ).
if ms_progpars-select-fi_documents = abap_true or
    ms_progpars-select-co_documents = abap_true.
    ms_progpars-r_activ_fico = prepare_activity_range-fi-co( ).
else.
    ms_progpars-r_activ_me = prepare_activity_range-me( ).
    ms_progpars-r_activ_op = prepare_activity_range-op( ).
endif.
"Posting date handling
if ms_progpars-use_new_posting_date is initial.
    clear ms_progpars-posting_date.
endif.
if sy-batch is initial.
    data(lv_count) = count_jv_documents_total( is_progpars = ms_progpars ).
    if lv_count > 100000.
        lt_parameter = value #( ( param = 'COUNT' value = lv_count ) ).
        mo_output->popup_to_confirm(
```

```

        exporting
            iv_titlebar      = 'Foreground Processing Despite Long Runtime'(fpd)
            iv_text_question = 'Do you really want to run the report in the
foreground?'(rrf)
                                && ` - ` && 'You have entered parameters to process
&COUNT& documents.'(nep)
                                && ` ` && 'The runtime of this report might therefore
be very long.'(lrt)
            iv_text_button_1 = 'Yes'(yes)
            iv_text_button_2 = 'No'(ano)
            iv_popup_type    = ''
            it_parameter      = lt_parameter
        importing
            ev_answer        = data(lv_answer)
            ev_subrc         = data(lv_subrc) ).
    if lv_subrc = 0.
        case lv_answer.
            when '1'.
                "Continue with the processing
            when others.
                rv_error = abap_true.
                return.
            endcase.
        endif.
    endif.
endif.
if sy-batch is not initial or
    ms_progpars-create_spools_in_foreground = abap_true.
    ms_progpars-print_results = abap_true.
else.
    if ms_progpars-parallel_processing = abap_true and
        ms_progpars-create_spool_for_each_task = abap_true.
        ms_progpars-print_results = abap_true.
    else.
        ms_progpars-print_results = abap_false.
    endif.
endif.
endmethod.

```

GET_LEDGERS

Description:

Instance mthd

Importing parameter

IV_BUKRS TYPE BUKRS

```

method get_ledgers.
    data: lt_ledgers type rldnr_t.
    "Temporary: Update the new field RLDNR_ACD in the log table
    update jva_acd_upd_log set rldnr_acd = @ms_progpars-jv_ledger
                                where rbukrs = @iv_bukrs
                                and rldnr_acd is initial.

    commit work.
    refresh: mt_ledgers, mr_ledgers.

```

```

    if ms_progpars-correct_all_ledgers <> abap_true.
      mt_ledgers = value #( ( rldnr = ms_progpars-jv_ledger ) ).
      mr_ledgers = value #( ( sign = 'I' option = 'EQ' low = ms_progpars-jv_ledger )
    ).
    return.
  endif.
  select rldnr from finsc_ld_cmp
    where bukrs = @iv_bukrs
    order by rldnr
    into table @lt_ledgers
    bypassing buffer.
  mt_ledgers = value #( ( rldnr = ms_progpars-jv_ledger ) ).
  loop at lt_ledgers into data(lv_ledger).
    if lv_ledger <> ms_progpars-jv_ledger.
      mt_ledgers = value #( base mt_ledgers
        ( rldnr = lv_ledger ) ).
    endif.
    mr_ledgers = value #( base mr_ledgers
      ( sign = 'I' option = 'EQ' low = lv_ledger ) ).
  endloop.
endmethod.
```

GET_LOG_ITEM

Description:
Instance mthd

Importing parameter

IS_DOC TYPE GTY_S_DOC
IT_CORR_LOG TYPE GTY_T_CORR_LOG

Returning parameter

VALUE(RS_CORR_LOG) TYPE JVA_ACD_UPD_LOG

```

method get_log_item.
  read table it_corr_log into rs_corr_log
    with key rldnr = is_doc-rldnr
           rbukrs = is_doc-rbukrs
           ryear  = is_doc-ryear
           docnr  = is_doc-docnr
           "docln = is_doc-docln
    binary search.
endmethod.
```

GET_SPLIT_CRITERIA

Description:
Instance mthd

Returning parameter

VALUE(RV_ERROR) TYPE ABAP_BOOL

```

method get_split_criteria.
  "Check if the GL splitter is active
  try.
    cl_fagl_split_services=>get_active_cc(
      exporting
```

```
        it_cc      = ms_progpars-t_bukrs
        id_budat   = sy-datum
        receiving
        rt_cc      = data(lt_cc) ).
    if lt_cc[] is not initial.
        data(lv_splitter_active) = abap_true.
    endif.
    catch cx_root.                "#EC NEED_CX_ROOT
        lv_splitter_active = abap_false.
    endtry.
    if lv_splitter_active <> abap_true.
        rv_error = abap_true.
        message e826(g5) into mv_message_text.
        add_sy_message_to_results( changing cs_results = ms_results_global ).
        return.
    endif.
    "Get the split criteria
    select * from fagl_split_field
           into table @data(lt_split_criteria)
           order by field.
    "Check if the venture is defined as split criteria
    read table lt_split_criteria transporting no fields
           with key field = 'VNAME'
           binary search.

    if sy-subrc <> 0.
        rv_error = abap_true.
        message e827(g5) into mv_message_text.
        add_sy_message_to_results( changing cs_results = ms_results_global ).
        return.
    endif.
    * ms_co_split_criteria-all_cost_objects = abap_true.
    * ms_co_split_criteria-no_cost_object = abap_true.
    *
    * "Check if the cost center is defined as split criteria
    * read table lt_split_criteria transporting no fields
    *           with key field = 'KOSTL'
    *           binary search.
    *
    * if sy-subrc = 0.
    *     ms_co_split_criteria-cost_center = abap_true.
    *     ms_co_split_criteria-no_cost_object = abap_false.
    * else.
    *     ms_co_split_criteria-all_cost_objects = abap_false.
    * endif.
    *
    * "Check if the order is defined as split criteria
    * read table lt_split_criteria transporting no fields
    *           with key field = 'AUFNR'
    *           binary search.
    *
    * if sy-subrc = 0.
    *     ms_co_split_criteria-order = abap_true.
    *     ms_co_split_criteria-no_cost_object = abap_false.
    * else.
    *     ms_co_split_criteria-all_cost_objects = abap_false.
    * endif.
```

```
*
*   "Check if the project/WBS element is defined as split criteria
*   read table lt_split_criteria transporting no fields
*                                   with key field = 'PS_PSP_PNR'
*                                   binary search.
*
*   if sy-subrc = 0.
*       ms_co_split_criteria-wbs_element = abap_true.
*       ms_co_split_criteria-no_cost_object = abap_false.
*   else.
*       ms_co_split_criteria-all_cost_objects = abap_false.
*   endif.
*
*   "Check if the network is defined as split criteria
*   read table lt_split_criteria transporting no fields
*                                   with key field = 'NPLNR'
*                                   binary search.
*
*   if sy-subrc = 0.
*       ms_co_split_criteria-network = abap_true.
*       ms_co_split_criteria-no_cost_object = abap_false.
*   else.
*       ms_co_split_criteria-all_cost_objects = abap_false.
*   endif.
*   endmethod.
```

GET_TIMESTAMP

Description:
Instance mthd

Returning parameter

```
VALUE(RV_TIME_STAMP) TYPE TIMESTAMPL

method get_timestamp.
  get time stamp field rv_time_stamp.
endmethod.
```

IS_CRITICAL_ERROR

Description:
Instance mthd

Importing parameter

```
IV_MSG_TYPE TYPE CHAR1
```

Returning parameter

```
VALUE(RV_RESULT) TYPE ABAP_BOOL

method is_critical_error.
  case iv_msg_type.
    when gc_error_type-shortdump.
      rv_result = abap_true.
    when gc_error_type-abend.
      rv_result = abap_true.
    when gc_error_type-error.
      rv_result = abap_true.
    when others.
      rv_result = abap_false.
```

```
endcase.  
endmethod.
```

JV_CURTP_EXISTS_AS_GL_CURTP

Description:
Instance mthd

Importing parameter

```
IV_CURTP_JV TYPE CURTP
```

Returning parameter

```
VALUE(RV_EXISTS) TYPE ABAP_BOOL
```

```
method jv_curtp_exists_as_gl_curtp.  
  if ms_curr_info-curtype1 is not initial and  
    iv_curtp_jv = ms_curr_info-curtype1.  
    rv_exists = abap_true.  
  elseif ms_curr_info-curtype2 is not initial and  
    iv_curtp_jv = ms_curr_info-curtype2.  
    rv_exists = abap_true.  
  elseif ms_curr_info-curtype3 is not initial and  
    iv_curtp_jv = ms_curr_info-curtype3.  
    rv_exists = abap_true.  
  elseif ms_curr_info-curtype4 is not initial and  
    iv_curtp_jv = ms_curr_info-curtype4.  
    rv_exists = abap_true.  
  elseif ms_curr_info-curtype5 is not initial and  
    iv_curtp_jv = ms_curr_info-curtype5.  
    rv_exists = abap_true.  
  elseif ms_curr_info-curtype6 is not initial and  
    iv_curtp_jv = ms_curr_info-curtype6.  
    rv_exists = abap_true.  
  elseif ms_curr_info-curtype7 is not initial and  
    iv_curtp_jv = ms_curr_info-curtype7.  
    rv_exists = abap_true.  
  elseif ms_curr_info-curtype8 is not initial and  
    iv_curtp_jv = ms_curr_info-curtype8.  
    rv_exists = abap_true.  
  elseif ms_curr_info-curtype9 is not initial and  
    iv_curtp_jv = ms_curr_info-curtype9.  
    rv_exists = abap_true.  
  elseif ms_curr_info-curtype10 is not initial and  
    iv_curtp_jv = ms_curr_info-curtype10.  
    rv_exists = abap_true.  
  endif.  
endmethod.
```

MAKE_WHERE_CLAUSE

Description:
Instance mthd

Importing parameter

```
IS_JVS0 TYPE JVS0  
IV_RLDNR TYPE RLDNR  
IV_IS_CO_DOC TYPE ABAP_BOOL
```

```

IT_JVACTIVITY TYPE GTY_T_JVACTIVITY
IV_SEARCH_WITH_ACCOUNT TYPE ABAP_BOOL
IV_SEARCH_WITH_AWORG TYPE ABAP_BOOL
IV_SEARCH_WITH_ACD_ONLY TYPE ABAP_BOOL
IV_SEARCH_WITH_POSTDATE TYPE ABAP_BOOL
IV_SEARCH_WITH_DOCDATE TYPE ABAP_BOOL
IV_SEARCH_WITH_PRODPER TYPE ABAP_BOOL
IV_SEARCH_WITH_RECID TYPE ABAP_BOOL
IV_PARTNER_AS_CUSTOMER TYPE ABAP_BOOL
IV_SEARCH_WITH_EGRUP TYPE ABAP_BOOL
IV_SEARCH_WITH_RBUSA TYPE ABAP_BOOL
IV_SEARCH_PREV_PERIODS TYPE ABAP_BOOL
IV_SEARCH_WITH_SGTXT TYPE ABAP_BOOL

```

Exporting parameter

```
ET_WHERE_CLAUSE TYPE GTY_T_WHERE_CONDITION
```

```
method make_where_clause.
```

```

data: lv_where_clause type char100,
      lv_text          type char100,
      lv_sus_year      type gjahr,
      lv_sus_period    type poper,
      lv_object_type   type char2.

```

```
clear et_where_clause.
```

```
lv_where_clause = |rldnr = '{ iv_rldnr }'| ##NO_TEXT.
```

```
append lv_where_clause to et_where_clause.
```

```
lv_where_clause = |AND rbukrs = '{ is_jvso-rbukrs }'| ##NO_TEXT.
```

```
append lv_where_clause to et_where_clause.
```

```
if is_jvso-rkokrs is not initial.
```

```
lv_where_clause = |AND kokrs = '{ is_jvso-rkokrs }'| ##NO_TEXT.
```

```
append lv_where_clause to et_where_clause.
```

```
endif.
```

```
if iv_search_prev_periods = abap_true.
```

```

lv_where_clause = |AND ( ( gjahr = '{ is_jvso-ryear }' AND poper <= '{
is_jvso-poper }' )| ##NO_TEXT.

```

```
append lv_where_clause to et_where_clause.
```

```

lv_where_clause = | OR ( gjahr < '{ is_jvso-ryear }' AND poper >= '001' AND
popper <= '012' )| ##NO_TEXT.

```

```
append lv_where_clause to et_where_clause.
```

```
lv_where_clause = ` `) ` ##NO_TEXT.
```

```
append lv_where_clause to et_where_clause.
```

```
else.
```

```

lv_where_clause = |AND gjahr = '{ is_jvso-ryear }' AND poper = '{ is_jvso-poper
}'| ##NO_TEXT.

```

```
append lv_where_clause to et_where_clause.
```

```
endif.
```

```
if iv_search_with_acd_only = abap_true.
```

```
"Identifier for direct ACDOCA postings made by JVA
```

```
lv_where_clause = `AND ( ` ##NO_TEXT.
```

```
append lv_where_clause to et_where_clause.
```

```
lv_where_clause = ` ( vorgn = 'RFRA' ` ##NO_TEXT.
```

```
append lv_where_clause to et_where_clause.
```

```
lv_where_clause = ` AND bstat = 'U' ` ##NO_TEXT.
```

```
append lv_where_clause to et_where_clause.
```

```
lv_where_clause = ` AND awtyp = 'BKPFF' ) ` ##NO_TEXT.
```

```
append lv_where_clause to et_where_clause.
lv_where_clause = ` OR ` ##NO_TEXT.
append lv_where_clause to et_where_clause.
"Identifier for migrated ACDOCA postings
lv_where_clause = ` ( mig_source = 'J' ` ##NO_TEXT.
append lv_where_clause to et_where_clause.
lv_where_clause = ` OR awtyp = 'JVAME' ` ##NO_TEXT.
append lv_where_clause to et_where_clause.
lv_where_clause = ` OR awtyp = 'JVAM' ) ` ##NO_TEXT.
append lv_where_clause to et_where_clause.
lv_where_clause = ` ) ` ##NO_TEXT.
append lv_where_clause to et_where_clause.
endif.
if it_jvactivity[] is not initial.
lv_where_clause = `AND (` .
loop at it_jvactivity into data(lv_jvactivity).
if sy-tabix = 1.
lv_where_clause &&= | jvactivity = '{ lv_jvactivity }'| ##NO_TEXT.
if sy-tabix = lines( it_jvactivity ).
lv_where_clause &&= ` ) ` .
endif.
append lv_where_clause to et_where_clause.
else.
lv_where_clause = | OR jvactivity = '{ lv_jvactivity }'| ##NO_TEXT.
if sy-tabix = lines( it_jvactivity ).
lv_where_clause &&= ` ) ` .
endif.
append lv_where_clause to et_where_clause.
endif.
endloop.
endif.
if iv_search_with_aworg = abap_true.
lv_where_clause = |AND aworg = '{ is_jvso-aworg }'| ##NO_TEXT.
append lv_where_clause to et_where_clause.
endif.
if iv_search_with_postdate = abap_true.
lv_where_clause = |AND budat = '{ is_jvso-budat }'| ##NO_TEXT.
append lv_where_clause to et_where_clause.
endif.
if iv_search_with_docdate = abap_true.
lv_where_clause = |AND bldat = '{ is_jvso-bldat }'| ##NO_TEXT.
append lv_where_clause to et_where_clause.
endif.
if iv_search_with_prodper = abap_true.
lv_where_clause = |AND prodper = '{ is_jvso-prodper }'| ##NO_TEXT.
append lv_where_clause to et_where_clause.
endif.
lv_where_clause = |AND prctr = '{ is_jvso-rprctr }'| ##NO_TEXT.
append lv_where_clause to et_where_clause.
if iv_search_with_account = abap_true.
lv_where_clause = |AND racct = '{ is_jvso-racct }'| ##NO_TEXT.
append lv_where_clause to et_where_clause.
endif.
if iv_search_with_rbusa = abap_true.
```

```
lv_where_clause = |AND rbusa = '{ is_jvso-rbusa }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
endif.  
lv_where_clause = |AND vname = '{ is_jvso-rjvnam }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
if iv_search_with_egrup = abap_true.  
lv_where_clause = |AND egrup = '{ is_jvso-regrou }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
endif.  
if iv_search_with_recid = abap_true.  
lv_where_clause = |AND recid = '{ is_jvso-rrecin }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
endif.  
if is_jvso-rpartn is not initial.  
if iv_partner_as_customer = abap_true.  
lv_where_clause = |AND kunnr = '{ is_jvso-rpartn }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
else.  
lv_where_clause = |AND vptnr = '{ is_jvso-rpartn }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
endif.  
endif.  
lv_where_clause = |AND rcntr = '{ is_jvso-rcntr }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
lv_where_clause = |AND aufnr = '{ is_jvso-rordnr }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
lv_where_clause = |AND ps_psp_pnr = '{ is_jvso-rprojk }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
lv_where_clause = |AND nplnr = '{ is_jvso-nplnr }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
lv_where_clause = |AND runit = '{ is_jvso-runit }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
lv_where_clause = |AND msl = '{ is_jvso-msl }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
lv_where_clause = |AND mwszk = '{ is_jvso-mwszk }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
lv_where_clause = |AND matnr = '{ is_jvso-matnr }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
lv_where_clause = |AND anln1 = '{ is_jvso-ranln1 }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
lv_where_clause = |AND anln2 = '{ is_jvso-ranln2 }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
lv_where_clause = |AND anbwa = '{ is_jvso-ranbwa }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
if iv_is_co_doc = abap_true.  
lv_where_clause = |AND co_buzei = '{ is_jvso-refdocln+2(3) }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
else.  
lv_where_clause = `AND co_buzei = ''` ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
endif.  
lv_where_clause = |AND wsl = '{ is_jvso-tsl }'| ##NO_TEXT.  
append lv_where_clause to et_where_clause.  
lv_where_clause = |AND hsl = '{ is_jvso-hsl }'| ##NO_TEXT.
```

```
append lv_where_clause to et_where_clause.
"Special handling for text field
if iv_search_with_sgtxt = abap_true and
  not is_jvso-sgtxt is initial.
  lv_text = is_jvso-sgtxt.
  replace all occurrences of `` in lv_text with `` ##NO_TEXT.
  lv_where_clause = |AND sgtxt = '{ lv_text }'| ##NO_TEXT.
  append lv_where_clause to et_where_clause.
endif.
endmethod.
```

MAKE_REFERENCE_CONDITION

Description:
Instance mthd

Importing parameter

IS_PROGPARS TYPE IF_JVA_MIG_CORRECT_ACDOCA~GTY_PROGRAM_PARAMETERS

Returning parameter

VALUE(LT_CONDITION) TYPE CL_JVA_MIG_CORRECT_ACDOCA=>GTY_T_WHERE_CONDITION

```
method make_reference_condition.
  if is_progpars-select_fi_documents = abap_true and
    is_progpars-select_co_documents = abap_true.
    lt_condition = value #(
      ( 'reffidoc in @ms_progpars-r_belnr      and' )
      ( 'refdocnr in @ms_progpars-r_refnr      and' )
      ( '( activ   = @gc_jvlinetype-fi_doc    or' )
      ( '  activ   = @gc_jvlinetype-fi_doc_mm or' )
      ( '    activ   = @gc_jvlinetype-co_doc )' )
    ).
  elseif is_progpars-select_fi_documents = abap_true.
    lt_condition = value #(
      ( 'reffidoc in @ms_progpars-r_belnr      and' )
      ( 'refdocnr in @ms_progpars-r_refnr      and' )
      ( '( activ   = @gc_jvlinetype-fi_doc    or' )
      ( '  activ   = @gc_jvlinetype-fi_doc_mm )' )
    ).
  elseif is_progpars-select_co_documents = abap_true.
    lt_condition = value #(
      ( 'reffidoc in @ms_progpars-r_belnr and' )
      ( 'refdocnr in @ms_progpars-r_refnr and' )
      ( 'activ      = @gc_jvlinetype-co_doc' )
    ).

  endif.
endmethod.
```

MULTIPLE_ACD_LINE_PROCESSING

Description:
Instance mthd

Importing parameter

IS_DOC TYPE GTY_S_DOC
IS_JVSO TYPE JVSO
IT_ACDOCA_SEL TYPE ACDOCA_JV_T

```

IT_ACDOCA_ORIGINAL TYPE ACDOCA_JV_T
IS_LINE_ASSIGNMENTS TYPE GTY_S_LINE_ASSIGNMENTS
IV_DO_HEURISTIC TYPE ABAP_BOOL
IV_COMPRESS_BY_PC TYPE ABAP_BOOL
IV_RECURSION_LEVEL TYPE I OPTIONAL
VALUE(IT_ACD_COMPR_INFO) TYPE GTY_T_ACD_COMPR_INFO OPTIONAL

```

Exporting parameter

```
VALUE(ET_ACD_COMPR_INFO) TYPE GTY_T_ACD_COMPR_INFO
```

Changing parameter

```
CS_LOG TYPE GTY_S_LOG
```

Returning parameter

```
VALUE(RT_ACDOCA) TYPE ACDOCA_JV_T
```

```
method multiple_acd_line_processing.
```

```

data: lt_acdoca_temp      type acdoca_jv_t,
      lt_acdoca_aggr      type acdoca_jv_t,
      lt_acdoca_compr     type acdoca_jv_t,
      lv_docln_acd        type acdoca-docln,
      ls_acd_compr_info   type gty_s_acd_compr_info,
      lv_tabix_hit        type sy-tabix,
      lt_docln            type gty_s_acd_compr_info-t_docln_original,
      lv_pos_count        type i,
      lv_neg_count        type i.

```

```
clear et_acd_compr_info.
```

```
"If the JV lines contains both positive and negative amounts,
```

```
"the search needs to be executed separately for each case
```

```
if is_jvso-tsl > 0. add 1 to lv_pos_count. endif.
```

```
if is_jvso-hsl > 0. add 1 to lv_pos_count. endif.
```

```
if is_jvso-ksl > 0. add 1 to lv_pos_count. endif.
```

```
if is_jvso-hsl_4c > 0. add 1 to lv_pos_count. endif.
```

```
if is_jvso-ksl_4c > 0. add 1 to lv_pos_count. endif.
```

```
if is_jvso-tsl < 0. add 1 to lv_neg_count. endif.
```

```
if is_jvso-hsl < 0. add 1 to lv_neg_count. endif.
```

```
if is_jvso-ksl < 0. add 1 to lv_neg_count. endif.
```

```
if is_jvso-hsl_4c < 0. add 1 to lv_neg_count. endif.
```

```
if is_jvso-ksl_4c < 0. add 1 to lv_neg_count. endif.
```

```
if lv_pos_count > 0 and lv_neg_count > 0.
```

```
data(ls_jvso_pos) = is_jvso.
```

```
if ls_jvso_pos-tsl < 0. clear ls_jvso_pos-tsl. endif.
```

```
if ls_jvso_pos-hsl < 0. clear ls_jvso_pos-hsl. endif.
```

```
if ls_jvso_pos-ksl < 0. clear ls_jvso_pos-ksl. endif.
```

```
if ls_jvso_pos-hsl_4c < 0. clear ls_jvso_pos-hsl_4c. endif.
```

```
if ls_jvso_pos-ksl_4c < 0. clear ls_jvso_pos-ksl_4c. endif.
```

```
data(ls_jvso_neg) = is_jvso.
```

```
if ls_jvso_neg-tsl > 0. clear ls_jvso_neg-tsl. endif.
```

```
if ls_jvso_neg-hsl > 0. clear ls_jvso_neg-hsl. endif.
```

```
if ls_jvso_neg-ksl > 0. clear ls_jvso_neg-ksl. endif.
```

```
if ls_jvso_neg-hsl_4c > 0. clear ls_jvso_neg-hsl_4c. endif.
```

```
if ls_jvso_neg-ksl_4c > 0. clear ls_jvso_neg-ksl_4c. endif.
```

```
if lv_pos_count >= lv_neg_count.
```

```

rt_acdoca = multiple_acd_line_processing(
    exporting
    is_doc          = is_doc

```

```

        is_jvso      = ls_jvso_pos
        it_acdoca_sel = it_acdoca_sel
        it_acdoca_original = it_acdoca_original
        is_line_assignments = is_line_assignments
        iv_do_heuristic = iv_do_heuristic
        iv_compress_by_pc = iv_compress_by_pc
        iv_recursion_level = iv_recursion_level
        it_acd_compr_info = it_acd_compr_info
    importing
        et_acd_compr_info = et_acd_compr_info
    changing
        cs_log            = cs_log ).
endif.
if lv_neg_count > lv_pos_count.
    rt_acdoca = multiple_acd_line_processing(
        exporting
            is_doc      = is_doc
            is_jvso      = ls_jvso_neg
            it_acdoca_sel = it_acdoca_sel
            it_acdoca_original = it_acdoca_original
            is_line_assignments = is_line_assignments
            iv_do_heuristic = iv_do_heuristic
            iv_compress_by_pc = iv_compress_by_pc
            iv_recursion_level = iv_recursion_level
            it_acd_compr_info = it_acd_compr_info
        importing
            et_acd_compr_info = et_acd_compr_info
        changing
            cs_log            = cs_log ).
endif.
return.
endif.
"Check if there is exactly one line with the right amounts
loop at it_acdoca_sel into data(ls_acdoca)
    where wsl      = is_jvso-tsl
    and hsl        = is_jvso-hsl
    and ksl        = is_jvso-ksl
    and hsl_4c     = is_jvso-hsl_4c
    and ksl_4c     = is_jvso-ksl_4c.
if is_jvso-activ <> gc_jvlinetype-balancing_line and
is_jvso-refactiv <> gc_jvlinetype-balancing_line.
    if is_jvso-racct <> is_jvso-racct.
        continue.
    endif.
endif.
"Check if the ACDOCA line was already assigned to another JV doc line
data(lv_already_assigned) = abap_false.
loop at is_line_assignments-jv_acd assigning field-symbol(<ls_jv_acd>)
    where docnr = is_jvso-docnr
    and docln <> is_jvso-docln.
    loop at <ls_jv_acd>-t_acd_docln transporting no fields
        where table_line = ls_acdoca-docln.
        lv_already_assigned = abap_true.
    exit.

```

```
        endloop.
        if lv_already_assigned = abap_true.
            exit.
        endif.
    endloop.
    if lv_already_assigned = abap_false.
        lv_tabix_hit = sy-tabix.
        append ls_acdoca to lt_acdoca_temp.
    endif.
endloop.
if lines( lt_acdoca_temp ) = 1.
    rt_acdoca = return_multiple_lines_hit( exporting
        iv_recursion_level =
it_acdoca_sel            = lt_acdoca_temp
it_acdoca_original      =
it_acdoca_original
        iv_tabix_hit      = lv_tabix_hit
it_acd_compr_info       = it_acd_compr_info
        changing
        cs_log            = cs_log ).
elseif lines( lt_acdoca_temp ) > 1.
    "If more than one item was retrieved, check if there is
    "exactly one line with the right amounts and the right profit center
    clear lt_acdoca_temp.
    loop at it_acdoca_sel into ls_acdoca
        where wsl      = is_jvso-tsl
        and hsl      = is_jvso-hsl
        and ksl      = is_jvso-ksl
        and hsl_4c    = is_jvso-hsl_4c
        and ksl_4c    = is_jvso-ksl_4c
        and racct     = is_jvso-racct
        and prctr     = is_jvso-rprctr.

        lv_tabix_hit = sy-tabix.
        append ls_acdoca to lt_acdoca_temp.
    endloop.
    if lines( lt_acdoca_temp ) = 1.
        rt_acdoca = return_multiple_lines_hit( exporting
            iv_recursion_level =
it_acdoca_sel            = lt_acdoca_temp
it_acdoca_original      =
it_acdoca_original
            iv_tabix_hit      = lv_tabix_hit
it_acd_compr_info       =
it_acd_compr_info
            changing
            cs_log            = cs_log ).
    else.
        "Check if there is one item with the right amount
        "with the right amounts, profit center, asset data and cost objects
        "The cost objects have to be linked with OR because the ACDOCA line can have
        "more than one cost object (statistical cost objects)
        clear lt_acdoca_temp.
```

```

loop at it_acdoca_sel into ls_acdoca
    where wsl      = is_jvso-tsl
    and hsl      = is_jvso-hsl
    and ksl      = is_jvso-ksl
    and hsl_4c   = is_jvso-hsl_4c
    and ksl_4c   = is_jvso-ksl_4c
    and racct    = is_jvso-racct
    and prctr    = is_jvso-rprctr
    and anln1    = is_jvso-ranln1
    and anln2    = is_jvso-ranln2
    and anbwa    = is_jvso-ranbwa.
if ( is_jvso-rcntr is not initial and
    ls_acdoca-rcntr = is_jvso-rcntr ) or
( is_jvso-rordnr is not initial and
  ls_acdoca-aufnr = is_jvso-rordnr ) or
( is_jvso-nplnr is not initial and
  ls_acdoca-nplnr = is_jvso-nplnr ) or
( is_jvso-rprojk is not initial and
  ls_acdoca-ps_psp_pnr = is_jvso-rprojk ).
lv_tabix_hit = sy-tabix.
append ls_acdoca to lt_acdoca_temp.
endif.
endloop.
if lines( lt_acdoca_temp ) = 1.
    rt_acdoca = return_multiple_lines_hit( exporting
iv_recursion_level                                iv_recursion_level =
lt_acdoca_temp                                    it_acdoca_sel      =
it_acdoca_original                               it_acdoca_original =
iv_tabix_hit                                     iv_tabix_hit      = lv_tabix_hit
it_acd_compr_info                               it_acd_compr_info =
changing
cs_log                                           = cs_log ).
else.
    "If more than one item was retrieved, check if there is
    "exactly one line with the right amounts and the right venture
    clear lt_acdoca_temp.
    loop at it_acdoca_sel into ls_acdoca
        where wsl      = is_jvso-tsl
        and hsl      = is_jvso-hsl
        and ksl      = is_jvso-ksl
        and hsl_4c   = is_jvso-hsl_4c
        and ksl_4c   = is_jvso-ksl_4c
        and racct    = is_jvso-racct
        and vname    = is_jvso-rjvnam.

        lv_tabix_hit = sy-tabix.
        append ls_acdoca to lt_acdoca_temp.
    endloop.
    if lines( lt_acdoca_temp ) = 1.
        rt_acdoca = return_multiple_lines_hit( exporting
iv_recursion_level                                iv_recursion_level =

```

```

                                it_acdoca_sel      =
lt_acdoca_temp                                it_acdoca_original =
it_acdoca_original                                iv_tabix_hit      =
lv_tabix_hit                                it_acd_compr_info  =
it_acd_compr_info                                changing
                                                cs_log          = cs_log ).

elseif lines( lt_acdoca_temp ) > 1.
    "If more than one item was retrieved, check if there is a line
    "with the right amounts, venture, and equity group
    clear lt_acdoca_temp.
    loop at it_acdoca_sel into ls_acdoca
        where wsl      = is_jvso-tsl
          and hsl      = is_jvso-hsl
          and ksl      = is_jvso-ksl
          and hsl_4c   = is_jvso-hsl_4c
          and ksl_4c   = is_jvso-ksl_4c
          and racct    = is_jvso-racct
          and vname    = is_jvso-rjvnam
          and egrup    = is_jvso-regrou.

        lv_tabix_hit = sy-tabix.
        append ls_acdoca to lt_acdoca_temp.
    endloop.
    if lines( lt_acdoca_temp ) = 1.
        rt_acdoca = return_multiple_lines_hit( exporting
                                                iv_recursion_level =
iv_recursion_level                                it_acdoca_sel      =
lt_acdoca_temp                                it_acdoca_original =
it_acdoca_original                                iv_tabix_hit      =
lv_tabix_hit                                it_acd_compr_info  =
it_acd_compr_info                                changing
                                                cs_log          = cs_log ).

elseif lines( lt_acdoca_temp ) > 1.
    "If more than one item was retrieved, check if there is a line
    "with the right amounts, venture, equity group, and recovery indicators
    clear lt_acdoca_temp.
    loop at it_acdoca_sel into ls_acdoca
        where wsl      = is_jvso-tsl
          and hsl      = is_jvso-hsl
          and ksl      = is_jvso-ksl
          and hsl_4c   = is_jvso-hsl_4c
          and ksl_4c   = is_jvso-ksl_4c
          and racct    = is_jvso-racct
          and vname    = is_jvso-rjvnam
          and egrup    = is_jvso-regrou
          and recid    = is_jvso-rrecin.

```

```

        lv_tabix_hit = sy-tabix.
        append ls_acdoca to lt_acdoca_temp.
    endloop.
    if lines( lt_acdoca_temp ) = 1.
        rt_acdoca = return_multiple_lines_hit( exporting
            iv_recursion_level =
iv_recursion_level
            it_acdoca_sel      =
lt_acdoca_temp
            it_acdoca_original =
it_acdoca_original
            iv_tabix_hit      =
lv_tabix_hit
            it_acd_compr_info =
it_acd_compr_info
            changing
            cs_log            = cs_log
        ).
    else.
        "Check if there is one item with the right amount
        "with the right amounts, venture, equity group and profit center
        clear lt_acdoca_temp.
        loop at it_acdoca_sel into ls_acdoca
            where wsl      = is_jvso-tsl
            and hsl        = is_jvso-hsl
            and ksl        = is_jvso-ksl
            and hsl_4c     = is_jvso-hsl_4c
            and ksl_4c     = is_jvso-ksl_4c
            and racct      = is_jvso-racct
            and vname      = is_jvso-rjvnam
            and egrup      = is_jvso-regrou
            and prctr      = is_jvso-rprctr.

            lv_tabix_hit = sy-tabix.
            append ls_acdoca to lt_acdoca_temp.
        endloop.
        if lines( lt_acdoca_temp ) = 1.
            rt_acdoca = return_multiple_lines_hit( exporting
                iv_recursion_level =
iv_recursion_level
                it_acdoca_sel      =
lt_acdoca_temp
                it_acdoca_original =
it_acdoca_original
                iv_tabix_hit      =
lv_tabix_hit
                it_acd_compr_info =
it_acd_compr_info
                changing
                cs_log            =
cs_log ).
        elseif lines( lt_acdoca_temp ) > 1.
            "Check if there is one item with the right amount
            "with the right amounts, venture, equity group, profit center,
            asset data and cost objects

```

can have "The cost objects have to be linked with OR because the ACDOCA line

```

"more than one cost object (statistical cost objects)
clear lt_acdoca_temp.
loop at it_acdoca_sel into ls_acdoca
    where wsl      = is_jvso-tsl
    and hsl      = is_jvso-hsl
    and ksl      = is_jvso-ksl
    and hsl_4c   = is_jvso-hsl_4c
    and ksl_4c   = is_jvso-ksl_4c
    and racct    = is_jvso-racct
    and vname    = is_jvso-rjvnam
    and egrup    = is_jvso-regrou
    and prctr    = is_jvso-rprctr
    and anln1    = is_jvso-ranln1
    and anln2    = is_jvso-ranln2
    and anbwa    = is_jvso-ranbwa.
    if ( is_jvso-rcntr is not initial and
        ls_acdoca-rcntr = is_jvso-rcntr ) or
        ( is_jvso-rordnr is not initial and
          ls_acdoca-aufnr = is_jvso-rordnr ) or
        ( is_jvso-nplnr is not initial and
          ls_acdoca-nplnr = is_jvso-nplnr ) or
        ( is_jvso-rprojk is not initial and
          ls_acdoca-ps_psp_pnr = is_jvso-rprojk ).
        lv_tabix_hit = sy-tabix.
        append ls_acdoca to lt_acdoca_temp.
    endif.
endloop.
if lines( lt_acdoca_temp ) = 1.
    data(lt_acdoca_temp2) = lt_acdoca_temp[].
elseif lines( lt_acdoca_temp ) > 1 and is_jvso-refdocct = 'K'.
    loop at lt_acdoca_temp into ls_acdoca.
        append ls_acdoca to lt_acdoca_temp2.
    exit.
endloop.
add_message_to_log(
    exporting
        iv_msg_text = 'Multiple ACDOCA lines found for JV doc
line'(mlf) && ` ` && condense( |{ is_jvso-docln alpha
        iv_msg_type = gc_error_type-warning
    changing
        cs_log      = cs_log ).
endif.
if lines( lt_acdoca_temp2 ) = 1.
    rt_acdoca = return_multiple_lines_hit( exporting
iv_recursion_level      iv_recursion_level =
lt_acdoca_temp2         it_acdoca_sel      =
it_acdoca_original      it_acdoca_original =
lv_tabix_hit            iv_tabix_hit      =

```

```

it_acd_compr_info      it_acd_compr_info  =
                        changing
cs_log ).              cs_log             =

                        endif.
                        endif.
                        endif.
                        endif.
                        endif.
                        endif.
                        endif.
if rt_acdoca[] is not initial.
  et_acd_compr_info[] = it_acd_compr_info[].
  return.
endif.
if ( is_doc-activ <> gc_jvlinetype-co_doc and
    is_doc-activ <> gc_jvlinetype-fi_doc and
    is_doc-activ <> gc_jvlinetype-fi_doc_mm ) or
  iv_do_heuristic = abap_false or
  lines( it_acdoca_sel ) <= 1 or
  ( iv_compress_by_pc = abap_false and
    iv_recursion_level >= 3 ) or
  ( iv_compress_by_pc = abap_true and
    iv_recursion_level >= 4 ).
  return.
endif.
"Try to aggregate the ACDOCA lines
loop at it_acdoca_sel assigning field-symbol(<ls_acdoca>).
  data(lv_tabix_sel) = sy-tabix.
  data(lv_docln) = <ls_acdoca>-docln.
  "Check if the line was already assigned to a JV doc line
  "If yes, it should take part in the compression
  read table is_line_assignments-acd_jv transporting no fields
                                with key docln = lv_docln
                                binary search.

  if sy-subrc = 0.
    continue.
  endif.
  ls_acdoca = <ls_acdoca>.
  if iv_recursion_level >= 0.
    clear: ls_acdoca-docln,
           ls_acdoca-timestamp,
           ls_acdoca-sgtxt.
  endif.
  if iv_recursion_level >= 1.
    clear: ls_acdoca-vname,
           ls_acdoca-egrup,
           ls_acdoca-recid,
           ls_acdoca-btype,
           ls_acdoca-vptnr,
           ls_acdoca-prodper.
  endif.

```

```
if iv_recursion_level >= 2.
  clear: ls_acdoca-co_belnr,
         ls_acdoca-awitem,
         ls_acdoca-co_buzei.
  if ls_acdoca-accasty <> 'KS' and
     ls_acdoca-accasty <> 'OR' and
     ls_acdoca-accasty <> 'OV' and
     ls_acdoca-accasty <> 'PR' and
     ls_acdoca-accasty <> 'NV' and
     ls_acdoca-accasty <> 'NP'.
    clear: ls_acdoca-accasty,
           ls_acdoca-rcntr,
           ls_acdoca-aufnr,
           ls_acdoca-ps_psp_pnr,
           ls_acdoca-nplnr,
           ls_acdoca-nplnr_vorgn,
           ls_acdoca-vornr,
           ls_acdoca-runit.
  endif.
  if iv_recursion_level >= 3 and
     iv_compress_by_pc = abap_true.
    clear: ls_acdoca-prctr.
  endif.
endif.
collect ls_acdoca into lt_acdoca_aggr.
data(lv_tabix_aggr) = sy-tabix.
clear lt_docln.
if lv_docln is not initial.
  append lv_docln to lt_docln.
else.
  read table it_acd_compr_info assigning field-symbol(<ls_acd_compr_info>)
                                with key tabix_aggr = lv_tabix_sel.

  if sy-subrc = 0 and
     <ls_acd_compr_info> is assigned.
    lt_docln[] = <ls_acd_compr_info>-t_docln_original[].
  endif.
endif.
"Save links between the aggregated item and the original items
"First check if entry already exist
read table et_acd_compr_info assigning <ls_acd_compr_info>
                                with key tabix_aggr = lv_tabix_aggr.

if sy-subrc = 0 and
   <ls_acd_compr_info> is assigned.
  append lines of lt_docln to <ls_acd_compr_info>-t_docln_original.
else.
  clear ls_acd_compr_info.
  ls_acd_compr_info-tabix_aggr = lv_tabix_aggr.
  ls_acd_compr_info-t_docln_original[] = lt_docln[].
  append ls_acd_compr_info to et_acd_compr_info.
endif.
endloop.
rt_acdoca = multiple_acd_line_processing(
  exporting
    is_doc          = is_doc
```

```

        is_jvso          = is_jvso
        it_acdoca_sel     = lt_acdoca_aggr
        it_acdoca_original = it_acdoca_original
        is_line_assignments = is_line_assignments
        iv_do_heuristic   = iv_do_heuristic
        iv_compress_by_pc = iv_compress_by_pc
        iv_recursion_level = iv_recursion_level + 1
        it_acd_compr_info  = et_acd_compr_info
importing
        et_acd_compr_info = et_acd_compr_info
changing
        cs_log            = cs_log ).

endmethod.
```

POST_MISSING_DOCUMENT

Description:

Instance mthd

Importing parameter

```

IS_DOC TYPE GTY_S_DOC
IT_DOC_COMB TYPE GTY_T_DOC
IT_JVSO_DOC TYPE JVSO_T
IV_LEDGER TYPE RLDNR
IV_POSTING_MODE TYPE CHAR1
```

Changing parameter

```

CT_LOG_DOC TYPE GTY_T_LOG
```

```

method post_missing_document.
  data: ls_jvso          type jvso,
        ls_acdoca        type acdoca_jv,
        lo_message_handler type ref to if_jva_message_collection,
        lv_critical_error type abap_bool,
        lv_belnr         type belnr_d,
        lv_gjahr         type gjahr,
        lv_msg_type      type char1,
        lv_message_text  type string,
        lv_updated       type abap_bool,
        lt_acdoca_doc    type acdoca_jv_t.
  if it_jvso_doc is initial.
    return.
  endif.
  if ms_progpars-analysis_only = abap_true.
    add_item_info_for_doc(
      iv_update      = abap_false
      is_doc         = is_doc
      it_doc_comb    = it_doc_comb
      it_log         = ct_log_doc
      it_acdoca_doc  = lt_acdoca_doc
      iv_msg_text    = space
      iv_msg_type    = gc_error_type-warning
      iv_updated     = lv_updated
      iv_belnr       = lv_belnr
      iv_gjahr       = lv_gjahr
      io_messages    = lo_message_handler ).
```

```
    return.
  endif.
  loop at it_jvso_doc into ls_jvso
    where activ <> gc_jvlinetype-balancing_line.
    exit.
  endloop.
  if ls_jvso-activ = gc_jvlinetype-cutback_operator or
    ls_jvso-activ = gc_jvlinetype-cashcall_operator.
    data(lo_post_fi_op) = cl_jva_mig_post_fi_op=>get_instance( ).
    lo_post_fi_op->post_document(
      exporting
        iv_test          = ms_progpars-testrun
        iv_activ         = ls_jvso-activ
        it_jvso          = it_jvso_doc
        iv_budat         = ms_progpars-posting_date
      importing
        ev_critical_error = lv_critical_error
        ev_belnr          = lv_belnr
        ev_gjahr          = lv_gjahr
      changing
        co_messages      = lo_message_handler ).
    if lv_critical_error <> abap_true.
      lv_updated = abap_true.
    endif.
  else.
    data(lo_post_acdoca) = cl_jva_mig_post_acdoca=>get_instance( ).
    lo_post_acdoca->post_new_acdoca_doc(
      exporting
        it_jvso_doc      = it_jvso_doc
        iv_ledger         = iv_ledger
        iv_posting_mode   = iv_posting_mode
        iv_budat         = ms_progpars-posting_date
        iv_test          = ms_progpars-testrun
        iv_jva_integrated = mv_jva_integrated
      changing
        cv_updated       = lv_updated
        cv_belnr         = lv_belnr
        cv_gjahr         = lv_gjahr
        co_messages      = lo_message_handler ).
    endif.
    if lv_updated = abap_true.
      lv_msg_type = gc_error_type-statement.
      if ms_progpars-testrun = abap_true.
        lv_message_text = 'New document tested successfully'(nst).
      else.
        lv_message_text = 'New document posted successfully'(nsu).
      endif.
    else.
      lv_msg_type = gc_error_type-error.
      if ms_progpars-testrun = abap_true.
        lv_message_text = 'ERROR when testing the new document'(etn).
      else.
        lv_message_text = 'ERROR when posting the new document'(epn).
      endif.
    endif.
  endif.
```

```
endif.  
add_item_info_for_doc(  
    iv_update      = abap_true  
    is_doc         = is_doc  
    it_doc_comb    = it_doc_comb  
    it_log         = ct_log_doc  
    it_acdoca_doc  = lt_acdoca_doc  
    iv_msg_text    = lv_message_text  
    iv_msg_type    = lv_msg_type  
    iv_updated     = lv_updated  
    iv_belnr       = lv_belnr  
    iv_gjahr       = lv_gjahr  
    io_messages    = lo_message_handler ).  
endmethod.
```

PREPARE_ACDOCA_BALANCES

Description:

Instance mthd

Importing parameter

IS_ACDOCA TYPE ACDOCA_JV

Changing parameter

CS_BALANCES TYPE GTY_S_BALANCES

```
method prepare_acdoca_balances.  
    data: ls_bal      type gty_s_balances_acdoca,  
          lv_last_day type sy-datum.  
    ls_bal-racct  = is_acdoca-racct.  
    if ms_progpars-check_venture = abap_true or  
       ms_progpars-check_jva_data = abap_true.  
        ls_bal-vname  = is_acdoca-vname.  
    endif.  
    if ms_progpars-check_equity_group = abap_true or  
       ms_progpars-check_jva_data = abap_true.  
        ls_bal-egrup  = is_acdoca-egrup.  
    endif.  
    if ms_progpars-check_venture_partner = abap_true or  
       ms_progpars-check_jva_data = abap_true.  
        ls_bal-vptnr  = is_acdoca-vptnr.  
    endif.  
    if ms_progpars-check_recovery_indicator = abap_true or  
       ms_progpars-check_jva_data = abap_true.  
        ls_bal-recid  = is_acdoca-recid.  
    endif.  
    if ms_progpars-check_billing_indicator = abap_true or  
       ms_progpars-check_jva_data = abap_true.  
        ls_bal-btype  = is_acdoca-btype.  
    endif.  
    if ( ms_progpars-check_production_period = abap_true or  
        ms_progpars-check_jva_data = abap_true ) and  
       mo_settings->ms_t8jz-pm_act = abap_true.  
        if ms_progpars-prodper_as_period = abap_true.  
            ls_bal-prodper = is_acdoca-prodper(6).  
        elseif ms_progpars-prodper_last_eq_first = abap_true.
```

```
call function 'LAST_DAY_OF_MONTHS'
  exporting
    day_in          = is_acdoca-prodper
  importing
    last_day_of_month = lv_last_day
  exceptions
    day_in_no_date    = 1
    others            = 2.
if sy-subrc = 0.
  if is_acdoca-prodper = lv_last_day.
    ls_bal-prodper = is_acdoca-prodper(6) && '01'.
  endif.
else.
  ls_bal-prodper = is_acdoca-prodper.
endif.
else.
  ls_bal-prodper = is_acdoca-prodper.
endif.
endif.
ls_bal-wsl      = is_acdoca-wsl.
ls_bal-hsl      = is_acdoca-hsl.
ls_bal-ksl      = is_acdoca-ksl.
ls_bal-hsl_4c   = is_acdoca-hsl_4c.
ls_bal-ksl_4c   = is_acdoca-ksl_4c.
collect ls_bal into cs_balances-t_acdoca_jv.
ls_bal-rcntr     = is_acdoca-rcntr.
ls_bal-aufnr     = is_acdoca-aufnr.
ls_bal-ps_psp_pnr = is_acdoca-ps_psp_pnr.
ls_bal-nplnr     = is_acdoca-nplnr.
collect ls_bal into cs_balances-t_acdoca_jv_co.
endmethod.
```

PREPARE_ACDOCA_DOCUMENT

Description:
Instance mthd

Changing parameter

CS_DOC TYPE GTY_S_DOC
CT_ACDOCA TYPE ACDOCA_JV_T
CS_BALANCES TYPE GTY_S_BALANCES

```
method prepare_acdoca_document.
  loop at ct_acdoca assigning field-symbol(<ls_acdoca>).
    data(lv_tabix) = sy-tabix.
    "Special handling for migrated documents: Delete irrelevant items
    if mv_has_migrated_jv_docs = abap_true.
      if <ls_acdoca>-mig_source is not initial and
        <ls_acdoca>-mig_source <> 'J'          and
        <ls_acdoca>-awtyp      <> 'JVAME'      and
        <ls_acdoca>-awtyp      <> 'JVAM'       and
        <ls_acdoca>-vname      is initial.
        delete ct_acdoca index lv_tabix.
        continue.
      endif.
```

```
elseif mv_has_migrated_docs = abap_true.
  if cs_doc-jvdoctype = gc_jvdoctype-fi_doc.
    if <ls_acdoca>-mig_source is not initial and
       <ls_acdoca>-mig_source <> 'G'           and
       <ls_acdoca>-vname      is initial.
      delete ct_acdoca index lv_tabix.
      continue.
    endif.
  endif.
endif.
if ms_progpars-exclude_balancing_lines = abap_true.
  "Delete the venture balancing lines for FI docs
  "and if it is a proper FI doc (BSTAT is initial)
  if cs_doc-jvdoctype = gc_jvdoctype-fi_doc or
     cs_doc-jvdoctype = gc_jvdoctype-cutback or
     cs_doc-jvdoctype = gc_jvdoctype-cash_call.
    if cs_doc-bstat is initial.
      if cs_doc-mig_source = 'J'.
        if <ls_acdoca>-awitem is initial.
          delete ct_acdoca index lv_tabix.
          cs_doc-gl_no_of_lines_excluded += 1.
          continue.
        endif.
      else.
        if <ls_acdoca>-buzei      is initial and
           <ls_acdoca>-co_buzei is initial and
           <ls_acdoca>-awitem    is initial.
          delete ct_acdoca index lv_tabix.
          cs_doc-gl_no_of_lines_excluded += 1.
          continue.
        endif.
      endif.
    endif.
  endif.
endif.
endif.
determine_jv_amnts_in_acd_line( changing cs_acdoca = <ls_acdoca> ).
if mv_delete_zero_amount_items = abap_true.
  if <ls_acdoca>-wsl      = 0 and
     <ls_acdoca>-hsl      = 0 and
     <ls_acdoca>-ksl      = 0 and
     <ls_acdoca>-hsl_4c = 0 and
     <ls_acdoca>-ksl_4c = 0.
    delete ct_acdoca index lv_tabix.
    continue.
  endif.
endif.
prepare_acdoca_balances( exporting
                        is_acdoca    = <ls_acdoca>
                        changing
                        cs_balances = cs_balances ).

endloop.
endmethod.
```

PREPARE_ACTIVITY_RANGE_FI_CO

Description:

Instance mthd

Returning parameter

VALUE(RR_ACTIV) TYPE GTY_T_ACTIV

```
method prepare_activity_range_fi_co.  
  if ms_progpars-select_fi_documents = abap_true.  
    rr_activ = value #(   
      (   
        sign    = 'I'  
        option = 'EQ'  
        low     = gc_jvlinetype-fi_doc  
      )  
      (   
        sign    = 'I'  
        option = 'EQ'  
        low     = gc_jvlinetype-fi_doc_mm  
      )  
    ).  
  endif.  
  if ms_progpars-select_co_documents = abap_true.  
    rr_activ = value #(   
      base rr_activ  
      (   
        sign    = 'I'  
        option = 'EQ'  
        low     = gc_jvlinetype-co_doc  
      )  
    ).  
  endif.  
endmethod.
```

PREPARE_ACTIVITY_RANGE_ME

Description:

Instance mthd

Returning parameter

VALUE(RR_ACTIV) TYPE GTY_T_ACTIV

```
method prepare_activity_range_me.  
  data: lr_activ like line of rr_activ.  
  lr_activ-sign    = 'I'.  
  lr_activ-option = 'EQ'.  
  if ms_progpars-post_equity_change = abap_true.  
    lr_activ-low = gc_jvlinetype-equity_change.  
    append lr_activ to rr_activ.  
  endif.  
  if ms_progpars-post_equity_adjust = abap_true.  
    lr_activ-low = gc_jvlinetype-eq_adjust_reversal.  
    append lr_activ to rr_activ.  
    lr_activ-low = gc_jvlinetype-eq_adjust_reposting.  
    append lr_activ to rr_activ.
```

```
endif.
if ms_progpars-post_farm_in_out = abap_true.
  lr_activ-low = gc_jvlinetype-farm_io_reversal.
  append lr_activ to rr_activ.
  lr_activ-low = gc_jvlinetype-farm_io_reposting.
  append lr_activ to rr_activ.
endif.
if ms_progpars-post_suspende = abap_true.
  lr_activ-low = gc_jvlinetype-suspende.
  append lr_activ to rr_activ.
  lr_activ-low = gc_jvlinetype-unsuspende.
  append lr_activ to rr_activ.
endif.
endmethod.
```

PREPARE_ACTIVITY_RANGE_OP

Description:
Instance mthd

Returning parameter

VALUE(RR_ACTIV) TYPE GTY_T_ACTIV

```
method prepare_activity_range_op.
  data: lr_activ like line of rr_activ.
  lr_activ-sign   = 'I'.
  lr_activ-option = 'EQ'.
  if ms_progpars-post_cutback = abap_true.
    lr_activ-low = gc_jvlinetype-cutback_operator.
    append lr_activ to rr_activ.
  endif.
  if ms_progpars-post_cash_call = abap_true.
    lr_activ-low = gc_jvlinetype-cashcall_operator.
    append lr_activ to rr_activ.
  endif.
endmethod.
```

PREPARE_JVSO_BALANCES

Description:
Instance mthd

Importing parameter

IS_JVSO TYPE JVSO

Changing parameter

CS_BALANCES TYPE GTY_S_BALANCES

```
method prepare_jvso_balances.
  data: ls_bal      type gty_s_balances_jvso,
        lv_last_day type sy-datum.
  ls_bal-racct   = is_jvso-racct.
  if ms_progpars-check_venture = abap_true or
     ms_progpars-check_jva_data = abap_true.
    ls_bal-rjvnam = is_jvso-rjvnam.
  endif.
  if ms_progpars-check_equity_group = abap_true or
```

```
    ms_progpars-check_jva_data = abap_true.
    ls_bal-regrou = is_jvso-regrou.
endif.
if ms_progpars-check_venture_partner = abap_true or
   ms_progpars-check_jva_data = abap_true.
    ls_bal-rpartn = is_jvso-rpartn.
endif.
if ms_progpars-check_recovery_indicator = abap_true or
   ms_progpars-check_jva_data = abap_true.
    ls_bal-rrecin = is_jvso-rrecin.
endif.
if ms_progpars-check_billing_indicator = abap_true or
   ms_progpars-check_jva_data = abap_true.
    ls_bal-bilid = is_jvso-bilid.
endif.
if ( ms_progpars-check_production_period = abap_true or
    ms_progpars-check_jva_data = abap_true ) and
   mo_settings->ms_t8jz-pm_act = abap_true.
    if ms_progpars-prodper_as_period = abap_true.
        ls_bal-prodper = is_jvso-prodper(6).
    elseif ms_progpars-prodper_last_eq_first = abap_true.
        call function 'LAST_DAY_OF_MONTHS'
            exporting
                day_in          = is_jvso-prodper
            importing
                last_day_of_month = lv_last_day
            exceptions
                day_in_no_date    = 1
                others            = 2.
        if sy-subrc = 0.
            if is_jvso-prodper = lv_last_day.
                ls_bal-prodper = is_jvso-prodper(6) && '01'.
            endif.
        else.
            ls_bal-prodper = is_jvso-prodper.
        endif.
    else.
        ls_bal-prodper = is_jvso-prodper.
    endif.
endif.
ls_bal-ksl = is_jvso-ksl.
ls_bal-hsl = is_jvso-hsl.
ls_bal-ksl_4c = is_jvso-ksl_4c.
ls_bal-hsl_4c = is_jvso-hsl_4c.
collect ls_bal into cs_balances-t_jvso_jv.
ls_bal-rcntr = is_jvso-rcntr.
ls_bal-rordnr = is_jvso-rordnr.
ls_bal-rprojk = is_jvso-rprojk.
ls_bal-nplnr = is_jvso-nplnr.
collect ls_bal into cs_balances-t_jvso_jv_co.
endmethod.
```

PROCESS_COMPANY_CODE

Description:

Instance mthd

Importing parameter

IS_PROGPARS TYPE IF_JVA_MIG_CORRECT_ACDOCA=>GTY_PROGRAM_PARAMETERS

```
method process_company_code.  
  ms_progpars = is_progpars.  
  clear ms_results.  
  ms_results-bukrs = ms_progpars-bukrs.  
  ms_results-s_progpars = ms_progpars.  
  data(lv_error) = get_company_code_settings( ).  
  if lv_error = abap_false.  
    retrieve_and_process_data( ).  
  endif.  
  update_log_db_table( ).  
endmethod.
```

PREPARE_PERIOD_RANGE

Description:

Instance mthd

```
method prepare_period_range.  
  if ms_progpars-r_poper is not initial.  
    return.  
  endif.  
  ms_progpars-r_poper = value #(  
    (  
      sign    = 'I'  
      option  = 'BT'  
      low     = '001'  
      high    = '012'  
    )  
  ).  
  
endmethod.
```

PROCESS_DATA_PARALLELLY

Description:

Instance mthd

```
method process_data_parallelly.  
  data: lv_subrc      type sy-subrc,  
        lv_tabix_results type sy-tabix,  
        ls_taskdata   type gty_s_taskdata,  
        lv_package_no type i,  
        ls_results_new like line of mt_results,  
        ls_task_info   like line of ls_results_new-t_tasks.  
  clear mt_taskdata.  
  data(ls_progpars) = ms_progpars.  
  clear ls_progpars-t_bukrs.  
  loop at ms_progpars-t_bukrs into data(lv_bukrs).  
    ls_progpars = ms_progpars.  
    ls_progpars-bukrs = lv_bukrs.
```

```

"Add one item per company code to the results table
read table mt_results assigning field-symbol(<ls_results>)
                        with key bukrs = lv_bukrs
                        binary search.
if sy-subrc <> 0 or <ls_results> is not assigned.
  ls_results_new-bukrs = ls_progpars-bukrs.
  ls_results_new-s_progpars = ls_progpars.
  ls_results_new-s_progpars-is_parallel_task = abap_true.
  insert ls_results_new into mt_results index sy-tabix
                        assigning <ls_results>.
endif.
data(lt_packages) = calculate_task_packages( ls_progpars ).
if lines( lt_packages ) = 0.
  message i708(gi) into mv_message_text.
  add_sy_message_to_results( changing cs_results = <ls_results> ).
  continue.
endif.
loop at lt_packages into data(ls_package).
  if ls_package-doc_count > 0.
    lv_package_no += 1.
    clear ls_taskdata.
    ls_taskdata-obj_id = ls_progpars-bukrs && `_` && lv_package_no.
    ls_progpars-is_parallel_task = abap_true.
    ls_progpars-s_task_package = ls_package.
    ls_taskdata-s_progpars = ls_progpars.
    append ls_taskdata to mt_taskdata.
    ls_taskinfo-obj_id = ls_taskdata-obj_id.
    ls_taskinfo-bukrs = ls_progpars-bukrs.
    ls_taskinfo-s_progpars = ls_progpars.
    append ls_taskinfo to <ls_results>-t_tasks.
  endif.
endloop.
sort <ls_results>-t_tasks by obj_id.
endloop.
sort mt_taskdata by obj_id.
if lines( mt_taskdata ) <= 1.
  "If there is only one task, there is no point in parallel processing
  ms_progpars-parallel_processing = abap_false.
  ms_results_global-s_progpars-parallel_processing = abap_false.
  if sy-batch is initial and
    ms_progpars-create_spools_in_foreground = abap_false.
    ms_progpars-print_results = abap_false.
  endif.
  clear: mt_results,
         ms_results_global-t_tasks.
  process_data_synchronously( ).
  return.
endif.
"Execute the parallel processing
mo_output->spta_para_process_start_2( exporting
                                     iv_server_group           =
ms_progpars-server_group
                                     iv_max_no_of_tasks        =
ms_progpars-maximum_number_of_tasks

```

```

                                iv_before_rfc_callback_form =
'BEFORE_RFC'
                                iv_in_rfc_callback_form      = 'IN_RFC'
                                iv_after_rfc_callback_form    = 'AFTER_RFC'
                                iv_callback_prog              =
ms_progpars-repid_callback
                                importing
                                ev_subrc                      = lv_subrc ).

if lv_subrc > 0.
  "Show parallelization error message
  case lv_subrc.
    when 1.
      message i719(shdb_pfw) with ms_progpars-server_group into mv_message_text.
    when 2.
      message i005(spta) with ms_progpars-server_group into mv_message_text.
    when 3.
      message i160(fcmlacc) into mv_message_text.
  endcase.
  add_sy_message_to_results( changing cs_results = ms_results_global ).
endif.
endmethod.

```

PROCESS_DATA_SYNCHRONOUSLY

Description:

Instance mthd

```

method process_data_synchronously.
  loop at ms_progpars-t_bukrs into data(lv_bukrs).
    data(lv_timestamp_start) = get_timestamp( ).
    ms_progpars-bukrs = lv_bukrs.
    clear ms_results.
    ms_results-bukrs = ms_progpars-bukrs.
    ms_results-s_progpars = ms_progpars.
    process_company_code( ms_progpars ).
    ms_results-runtime_cocode = calc_runtime( iv_timestamp_start =
lv_timestamp_start
                                iv_timestamp_end   = get_timestamp( )
).
    if ms_progpars-print_results = abap_true and
      ms_results-doc_count > 0.
      ms_results-t_spool = cl_jva_mig_alv_output=>get_instance( )->print_results(
is_results = ms_results ).
      clear ms_results-t_item_info.
      save_results( is_results = ms_results ).
    else.
      if lines( ms_progpars-t_bukrs ) > 1.
        save_results( is_results = ms_results ).
      endif.
    endif.
  endloop.
endmethod.

```

RETRIEVE_ACDOCA_LINES

Description:

Instance mthd

Importing parameter

IV_TYPE TYPE CHAR2
IT_DOC TYPE GTY_T_DOC OPTIONAL
IT_REF TYPE GTY_T_REF OPTIONAL
IT_CORR_LOG TYPE GTY_T_CORR_LOG OPTIONAL

Changing parameter

CT_ACDOCA TYPE ACDOCA_JV_T

```
method retrieve_acdoca_lines.  
  data: lt_cond_mig      type gty_t_where_condition,  
        lv_ledger_cond type string.  
  if mt_fields_acdoca[] is initial.  
    cl_jva_util=>get_selection_fields(  
      exporting  
        it_table      = ct_acdoca  
        iv_source_table = 'ACDOCA'  
        iv_aggr_amnts  = abap_false  
        iv_prefix      = space  
      changing  
        ct_sel_fields  = mt_fields_acdoca  
        ct_group_fields = mt_group_acdoca ).  
  endif.  
  if ms_progpars-correct_all_ledgers = abap_true.  
    lv_ledger_cond = 'rldnr in @mr_ledgers'.  
  else.  
    lv_ledger_cond = 'rldnr = @ms_progpars-jv_ledger'.  
  endif.  
  if mv_has_migrated_docs = abap_true.  
    if mv_has_migrated_jv_docs = abap_true.  
      "Do only retrieve items that were not migrated or where the venture is filled  
      lt_cond_mig = value #(   
                            ( ' ( mig_source is initial OR ' )  
                            ( '   vname is not initial )' )  
                            ).  
    else.  
      "Exclude the totals corrections  
      lt_cond_mig = value #(   
                            ( ' ( mig_source <> ''S'' AND ' )  
                            ( '   mig_source <> ''T'' AND ' )  
                            ( '   mig_source <> ''U'' AND ' )  
                            ( '   mig_source <> ''V'' )' )  
                            ).  
    endif.  
  endif.  
  case iv_type.  
    when 'FI'.  
      if it_doc[] is not initial.  
        select (mt_fields_acdoca) from acdoca  
          for all entries in @it_doc
```

```

                                where (lv_ledger_cond)
                                    and rbukrs = @ms_progpars-bukrs
                                    and gjahr  = @it_doc-ryear
                                    and belnr  = @it_doc-reffidoc
                                    and (lt_cond_mig)
                                appending corresponding fields of table
@ct_acdoca.
    endif.
    when 'CO'.
        if it_doc[] is not initial.
            select (mt_fields_acdoca) from acdoca
                                for all entries in @it_doc
                                where (lv_ledger_cond)
                                    and rbukrs  = @ms_progpars-bukrs
                                    and gjahr   = @it_doc-ryear
                                    and co_belnr = @it_doc-refdocnr
                                    and (lt_cond_mig)
                                appending corresponding fields of table
@ct_acdoca.
    endif.
    when '4A'.
        if it_doc[] is not initial.
            select (mt_fields_acdoca) from acdoca
                                for all entries in @it_doc
                                where (lv_ledger_cond)
                                    and rbukrs = @ms_progpars-bukrs
                                    and gjahr  = @it_doc-ryear
                                    and awtyp  = 'BKPFF'
                                    and zuonr  = @it_doc-zuonr
                                appending corresponding fields of table
@ct_acdoca.
    endif.
    when '4B'.
        if it_doc[] is not initial.
            select (mt_fields_acdoca) from acdoca
                                for all entries in @it_doc
                                where (lv_ledger_cond)
                                    and rbukrs = @ms_progpars-bukrs
                                    and gjahr  = @it_doc-ryear
                                    and awtyp  = 'BKPFF'
                                    and zuonr  = @it_doc-zuonr
                                appending corresponding fields of table
@ct_acdoca.
    endif.
    when 'IN'.
        if it_ref[] is not initial.
            "Retrieve the documents that were posted via ACDOCA-only posting
functionality
            select (mt_fields_acdoca) from acdoca
                                for all entries in @it_ref
                                where (lv_ledger_cond)
                                    and rbukrs      = @ms_progpars-bukrs
                                    and gjahr       = @it_ref-ryear
                                    and vorgn       = 'RFRA'
```

```
                and ( prec_awtyp = @it_ref-awtyp or
                      prec_awtyp is initial )
                and ( prec_aworg = @it_ref-aworg or
                      prec_aworg is initial )
                and prec_awref = @it_ref-refdocnr
                and prec_awitem = @it_ref-refdocln
                appending corresponding fields of table

@ct_acdoca.
    endif.
    when 'MI'.
        "Special handling for documents migrated during S4HANA conversion
        "If a document is in the JV ledger only, but not in FI,
        "it should exist with mig_source = 'J' and/or awtyp = 'JVAME'/'JVAM'
        if it_doc[] is not initial.
            select (mt_fields_acdoca) from acdoca
                for all entries in @it_doc
                where (lv_ledger_cond)
                    and rbukrs = @ms_progpars-bukrs
                    and gjahr = @it_doc-ryear
                    and ( mig_source = 'J' or
                        awtyp = 'JVAME' or
                        awtyp = 'JVAM' )
                    and awref = @it_doc-refdocnr
                appending corresponding fields of table

@ct_acdoca.
    endif.
    if it_ref[] is not initial.
        select (mt_fields_acdoca) from acdoca
            for all entries in @it_ref
            where (lv_ledger_cond)
                and rbukrs = @ms_progpars-bukrs
                and gjahr = @it_ref-ryear
                and ( mig_source = 'J' or
                    awtyp = 'JVAME' or
                    awtyp = 'JVAM' )
                and awref = @it_ref-refdocnr
                and jvactivity is not initial
            appending corresponding fields of table

@ct_acdoca.
    endif.
    when 'LG'.
        if it_corr_log[] is not initial.
            "Retrieve the documents that were posted through this program in a
different fiscal year
            select (mt_fields_acdoca) from acdoca
                for all entries in @it_corr_log
                where (lv_ledger_cond)
                    and rbukrs = @ms_progpars-bukrs
                    and gjahr = @it_corr_log-gjahr_new
                    and belnr = @it_corr_log-belnr_new
                appending corresponding fields of table

@ct_acdoca.
    endif.
endcase.
```

endmethod.

RETRIEVE_AND_PROCESS_DATA

Description:
Instance mthd

Returning parameter

VALUE(RV_COUNT) TYPE I

```
method retrieve_and_process_data.
  data: lt_doc_all      type gty_t_doc,
        lt_doc         type gty_t_doc,
        lt_doc_fi      type gty_t_doc,
        lt_doc_co      type gty_t_doc,
        lt_doc_4a      type gty_t_doc,
        lt_doc_4b      type gty_t_doc,
        ls_doc_next    type gty_s_doc,
        lt_jvso        type jvso_t,
        lt_jvso_fi     type jvso_t,
        lt_jvso_4a     type jvso_t,
        lt_jvso_4b     type jvso_t,
        ls_item_info   type
cl_jva_mig_alv_output=>if_jva_mig_alv_output~gty_s_item_info,
        lv_doc_count_fico type i,
        lv_doc_count_me  type i,
        lv_doc_count_op  type i,
        lv_doc_count     type i,
        lt_acdoca        type acdoca_jv_t,
        lt_acdoca_int    type acdoca_jv_t,
        lt_acdoca_mig    type acdoca_jv_t,
        lt_ref           type gty_t_ref,
        lt_corr_log      type gty_t_corr_log,
        lt_corr_log_4b   type gty_t_corr_log,
        lv_last_bulk     type abap_bool.
  "Get all JV documents according to the selection parameters
  "In the first step the lines are not retrieved, but only the
  "document numbers and some basic data
  do.
    if ms_progpar~update_fagl_split = abap_true.
      "lt_open_items = retrieve_open_items( ).
      "lt_doc_all = retrieve_open_jv_items( ).
    else.
      lt_doc_all = retrieve_jv_documents( changing
                                         cv_doc_count_fico = lv_doc_count_fico
                                         cv_doc_count_me   = lv_doc_count_me
                                         cv_doc_count_op   = lv_doc_count_op
                                         cv_last_bulk      = lv_last_bulk ).
    endif.
    rv_count = lines( lt_doc_all ).
    if rv_count = 0.
      exit.
    endif.
    "Process the JV documents in bulks
    loop at lt_doc_all into data(ls_doc).
```

```
data(lv_tabix) = sy-tabix.
if ms_progpars-select_fi_documents = abap_true or
   ms_progpars-select_co_documents = abap_true.
  "Ignore documents where the relevant reference number is missing
  if ls_doc-activ = gc_jvlinetype-fi_doc or
     ls_doc-activ = gc_jvlinetype-fi_doc_mm.
    if ls_doc-reffidoc is initial.
      continue.
    endif.
  elseif ls_doc-activ = gc_jvlinetype-co_doc.
    if ls_doc-refdocnr is initial.
      continue.
    endif.
  endif.
  ms_results-doc_count += 1.
  "If the next JV doc has the same reference data, do not
  "count it as an extra document
  clear ls_doc_next.
  read table lt_doc_all into ls_doc_next index lv_tabix + 1.
  case ls_doc-activ.
    when gc_jvlinetype-fi_doc or gc_jvlinetype-fi_doc_mm.
      if ls_doc-activ <> ls_doc_next-activ or
         ls_doc-ryear <> ls_doc_next-ryear or
         ls_doc-reffidoc <> ls_doc_next-reffidoc.
        lv_doc_count += 1.
      endif.
    when gc_jvlinetype-co_doc.
      if ls_doc-activ <> ls_doc_next-activ or
         ls_doc-ryear <> ls_doc_next-ryear or
         ls_doc-refdocnr <> ls_doc_next-refdocnr.
        lv_doc_count += 1.
      endif.
    when others.
      if ls_doc-activ <> ls_doc_next-activ or
         ls_doc-ryear <> ls_doc_next-ryear or
         ls_doc-reffidoc <> ls_doc_next-reffidoc or
         ls_doc-refdocnr <> ls_doc_next-refdocnr.
        lv_doc_count += 1.
      endif.
    endcase.
  else.
    "Process every internal JV document
    lv_doc_count += 1.
    ms_results-doc_count += 1.
  endif.
if ms_progpars-select_fi_documents = abap_true or
   ms_progpars-select_co_documents = abap_true.
  if ls_doc-activ = gc_jvlinetype-co_doc.
    ls_doc-is_co_doc = abap_true.
    append ls_doc to lt_doc_co.
    if ls_doc-reffidoc is not initial.
      append ls_doc to lt_doc_fi.
    endif.
  else.
```

```
        append ls_doc to lt_doc_fi.
    endif.
else.
    if ls_doc-rldnr = '4A'.
        if ms_progpars-newgl_integration_active = abap_true.
            append ls_doc to lt_doc_fi.
        else.
            append ls_doc to lt_doc_4a.
        endif.
    else.
        ls_doc-zuonr = ls_doc-docnr.
        append ls_doc to lt_doc_4b.
    endif.
endif.
if lv_doc_count >= ms_progpars-bulk_no_of_docs_for_line_proc or
lv_tabix >= lines( lt_doc_all ).
if lt_doc_fi[] is not initial or
lt_doc_co[] is not initial or
lt_ref[] is not initial.
if ms_progpars-select_fi_documents = abap_true or
ms_progpars-select_co_documents = abap_true.
if lt_doc_fi[] is not initial.
    retrieve_fi_doc_header_data( changing
                                ct_doc_fi = lt_doc_fi
                                ct_doc_co = lt_doc_co ).

    append lines of lt_doc_fi to lt_doc.
endif.
append lines of lt_doc_co to lt_doc.
retrieve_jv_doc_lines( exporting
                        iv_rldnr      = '4A'
                        importing
                        et_jvso       = lt_jvso
                        et_ref        = lt_ref
                        changing
                        ct_doc        = lt_doc ).
lt_corr_log = retrieve_log_items_diff_years( it_jvso = lt_jvso ).
"Retrieve the FI documents from ACDOCA
retrieve_acdoca_lines(
    exporting
        iv_type   = 'FI'
        it_doc    = lt_doc_fi
    changing
        ct_acdoca = lt_acdoca ).
if mv_has_migrated_jv_docs = abap_true.
    "Retrieve the migrated internal FI documents from ACDOCA
    retrieve_acdoca_lines(
        exporting
            iv_type   = 'MI'
            it_doc    = lt_doc_fi
        changing
            ct_acdoca = lt_acdoca_mig ).
endif.
"Retrieve the CO documents from ACDOCA
retrieve_acdoca_lines(
```

```
    exporting
      iv_type   = 'CO'
      it_doc    = lt_doc_co
    changing
      ct_acdoca = lt_acdoca ).
if mv_has_migrated_jv_docs = abap_true.
  "Retrieve the migrated internal CO documents from ACDOCA
  retrieve_acdoca_lines(
    exporting
      iv_type   = 'MI'
      it_doc    = lt_doc_co
    changing
      ct_acdoca = lt_acdoca_mig ).
endif.
else.
  retrieve_jv_doc_lines( exporting
                        iv_rldnr = '4A'
                        importing
                          et_jvso = lt_jvso_fi
                          et_ref  = lt_ref
                        changing
                          ct_doc  = lt_doc_fi ).
  lt_corr_log = retrieve_log_items_diff_years( it_jvso = lt_jvso ).
  retrieve_fi_doc_header_data( changing
                              ct_doc_fi = lt_doc_fi
                              ct_doc_co = lt_doc_co ).

  lt_doc[] = lt_doc_fi[].
  lt_jvso[] = lt_jvso_fi[].
  delete lt_doc_fi where is_fi_doc = abap_false.
  "Retrieve the FI documents from ACDOCA
  retrieve_acdoca_lines(
    exporting
      iv_type   = 'FI'
      it_doc    = lt_doc_fi
    changing
      ct_acdoca = lt_acdoca ).
endif.
endif.
if lt_doc_4a[] is not initial.
  retrieve_jv_doc_lines( exporting
                        iv_rldnr = '4A'
                        importing
                          et_jvso = lt_jvso_4a
                          et_ref  = lt_ref
                        changing
                          ct_doc  = lt_doc_4a ).
  append lines of lt_doc_4a to lt_doc.
  append lines of lt_jvso_4a to lt_jvso.
  "Retrieve the internal JVA documents from ACDOCA
  retrieve_acdoca_lines(
    exporting
      iv_type   = '4A'
      it_doc    = lt_doc_4a
    changing
```

```
        ct_acdoca = lt_acdoca ).
endif.
if lt_ref[] is not initial.
    "Retrieve the internal documents from ACDOCA
    retrieve_acdoca_lines(
        exporting
            iv_type   = 'IN'
            it_ref    = lt_ref
        changing
            ct_acdoca = lt_acdoca_int ).
if mv_has_migrated_jv_docs = abap_true.
    "Retrieve the migrated internal documents from ACDOCA
    retrieve_acdoca_lines(
        exporting
            iv_type   = 'MI'
            it_ref    = lt_ref
        changing
            ct_acdoca = lt_acdoca_mig ).
endif.
endif.
if lt_doc_4b[] is not initial.
    append lines of lt_doc_4b to lt_doc.
    retrieve_jv_doc_lines( exporting
                            iv_rldnr = '4B'
                            importing
                                et_jvso = lt_jvso_4b
                            changing
                                ct_doc  = lt_doc_4b ).
    lt_corr_log_4b = retrieve_log_items_diff_years( it_jvso = lt_jvso_4b ).
    append lines of lt_jvso_4b to lt_jvso.
    append lines of lt_corr_log_4b to lt_corr_log.
    "Retrieve the billing documents from ACDOCA
    retrieve_acdoca_lines(
        exporting
            iv_type   = '4B'
            it_doc    = lt_doc_4b
        changing
            ct_acdoca = lt_acdoca ).
endif.
sort lt_corr_log.
"Retrieve the new documents posted by this report in different years
retrieve_acdoca_lines(
    exporting
        iv_type   = 'LG'
        it_corr_log = lt_corr_log
    changing
        ct_acdoca = lt_acdoca ).
if ms_progpars-select_fi_documents = abap_true or
   ms_progpars-select_co_documents = abap_true.
    sort lt_doc by activ rbukrs ryear reffidoc reffdocnr docnr.
else.
    sort lt_doc by rbukrs ryear docnr.
endif.
sort lt_jvso by rbukrs ryear docnr docln.
```

```

    sort lt_acdoca by rldnr rbukrs gjahr belnr buzei co_buzei.
    sort lt_acdoca_int by rldnr rbukrs gjahr prec_awtyp prec_aworg prec_awref
prec_awitem belnr docln.
    sort lt_acdoca_mig by rldnr rbukrs gjahr awref awitem jvactivity bttype
belnr docln.
    delete adjacent duplicates from lt_acdoca_mig.
    compare_and_correct_documents( exporting
                                it_jvso      = lt_jvso
                                it_acdoca    = lt_acdoca
                                it_acdoca_int = lt_acdoca_int
                                it_acdoca_mig = lt_acdoca_mig
                                it_corr_log  = lt_corr_log
                                changing
                                ct_doc_jv    = lt_doc ).

    clear lv_doc_count.
    clear: lt_doc,
           lt_doc_fi,
           lt_doc_co,
           lt_doc_4b,
           lt_jvso,
           lt_jvso_4b,
           lt_ref,
           lt_acdoca,
           lt_acdoca_int,
           lt_acdoca_mig,
           lt_corr_log,
           lt_corr_log_4b.

    endif.
endloop.
if lv_last_bulk = abap_true.
    exit.
endif.
enddo.
endmethod.
```

RETRIEVE_FI_DOC_HEADER_DATA

Description:
Instance mthd

Changing parameter

CT_DOC_FI TYPE GTY_T_DOC
CT_DOC_CO TYPE GTY_T_DOC

```

method retrieve_fi_doc_header_data.
    data: ls_cobk type cobk.
    "Get the FI document headers
    select bukrs, gjahr, belnr, awkey, glvor, tcode, bstat, btxt
    from bkp
    into table @data(lt_bkp)
    for all entries in @ct_doc_fi
    where bukrs = @ms_progpars-bukrs
    and gjahr = @ct_doc_fi-ryear
    and belnr = @ct_doc_fi-reffidoc
    order by primary key.
```

```
"Get the CO document headers
select kokrs, gjahr, belnr
  from cobk
  into table @data(lt_cobk)
  for all entries in @ct_doc_fi
  where kokrs = @ct_doc_fi-rkokrs
     and gjahr = @ct_doc_fi-ryear
     and belnr = @ct_doc_fi-refdocnr
  order by primary key.
if lt_cobk[] is initial and
  lt_bkpf[] is initial.
  return.
endif.
sort lt_cobk by kokrs gjahr belnr.
sort ct_doc_fi by ryear reffidoc.
sort ct_doc_co by ryear reffidoc.
loop at lt_bkpf into data(ls_bkpf).
  read table ct_doc_fi transporting no fields
    with key ryear      = ls_bkpf-gjahr
         reffidoc = ls_bkpf-belnr
    binary search.

  if sy-subrc <> 0.
    continue.
  endif.
  loop at ct_doc_fi assigning field-symbol(<ls_doc_fi>)
    from sy-tabix.
    if <ls_doc_fi>-ryear      <> ls_bkpf-gjahr or
       <ls_doc_fi>-reffidoc <> ls_bkpf-belnr.
      exit.
    endif.
    data(lv_tabix_doc) = sy-tabix.
    move-corresponding ls_bkpf to <ls_doc_fi>.
    "Special handling for internal JV documents
    if ls_bkpf-tcode(4) = 'GJEC' or
       ls_bkpf-tcode(4) = 'GJ19' or
       ls_bkpf-tcode(4) = 'GJ17' or
       ls_bkpf-tcode(6) = 'GJFARM' or
       ls_bkpf-bktxt(4) = 'GJEC' or
       ls_bkpf-bktxt(4) = 'GJ19' or
       ls_bkpf-bktxt(4) = 'GJ17' or
       ls_bkpf-bktxt(6) = 'GJFARM'.
      if <ls_doc_fi>-activ = gc_jvlinetype-fi_doc or
         <ls_doc_fi>-activ = gc_jvlinetype-fi_doc_mm.
        "Do not process internal JV documents as normal FI docs
        delete ct_doc_fi index lv_tabix_doc.
      elseif <ls_doc_fi>-activ <> gc_jvlinetype-co_doc.
        clear ls_cobk.
        "Check if the JV document was created for a CO document
        if <ls_doc_fi>-rkokrs is initial.
          "Get the controlling area if it is missing
          select single kokrs
            from tka02
           where bukrs = @<ls_doc_fi>-rbukrs
              and gsber is initial
```

```
        into @<ls_doc-fi>-rkokrs.
"Get the C0 document header
select single kokrs, gjahr, belnr
  from cobk
  where kokrs = @<ls_doc-fi>-rkokrs
        and gjahr = @<ls_doc-fi>-ryear
        and belnr = @<ls_doc-fi>-refdocnr
  into @ls_cobk.
if sy-subrc = 0.
  append ls_cobk to lt_cobk.
  sort lt_cobk by kokrs gjahr belnr.
endif.
else.
  read table lt_cobk into ls_cobk
    with key kokrs = <ls_doc-fi>-rkokrs
          gjahr = <ls_doc-fi>-ryear
          belnr = <ls_doc-fi>-refdocnr.

endif.
if ls_cobk is initial.
  <ls_doc-fi>-is-fi-doc = abap_true.
endif.
endif.
else.
  if <ls_doc-fi>-activ = gc_jvlinetype-fi-doc    or
     <ls_doc-fi>-activ = gc_jvlinetype-fi-doc_mm.
    <ls_doc-fi>-is-fi-doc = abap_true.
  elseif <ls_doc-fi>-is-co-doc = abap_true.
    if ls_bkpf-glvor = 'COFI'.
      "The REFFIDOC values is wrong
      delete ct_doc-fi index lv_tabix-doc.
    else.
      <ls_doc-fi>-is-fi-doc = abap_true.
      <ls_doc-fi>-is-co-doc = abap_false.
      "Delete the header item from the C0 doc table
      read table ct_doc-co into data(ls_doc-co)
        with key ryear      = <ls_doc-fi>-ryear
              reffidoc = <ls_doc-fi>-reffidoc.

      if sy-subrc = 0.
        delete ct_doc-co where ryear      = ls_doc-co-ryear
                          and reffidoc = <ls_doc-fi>-reffidoc.
      endif.
    endif.
  endif.
endif.
endloop.
endloop.
endmethod.
```

RETRIEVE_JV_DOCUMENTS

Description:

Instance mthd

Changing parameter

CV_DOC_COUNT_FICO TYPE I

```

CV_DOC_COUNT_ME TYPE I
CV_DOC_COUNT_OP TYPE I
CV_LAST_BULK TYPE ABAP_BOOL

```

Returning parameter

```

VALUE(RT_DOC) TYPE GTY_T_DOC

method retrieve_jv_documents.
  data: lt_doc_op      like rt_doc,
        lv_max_lines  type i,
        lv_cond       type string,
        lv_line_count type i,
        lv_line_count_me type i,
        lv_line_count_op type i,
        lt_period_condition type gty_t_where_condition.
  if ms_progpars-is_parallel_task = abap_true.
    * loop at ms_progpars-s_task_package-t_period_condition assigning
    * field-symbol(<ls_period_condition>).
    * data(ls_period_condition) = cl_abap_dyn_prg=>escape_quotes(
    <ls_period_condition> ).
    * append ls_period_condition to lt_period_condition.
    * endloop.
    loop at ms_progpars-s_task_package-t_periods into data(ls_periods).
      if lt_period_condition[] is not initial.
        lv_cond = ` OR`.
      else.
        clear lv_cond.
      endif.
      lv_cond &&= ` ryear = ` && ls_periods-ryear ##NO_TEXT.
      lv_cond &&= ` AND (` .
      loop at ls_periods-t_poper into data(lv_poper).
        if sy-tabix > 1.
          lv_cond &&= ` OR`.
        endif.
        lv_cond &&= ` poper = ' ` && lv_poper && ` ' ` ##NO_TEXT.
      endloop.
      lv_cond &&= ` )`.
      append lv_cond to lt_period_condition.
    endloop.
    insert `(` into lt_period_condition index 1.
    append `)` to lt_period_condition.
  endif.
  if lt_period_condition[] is initial.
    lt_period_condition = value #(
      ( ' ( ryear in @ms_progpars-r_ryear' )
      ( ' AND poper in @ms_progpars-r_poper )' )
      ) ##NO_TEXT.
  endif.
  if ms_progpars-select_fi_documents = abap_true or
    ms_progpars-select_co_documents = abap_true.
    data(lt_ref_cond) = make_reference_condition( ms_progpars ).
    select distinct rldnr, rbukrs, ryear, docnr, reffidoc, refdocnr, activ
      from jvso1
      where rldnr      = '4A'
      and rbukrs      = @ms_progpars-bukrs

```

```

        and (lt_period_condition)
        and (lt_ref_cond)
        and budat      in @ms_progpars-r_budat
        and bldat      in @ms_progpars-r_bldat
        and racct      in @ms_progpars-r_racct
        and rjvnam     in @ms_progpars-r_rjvnam
        and regrou     in @ms_progpars-r_regrou
        and rrecin     in @ms_progpars-r_rrecin
        and bilid      in @ms_progpars-r_bilid
        and prodper    in @ms_progpars-r_prodper
        and rcntr      in @ms_progpars-r_rcntr
        and rordnr     in @ms_progpars-r_rordnr
        and rprojk     in @ms_progpars-r_rprojk
        and nplnr      in @ms_progpars-r_nplnr
        and aufpl      in @ms_progpars-r_aufpl
        and aplzl      in @ms_progpars-r_aplzl
        order by activ, rbukrs, ryear, reffidoc, reffdocnr, docnr
        into corresponding fields of table @rt_doc
        up to @ms_progpars-bulk_no_of_doc_headers_in_mem rows
        offset @cv_doc_count_fico.
lv_line_count = lines( rt_doc ).
cv_doc_count_fico += lv_line_count.
else.
    if ms_progpars-r_activ_me[] is not initial.
        select distinct rldnr, rbukrs, ryear, docnr
        from jvso1
        where rldnr = '4A'
        and rbukrs = @ms_progpars-bukrs
        and (lt_period_condition)
        and docnr in @ms_progpars-r_docnr
        and activ in @ms_progpars-r_activ_me
        order by rbukrs, ryear, docnr
        into corresponding fields of table @rt_doc
        up to @ms_progpars-bulk_no_of_doc_headers_in_mem rows
        offset @cv_doc_count_me.
        lv_line_count_me = lines( rt_doc ).
        cv_doc_count_me += lv_line_count_me.
    endif.
    if ms_progpars-r_activ_op[] is not initial.
        lv_max_lines = ms_progpars-bulk_no_of_doc_headers_in_mem - lv_line_count_me.
        if lv_max_lines > 0.
            select distinct rldnr, rbukrs, ryear, docnr
            from jvso2
            where rldnr = '4B'
            and rbukrs = @ms_progpars-bukrs
            and (lt_period_condition)
            and docnr in @ms_progpars-r_docnr
            and activ in @ms_progpars-r_activ_op
            order by rbukrs, ryear, docnr
            into corresponding fields of table @lt_doc_op
            up to @lv_max_lines rows
            offset @cv_doc_count_op.          "#EC CI_NOFIELD
            lv_line_count_op = lines( lt_doc_op ).
            cv_doc_count_op += lv_line_count_op.
        endif.
    endif.

```

```
        append lines of lt_doc_op to rt_doc.
    endif.
endif.
lv_line_count = lv_line_count_me + lv_line_count_op.
endif.
if lv_line_count < ms_progpars-bulk_no_of_doc_headers_in_mem.
    cv_last_bulk = abap_true.
endif.
endmethod.                                "retrieve_jv_documents
```

RETRIEVE_JV_DOC_LINES

Description:
Instance mthd

Importing parameter

IV_RLDNR TYPE RLDNR

Exporting parameter

ET_JVSO TYPE JVS0_T
ET_REF TYPE GTY_T_REF

Changing parameter

CT_DOC TYPE GTY_T_DOC

```
method retrieve_jv_doc_lines.
    data: lt_jvso type jvso_t,
          lr_rldnr type range of rldnr.
    clear: et_jvso,
           et_ref.
    if iv_rldnr = '4A'.
        if mt_fields_jvso1[] is initial.
            cl_jva_util=>get_selection_fields(
                exporting
                    it_table      = lt_jvso
                    iv_source_table = 'JVS01'
                    iv_aggr_amnts  = abap_false
                    iv_prefix      = space
                changing
                    ct_sel_fields  = mt_fields_jvso1
                    ct_group_fields = mt_group_jvso1 ).
        endif.
        lr_rldnr = value #( ( sign   = 'I'
                             option = 'EQ'
                             low     = '4A' ) ).
        if ms_classic_currencies-curt3_jv is not initial or
           ms_classic_currencies-curt4_jv is not initial.
            lr_rldnr = value #( base lr_rldnr
                                ( sign   = 'I'
                                 option = 'EQ'
                                 low     = '4C' ) ).
        endif.
        "Get the JVS01 items from ledger 4A/C
        select (mt_fields_jvso1) from jvso1
            appending corresponding fields of table @lt_jvso
            for all entries in @ct_doc
            where rldnr in @lr_rldnr
```

```

                                and rbukrs = @ms_progpars-bukrs
                                and ryear  = @ct_doc-ryear
                                and docnr   = @ct_doc-docnr. "#EC CI_NOFIELD
elseif iv_rldnr = '4B'.
  if mt_fields_jvso2[] is initial.
    cl_jva_util=>get_selection_fields(
      exporting
        it_table      = lt_jvso
        iv_source_table = 'JVS02'
        iv_aggr_amnts  = abap_false
        iv_prefix      = space
      changing
        ct_sel_fields  = mt_fields_jvso2
        ct_group_fields = mt_group_jvso2 ).
  endif.
  lr_rldnr = value #( ( sign   = 'I'
                      option = 'EQ'
                      low    = '4B' ) ).
  if ms_classic_currencies-curt3_jv is not initial or
     ms_classic_currencies-curt4_jv is not initial.
    lr_rldnr = value #( base lr_rldnr
                      ( sign   = 'I'
                      option = 'EQ'
                      low    = '4D' ) ).
  endif.
  "Get the JVS02 items from ledger 4B/D
  select (mt_fields_jvso2) from jvso2
                                appending corresponding fields of table @lt_jvso
                                for all entries in @ct_doc
                                where rldnr in @lr_rldnr
                                and rbukrs = @ms_progpars-bukrs
                                and ryear  = @ct_doc-ryear
                                and docnr   = @ct_doc-docnr. "#EC CI_NOFIELD
  endif.
  sort lt_jvso by ryear docnr docln rldnr.
  "Combine ledger 4A/4B with ledger 4C/4D
  combine_and_analyze_jv_lines(
    importing
      et_jvso_combined = et_jvso
      et_ref            = et_ref
    changing
      ct_jvso          = lt_jvso
      ct_doc            = ct_doc ).
  ms_results-line_count += lines( et_jvso ).
endmethod.                      " retrieve_jv_doc_lines
```

RETRIEVE_LOG_ITEMS_DIFF_YEARS

Description:
Instance mthd

Importing parameter

IT_JVSO TYPE JVS0_T

Returning parameter

VALUE(RT_CORR_LOG) TYPE GTY_T_CORR_LOG

```
method retrieve_log_items_diff_years.  
  "Get the items from log for which new GL docs were posted  
  "in a different fiscal year (the others will be picked up differently)  
  select * from jva_acd_upd_log  
    for all entries in @it_jvso  
  where rbukrs = @it_jvso-rbukrs  
    and ryear  = @it_jvso-ryear  
    and docnr  = @it_jvso-docnr  
    and ( docln = @it_jvso-docln or  
          docln = @space )  
    and gjahr_new <> @it_jvso-ryear  
    and belnr_new is not initial  
  into table @rt_corr_log.  
endmethod.
```

RETURN_MULTIPLE_LINES_HIT

Description:
Instance mthd

Importing parameter

```
IV_RECURSION_LEVEL TYPE I  
IT_ACDOCA_SEL TYPE ACDOCA_JV_T  
IT_ACDOCA_ORIGINAL TYPE ACDOCA_JV_T  
IV_TABIX_HIT TYPE SY-TABIX  
IT_ACD_COMPR_INFO TYPE GTY_T_ACD_COMPR_INFO
```

Changing parameter

```
CS_LOG TYPE GTY_S_LOG
```

Returning parameter

```
VALUE(RT_ACDOCA) TYPE ACDOCA_JV_T
```

```
method return_multiple_lines_hit.  
  if iv_recursion_level = 0.  
    rt_acdoca[] = it_acdoca_sel[].  
    cs_log-heuristic_hit = abap_true.  
  else.  
    read table it_acd_compr_info into data(ls_acd_compr_info)  
                                with key tabix_aggr = iv_tabix_hit.  
  
    if sy-subrc = 0 and  
      ls_acd_compr_info is not initial.  
      loop at ls_acd_compr_info-t_docln_original into data(lv_docln).  
        read table it_acdoca_original into data(ls_acdoca)  
                                with key docln = lv_docln.  
  
        if sy-subrc = 0 and  
          ls_acdoca is not initial.  
          append ls_acdoca to rt_acdoca.  
        endif.  
      endloop.  
    endif.  
    if rt_acdoca[] is not initial.  
      cs_log-heuristic_hit = abap_true.  
    endif.  
  endif.  
endmethod.
```

REV_AND_REP_ACDOCA_DOC

Description:

Instance mthd

Importing parameter

IS_DOC TYPE GTY_S_DOC
IT_DOC_COMB TYPE GTY_T_DOC
IT_JVSO_DOC TYPE JVSO_T
IT_ACDOCA_DOC TYPE ACDOCA_JV_T
IV_LEDGER TYPE RLDNR
IV_POSTING_MODE TYPE CHAR1

Changing parameter

CT_LOG_DOC TYPE GTY_T_LOG

```
method rev_and_rep_acdoca_doc.  
  data: lv_updated      type abap_bool,  
        lv_updated_rev  type abap_bool,  
        lv_bukrs        type accit-bukrs,  
        lv_belnr        type accit-belnr,  
        lv_gjahr        type accit-gjahr,  
        lv_gjahr_rev    type accit-gjahr,  
        lv_belnr_rev    type accit-belnr,  
        lv_belnr_new    type accit-belnr,  
        lv_multiple_docs_found type abap_bool,  
        lo_messages     type ref to if_jva_message_collection,  
        lv_msg_type     type char1,  
        lv_message_text type string,  
        lv_doc_stats_updated type abap_bool.  
  if ms_progpars-analysis_only = abap_true.  
    add_item_info_for_doc(  
      iv_update      = abap_false  
      is_doc         = is_doc  
      it_doc_comb    = it_doc_comb  
      it_log         = ct_log_doc  
      it_acdoca_doc  = it_acdoca_doc  
      iv_msg_text    = space  
      iv_msg_type    = gc_error_type-warning  
      iv_updated     = lv_updated_rev  
      iv_belnr       = lv_belnr_rev  
      iv_gjahr       = lv_gjahr_rev  
      io_messages    = lo_messages ).  
    return.  
  endif.  
  "Get the document number and check for multiple document numbers  
  loop at it_acdoca_doc assigning field-symbol(<ls_acdoca>).  
    if lv_belnr is initial.  
      lv_bukrs = <ls_acdoca>-rbukrs.  
      lv_gjahr = <ls_acdoca>-gjahr.  
      lv_belnr = <ls_acdoca>-belnr.  
    elseif lv_belnr <> <ls_acdoca>-belnr.  
      lv_multiple_docs_found = abap_true.  
    endif.  
  endloop.
```

```
if lv_multiple_docs_found = abap_true.
  loop at ct_log_doc assigning field-symbol(<ls_log>).
    add_message_to_log(
      exporting
        iv_msg_text = 'Multiple ACDOCA docs found for JV doc'(mdf) && | {
<ls_log>-docnr }|
        iv_msg_type = gc_error_type-error
      changing
        cs_log = <ls_log> ).
    update_line_stats_alignm_fails( <ls_log> ).
    if lv_doc_stats_updated = abap_false.
      lv_doc_stats_updated = abap_true.
      update_doc_stats_alignm_fails( <ls_log> ).
    endif.
    add_item_info_for_line( iv_update = abap_false
                          is_log    = <ls_log> ).

  endloop.
  return.
endif.
if lv_belnr is initial.
  loop at ct_log_doc assigning <ls_log>.
    add_message_to_log(
      exporting
        iv_msg_text = conv string( 'Doc number for ACDOCA doc not found'(dnf) )
        iv_msg_type = gc_error_type-error
      changing
        cs_log = <ls_log> ).
    update_line_stats_alignm_fails( <ls_log> ).
    if lv_doc_stats_updated = abap_false.
      lv_doc_stats_updated = abap_true.
      update_doc_stats_alignm_fails( <ls_log> ).
    endif.
    add_item_info_for_line( iv_update = abap_false
                          is_log    = <ls_log> ).

  endloop.
  return.
endif.
data(lo_post_acdoca) = cl_jva_mig_post_acdoca=>get_instance( ).
lo_post_acdoca->reverse_acdoca_doc(
  exporting
    it_jvso_doc   = it_jvso_doc
    it_acdoca_doc = it_acdoca_doc
    iv_ledger     = iv_ledger
    iv_bukrs      = lv_bukrs
    iv_gjahr      = lv_gjahr
    iv_belnr      = lv_belnr
    iv_budat      = ms_progpars-posting_date
    iv_test       = ms_progpars-testrun
  changing
    cv_updated    = lv_updated_rev
    cv_belnr      = lv_belnr_rev
    cv_gjahr      = lv_gjahr_rev
    co_messages   = lo_messages ).
if lo_messages->has_critical_messages( ) = abap_true.
```

```
if ms_progpars-testrun = abap_true.
  lv_message_text = 'ERROR when testing the reversal document'(etr).
else.
  lv_message_text = 'ERROR when posting the reversal document'(epr).
endif.
add_item_info_for_doc(
  iv_update      = abap_true
  is_doc         = is_doc
  it_doc_comb    = it_doc_comb
  it_log         = ct_log_doc
  it_acdoca_doc  = it_acdoca_doc
  iv_msg_text    = lv_message_text
  iv_msg_type    = gc_error_type-error
  iv_updated     = lv_updated_rev
  iv_belnr       = lv_belnr_rev
  iv_gjahr       = lv_gjahr_rev
  io_messages    = lo_messages ).
else.
  lo_post_acdoca->post_new_acdoca_doc(
    exporting
      it_jvso_doc      = it_jvso_doc
      iv_ledger        = iv_ledger
      iv_posting_mode  = iv_posting_mode
      iv_budat         = ms_progpars-posting_date
      iv_test          = ms_progpars-testrun
      iv_jva_integrated = mv_jva_integrated
    changing
      cv_updated      = lv_updated
      cv_belnr        = lv_belnr
      cv_gjahr        = lv_gjahr
      co_messages     = lo_messages ).
  if lv_updated = abap_true.
    lv_msg_type = gc_error_type-statement.
    if ms_progpars-testrun = abap_true.
      lv_message_text = 'New document tested successfully'(nst).
    else.
      lv_message_text = 'New document posted successfully'(nsu).
    endif.
  else.
    lv_msg_type = gc_error_type-error.
    if ms_progpars-testrun = abap_true.
      lv_message_text = 'ERROR when testing the new document'(etn).
    else.
      lv_message_text = 'ERROR when posting the new document'(epn).
    endif.
  endif.
  add_item_info_for_doc(
    iv_update      = abap_true
    is_doc         = is_doc
    it_doc_comb    = it_doc_comb
    it_log         = ct_log_doc
    it_acdoca_doc  = it_acdoca_doc
    iv_msg_text    = lv_message_text
    iv_msg_type    = lv_msg_type
```

```
        iv_updated      = lv_updated
        iv_belnr         = lv_belnr
        iv_gjahr         = lv_gjahr
        io_messages      = lo_messages ).
    endif.
endmethod.
```

SAVE_RESULTS

Description:

Instance mthd

Importing parameter

```
IS_RESULTS TYPE IF_JVA_MIG_ALV_OUTPUT=>GTY_S_RESULTS
IS_TASKDATA TYPE GTY_S_TASKDATA  OPTIONAL
```

Returning parameter

```
VALUE(RV_COCODE_COMPLETE) TYPE ABAP_BOOL
```

```
method save_results.
    data: ls_results_new like line of mt_results.
    "Get the results structure for the given company code
    read table mt_results assigning field-symbol(<ls_results>)
        with key bukrs = is_results-bukrs
        binary search.
    if <ls_results> is not assigned.
        ls_results_new = is_results.
        insert ls_results_new into mt_results index sy-tabix
            assigning <ls_results>.
    return.
endif.
if is_taskdata is not initial.
    update_task_info( exporting
        is_taskdata = is_taskdata
        changing
        cs_results  = <ls_results> ).
    "Check if all tasks for a company code have been returned
    read table <ls_results>-t_tasks transporting no fields
        with key returned = abap_false.
    if sy-subrc <> 0.
        rv_cocode_complete = abap_true.
    endif.
endif.
"Append the messages and line items
append lines of is_results-t_messages to <ls_results>-t_messages.
append lines of is_results-t_item_info to <ls_results>-t_item_info.
append lines of is_results-t_spool to <ls_results>-t_spool.
add_up_all_results_stats(
    exporting
        is_results_to_add = is_results
    changing
        cs_results        = <ls_results> ).
endmethod.
```

SAVE_RESULTS_GLOBAL

Description:

Instance mthd

```
method save_results_global.  
  if lines( mt_results[] ) > 0.  
    loop at mt_results assigning field-symbol(<ls_results>).  
      add_up_results_stats_totals( changing cs_results = <ls_results> ).  
      append lines of <ls_results>-t_tasks to ms_results_global-t_tasks.  
      ms_results_global-runtime_tasks      += <ls_results>-runtime_tasks.  
      ms_results_global-runtime_tasks_admin += <ls_results>-runtime_tasks_admin.  
      ms_results_global-tasks_scheduled   += <ls_results>-tasks_scheduled.  
      ms_results_global-tasks_initialized += <ls_results>-tasks_initialized.  
      ms_results_global-tasks_executed    += <ls_results>-tasks_executed.  
      ms_results_global-tasks_processed   += <ls_results>-tasks_processed.  
      ms_results_global-tasks_returned    += <ls_results>-tasks_returned.  
      append lines of <ls_results>-t_spool to ms_results_global-t_spool.  
      add_up_all_results_stats(  
        exporting  
          is_results_to_add = <ls_results>  
        changing  
          cs_results      = ms_results_global ).  
      if ms_progpars-print_results = abap_true and  
        <ls_results>-t_tasks[] is not initial and  
        <ls_results>-t_spool[] is initial.  
        <ls_results>-t_spool = cl_jva_mig_alv_output=>get_instance(  
)->print_results( is_results = <ls_results> ).  
        clear <ls_results>-t_item_info.  
      endif.  
    endloop.  
  else.  
    add_up_all_results_stats(  
      exporting  
        is_results_to_add = ms_results  
      changing  
        cs_results      = ms_results_global ).  
    endif.  
    add_up_results_stats_totals( changing cs_results = ms_results_global ).  
    calculate_global_runtimes( ).  
    if ms_progpars-print_results = abap_true.  
      data(lt_spool) = cl_jva_mig_alv_output=>get_instance( )->print_results(  
        it_results = mt_results  
        is_results = ms_results_global ).  
      append lines of ms_results_global-t_spool to lt_spool.  
      ms_results_global-t_spool[] = lt_spool[].  
    endif.  
  endmethod.
```

UPDATE_TASK_INFO

Description:

Instance mthd

Importing parameter

IS_TASKDATA TYPE CL_JVA_MIG_CORRECT_ACDOCA=>GTY_S_TASKDATA

Changing parameter

CS_RESULTS TYPE IF_JVA_MIG_ALV_OUTPUT=>GTY_S_RESULTS

method update_task_info.

read table cs_results-t_tasks

assigning field-symbol(<fs_tasks>)

with key obj_id = is_taskdata-obj_id

binary search.

if <fs_tasks> is assigned.

<fs_tasks>-returned = abap_true.

<fs_tasks>-scheduled = is_taskdata-scheduled.

<fs_tasks>-initialized = is_taskdata-initialized.

<fs_tasks>-processed = is_taskdata-processed.

<fs_tasks>-runtime = is_taskdata-runtime.

endif.

cs_results-tasks_returned += 1.

cs_results-tasks_executed += 1.

if is_taskdata-scheduled = abap_true.

cs_results-tasks_scheduled += 1.

endif.

if is_taskdata-initialized = abap_true.

cs_results-tasks_initialized += 1.

endif.

if is_taskdata-processed = abap_true.

cs_results-tasks_processed += 1.

endif.

cs_results-runtime_tasks += is_taskdata-runtime.

cs_results-runtime_tasks_admin += is_taskdata-s_results-runtime_tasks_admin.

cs_results-runtime_cocode += is_taskdata-runtime +

is_taskdata-s_results-runtime_tasks_admin.

endmethod.

SET_ERROR_TYPE

Description:
Instance mthd

Importing parameter

IV_OLD_ERROR TYPE CHAR1

IV_NEW_ERROR TYPE CHAR1

Returning parameter

VALUE(RV_ERROR) TYPE CHAR1

method set_error_type.

rv_error = iv_old_error.

if iv_new_error is initial.

return.

endif.

if rv_error is initial.

rv_error = iv_new_error.

return.

endif.

case iv_new_error.

```
when gc_error_type-shortdump.  
    rv_error = iv_new_error.  
when gc_error_type-abend.  
    if iv_old_error <> gc_error_type-shortdump.  
        rv_error = iv_new_error.  
    endif.  
when gc_error_type-error.  
    if iv_old_error <> gc_error_type-shortdump and  
        iv_old_error <> gc_error_type-abend.  
        rv_error = iv_new_error.  
    endif.  
when gc_error_type-warning.  
    if iv_old_error <> gc_error_type-shortdump and  
        iv_old_error <> gc_error_type-abend and  
        iv_old_error <> gc_error_type-error.  
        rv_error = iv_new_error.  
    endif.  
when gc_error_type-statement.  
    if iv_old_error <> gc_error_type-shortdump and  
        iv_old_error <> gc_error_type-abend and  
        iv_old_error <> gc_error_type-error and  
        iv_old_error <> gc_error_type-warning.  
        rv_error = iv_new_error.  
    endif.  
when gc_error_type-info.  
    if iv_old_error <> gc_error_type-shortdump and  
        iv_old_error <> gc_error_type-abend and  
        iv_old_error <> gc_error_type-error and  
        iv_old_error <> gc_error_type-warning and  
        iv_old_error <> gc_error_type-statement.  
        rv_error = iv_new_error.  
    endif.  
when others.  
    "Do nothing  
endcase.  
endmethod.
```

UPDATE_DOC_STATISTICS_ISSUES

Description:

Instance mthd

Importing parameter

IS_LOG TYPE GTY_S_LOG

```
method update_doc_statistics_issues.  
    case is_log-jvdoctype.  
        when gc_jvdoctype-equity_change.  
            ms_results-equity_change-docs_with_issues += 1.  
        when gc_jvdoctype-equity_adjustment.  
            ms_results-equity_adjust-docs_with_issues += 1.  
        when gc_jvdoctype-suspense.  
            ms_results-suspense-docs_with_issues += 1.  
        when gc_jvdoctype-unsuspense.  
            ms_results-suspense-docs_with_issues += 1.
```

```
when gc_jvdoctype=cutback.  
  ms_results-cutback-docs_with_issues += 1.  
when gc_jvdoctype=cash_call.  
  ms_results-cash_call-docs_with_issues += 1.  
when gc_jvdoctype=fi_doc.  
  ms_results-fi_docs-docs_with_issues += 1.  
when gc_jvdoctype=co_doc.  
  ms_results-co_docs-docs_with_issues += 1.  
endcase.  
endmethod.
```

UPDATE_DOC_STATISTICS_UPDATES

Description:

Instance mthd

Importing parameter

IS_LOG TYPE GTY_S_LOG

```
method update_doc_statistics_updates.  
  if is_log-updated = abap_true.  
    case is_log-jvdoctype.  
      when gc_jvdoctype=equity_change.  
        ms_results-equity_change-docs_updated += 1.  
      when gc_jvdoctype=equity_adjustment.  
        ms_results-equity_adjust-docs_updated += 1.  
      when gc_jvdoctype=suspense.  
        ms_results-suspense-docs_updated += 1.  
      when gc_jvdoctype=unsuspense.  
        ms_results-suspense-docs_updated += 1.  
      when gc_jvdoctype=cutback.  
        ms_results-cutback-docs_updated += 1.  
      when gc_jvdoctype=cash_call.  
        ms_results-cash_call-docs_updated += 1.  
      when gc_jvdoctype=fi_doc.  
        ms_results-fi_docs-docs_updated += 1.  
      when gc_jvdoctype=co_doc.  
        ms_results-co_docs-docs_updated += 1.  
    endcase.  
  else.  
    case is_log-error.  
      when gc_error_type=error or gc_error_type=abend.  
        case is_log-jvdoctype.  
          when gc_jvdoctype=equity_change.  
            ms_results-equity_change-docs_update_failures += 1.  
          when gc_jvdoctype=equity_adjustment.  
            ms_results-equity_adjust-docs_update_failures += 1.  
          when gc_jvdoctype=suspense.  
            ms_results-suspense-docs_update_failures += 1.  
          when gc_jvdoctype=unsuspense.  
            ms_results-suspense-docs_update_failures += 1.  
          when gc_jvdoctype=cutback.  
            ms_results-cutback-docs_update_failures += 1.  
          when gc_jvdoctype=cash_call.  
            ms_results-cash_call-docs_update_failures += 1.
```

```

        when gc_jvdoctype-fi_doc.
            ms_results-fi_docs-docs_update_failures += 1.
        when gc_jvdoctype-co_doc.
            ms_results-co_docs-docs_update_failures += 1.
        endcase.
    when gc_error_type-warning.
        case is_log-jvdoctype.
            when gc_jvdoctype-equity_change.
                ms_results-equity_change-docs_warnings += 1.
            when gc_jvdoctype-equity_adjustment.
                ms_results-equity_adjust-docs_warnings += 1.
            when gc_jvdoctype-suspense.
                ms_results-suspense-docs_warnings += 1.
            when gc_jvdoctype-unsuspense.
                ms_results-suspense-docs_warnings += 1.
            when gc_jvdoctype-cutback.
                ms_results-cutback-docs_warnings += 1.
            when gc_jvdoctype-cash_call.
                ms_results-cash_call-docs_warnings += 1.
            when gc_jvdoctype-fi_doc.
                ms_results-fi_docs-docs_warnings += 1.
            when gc_jvdoctype-co_doc.
                ms_results-co_docs-docs_warnings += 1.
            endcase.
        endcase.
    endif.
endmethod.
```

UPDATE_DOC_STATISTIC_PROCESSED

Description:
Instance mthd

Importing parameter

IV_JVDOCTYPE TYPE CHAR2

```

method update_doc_statistic_processed.
    case iv_jvdoctype.
        when gc_jvdoctype-fi_doc.
            ms_results-fi_docs-docs_processed += 1.
        when gc_jvdoctype-co_doc.
            ms_results-co_docs-docs_processed += 1.
        when gc_jvdoctype-equity_change.
            ms_results-equity_change-docs_processed += 1.
        when gc_jvdoctype-equity_adjustment.
            ms_results-equity_adjust-docs_processed += 1.
        when gc_jvdoctype-suspense or gc_jvdoctype-unsuspense.
            ms_results-suspense-docs_processed += 1.
        when gc_jvdoctype-cutback.
            ms_results-cutback-docs_processed += 1.
        when gc_jvdoctype-cash_call.
            ms_results-cash_call-docs_processed += 1.
        endcase.
    endmethod.
```

UPDATE_DOC_STATS_ALIGNM_FAILS

Description:
Instance mthd

Importing parameter

IS_LOG TYPE GTY_S_LOG

```
method update_doc_stats_alignm_fails.  
  case is_log-jvdoctype.  
    when gc_jvdoctype-equity_change.  
      ms_results-equity_change-docs_alignm_failures += 1.  
    when gc_jvdoctype-equity_adjustment.  
      ms_results-equity_adjust-docs_alignm_failures += 1.  
    when gc_jvdoctype-suspense.  
      ms_results-suspense-docs_alignm_failures += 1.  
    when gc_jvdoctype-unsuspense.  
      ms_results-suspense-docs_alignm_failures += 1.  
    when gc_jvdoctype-cutback.  
      ms_results-cutback-docs_alignm_failures += 1.  
    when gc_jvdoctype-cash_call.  
      ms_results-cash_call-docs_alignm_failures += 1.  
    when gc_jvdoctype-fi_doc.  
      ms_results-fi_docs-docs_alignm_failures += 1.  
    when gc_jvdoctype-co_doc.  
      ms_results-co_docs-docs_alignm_failures += 1.  
  endcase.  
endmethod.
```

UPDATE_LINE_STATISTICS_ISSUES

Description:
Instance mthd

Importing parameter

IS_LOG TYPE GTY_S_LOG

```
method update_line_statistics_issues.  
  case is_log-activ.  
    when gc_jvlinetype-equity_change.  
      ms_results-equity_change-lines_with_issues += 1.  
    when gc_jvlinetype-eq_adjust_reversal.  
      ms_results-equity_adjust-lines_with_issues += 1.  
    when gc_jvlinetype-eq_adjust_reposting.  
      ms_results-equity_adjust-lines_with_issues += 1.  
    when gc_jvlinetype-suspense.  
      ms_results-suspense-lines_with_issues += 1.  
    when gc_jvlinetype-unsuspense.  
      ms_results-suspense-lines_with_issues += 1.  
    when gc_jvlinetype-cutback_operator.  
      ms_results-cutback-lines_with_issues += 1.  
    when gc_jvlinetype-cashcall_operator.  
      ms_results-cash_call-lines_with_issues += 1.  
    when gc_jvlinetype-fi_doc or gc_jvlinetype-fi_doc_mm.  
      ms_results-fi_docs-lines_with_issues += 1.  
    when gc_jvlinetype-co_doc.
```

```
        ms_results-co_docs-lines_with_issues += 1.  
    endcase.  
endmethod.
```

UPDATE_LINE_STATISTICS_UPDATES

Description:

Instance mthd

Importing parameter

IS_LOG TYPE GTY_S_LOG

```
method update_line_statistics_updates.  
    if is_log-updated = abap_true.  
        case is_log-activ.  
            when gc_jvlinetype-equity_change.  
                ms_results-equity_change-lines_updated += 1.  
            when gc_jvlinetype-eq_adjust_reversal.  
                ms_results-equity_adjust-lines_updated += 1.  
            when gc_jvlinetype-eq_adjust_reposting.  
                ms_results-equity_adjust-lines_updated += 1.  
            when gc_jvlinetype-suspense.  
                ms_results-suspense-lines_updated += 1.  
            when gc_jvlinetype-unsuspense.  
                ms_results-suspense-lines_updated += 1.  
            when gc_jvlinetype-cutback_operator.  
                ms_results-cutback-lines_updated += 1.  
            when gc_jvlinetype-cashcall_operator.  
                ms_results-cash_call-lines_updated += 1.  
            when gc_jvlinetype-fi_doc or gc_jvlinetype-fi_doc_mm.  
                ms_results-fi_docs-lines_updated += 1.  
            when gc_jvlinetype-co_doc.  
                ms_results-co_docs-lines_updated += 1.  
        endcase.  
    else.  
        case is_log-error.  
            when gc_error_type-error or gc_error_type-abend.  
                case is_log-activ.  
                    when gc_jvlinetype-equity_change.  
                        ms_results-equity_change-lines_update_failures += 1.  
                    when gc_jvlinetype-eq_adjust_reversal.  
                        ms_results-equity_adjust-lines_update_failures += 1.  
                    when gc_jvlinetype-eq_adjust_reposting.  
                        ms_results-equity_adjust-lines_update_failures += 1.  
                    when gc_jvlinetype-suspense.  
                        ms_results-suspense-lines_update_failures += 1.  
                    when gc_jvlinetype-unsuspense.  
                        ms_results-suspense-lines_update_failures += 1.  
                    when gc_jvlinetype-cutback_operator.  
                        ms_results-cutback-lines_update_failures += 1.  
                    when gc_jvlinetype-cashcall_operator.  
                        ms_results-cash_call-lines_update_failures += 1.  
                    when gc_jvlinetype-fi_doc or gc_jvlinetype-fi_doc_mm.  
                        ms_results-fi_docs-lines_update_failures += 1.  
                    when gc_jvlinetype-co_doc.
```

```

        ms_results-co_docs-lines_update_failures += 1.
    endcase.
when gc_error_type-warning.
    case is_log-activ.
        when gc_jvlinetype-equity_change.
            ms_results-equity_change-lines_warnings += 1.
        when gc_jvlinetype-eq_adjust_reversal.
            ms_results-equity_adjust-lines_warnings += 1.
        when gc_jvlinetype-eq_adjust_reposting.
            ms_results-equity_adjust-lines_warnings += 1.
        when gc_jvlinetype-suspense.
            ms_results-suspense-lines_warnings += 1.
        when gc_jvlinetype-unsuspense.
            ms_results-suspense-lines_warnings += 1.
        when gc_jvlinetype-cutback_operator.
            ms_results-cutback-lines_warnings += 1.
        when gc_jvlinetype-cashcall_operator.
            ms_results-cash_call-lines_warnings += 1.
        when gc_jvlinetype-fi_doc.
            ms_results-fi_docs-lines_warnings += 1.
        when gc_jvlinetype-co_doc.
            ms_results-co_docs-lines_warnings += 1.
    endcase.
endcase.
endif.
endmethod.
```

UPDATE_LINE_STATS_ALIGNM_FAILS

Description:
Instance mthd

Importing parameter

IS_LOG TYPE GTY_S_LOG

```

method update_line_stats_alignm_fails.
    case is_log-activ.
        when gc_jvlinetype-equity_change.
            ms_results-equity_change-lines_alignm_failures += 1.
        when gc_jvlinetype-eq_adjust_reversal.
            ms_results-equity_adjust-lines_alignm_failures += 1.
        when gc_jvlinetype-eq_adjust_reposting.
            ms_results-equity_adjust-lines_alignm_failures += 1.
        when gc_jvlinetype-suspense.
            ms_results-suspense-lines_alignm_failures += 1.
        when gc_jvlinetype-unsuspense.
            ms_results-suspense-lines_alignm_failures += 1.
        when gc_jvlinetype-cutback_operator.
            ms_results-cutback-lines_alignm_failures += 1.
        when gc_jvlinetype-cashcall_operator.
            ms_results-cash_call-lines_alignm_failures += 1.
        when gc_jvlinetype-fi_doc or gc_jvlinetype-fi_doc_mm.
            ms_results-fi_docs-lines_alignm_failures += 1.
        when gc_jvlinetype-co_doc.
            ms_results-co_docs-lines_alignm_failures += 1.
```

```
endcase.  
endmethod.
```

UPDATE_LINE_STATS_PROCESSED

Description:
Instance mthd

Importing parameter

IS_JVSO TYPE JVSO

```
method update_line_stats_processed.  
  case is_jvso-activ.  
    when gc_jvlinetype-fi_doc or gc_jvlinetype-fi_doc_mm.  
      ms_results-fi_docs-lines_processed += 1.  
    when gc_jvlinetype-co_doc.  
      ms_results-co_docs-lines_processed += 1.  
    when gc_jvlinetype-equity_change.  
      ms_results-equity_change-lines_processed += 1.  
    when gc_jvlinetype-eq_adjust_reversal.  
      ms_results-equity_adjust-lines_processed += 1.  
    when gc_jvlinetype-eq_adjust_reposting.  
      ms_results-equity_adjust-lines_processed += 1.  
    when gc_jvlinetype-suspense.  
      ms_results-suspense-lines_processed += 1.  
    when gc_jvlinetype-unsuspense.  
      ms_results-suspense-lines_processed += 1.  
    when gc_jvlinetype-cutback_operator.  
      ms_results-cutback-lines_processed += 1.  
    when gc_jvlinetype-cashcall_operator.  
      ms_results-cash_call-lines_processed += 1.  
  endcase.  
endmethod.
```

UPDATE_LOG_DB_TABLE

Description:
Instance mthd

```
method update_log_db_table.  
  if mt_jvaacdocacorrlog[] is initial.  
    return.  
  endif.  
  if ms_progpars-log_use_update_task_for_db_upd = abap_true.  
    call function 'JV_UPDATE_ACD_CORR_LOG' in update task  
      exporting  
        it_items          = mt_jvaacdocacorrlog  
        iv_set_timestamp = ms_progpars-log_set_timestamp_at_db_update.  
  else.  
    modify jva_acd_upd_log from table mt_jvaacdocacorrlog.  
  endif.  
  commit work.  
  clear mt_jvaacdocacorrlog.  
endmethod.
```

UPDATE_LOG_DB_TABLE_PREP

Description:
Instance mthd

Changing parameter

CS_LOG TYPE GTY_S_LOG

```
method update_log_db_table_prep.  
  data: ls_jvaacdocacorrlog type jva_acd_upd_log.  
  move-corresponding cs_log to ls_jvaacdocacorrlog.  
  ls_jvaacdocacorrlog-rcmnt = sy-mandt.  
  ls_jvaacdocacorrlog-message = cs_log-comment.  
  if ls_jvaacdocacorrlog-rldnr is initial.  
    ls_jvaacdocacorrlog-rldnr = '4A'.  
  endif.  
  ls_jvaacdocacorrlog-timestamp = ms_progpars-timestamp_db_updates.  
  append ls_jvaacdocacorrlog to mt_jvaacdocacorrlog.  
  if lines( mt_jvaacdocacorrlog ) >= ms_progpars-log_bulk_no_of_itms_for_db_upd.  
    update_log_db_table( ).  
  endif.  
endmethod.
```

UPDATE_LOG_WITH_POSTING_RESULT

Description:
Instance mthd

Importing parameter

IO_MESSAGES TYPE REF TO IF_JVA_MESSAGE_COLLECTION
IV_UPDATED TYPE ABAP_BOOL
IV_BELNR TYPE BELNR_D
IV_GJAHR TYPE GJAHR
IV_DOCLN TYPE DOCLN6
IV_COMMENT TYPE STRING

Changing parameter

CS_LOG TYPE GTY_S_LOG

```
method update_log_with_posting_result.  
  cs_log-updated = iv_updated.  
  cs_log-docln_acd = iv_docln.  
  if iv_updated = abap_true.  
    if cs_log-corr_type <> gc_corr_type-update_line.  
      if ms_progpars-testrun = abap_true.  
        cs_log-belnr_new = 'TEST'.  
      else.  
        cs_log-belnr_new = iv_belnr.  
        cs_log-gjahr_new = iv_gjahr.  
        cs_log-docln_new = iv_docln.  
      endif.  
    endif.  
    add_message_to_log(  
      exporting  
        iv_msg_text    = iv_comment  
        iv_msg_type    = gc_error_type-statement  
        iv_is_upd_msg  = abap_true
```

```
        changing
          cs_log = cs_log ).
    else.
      cs_log-belnr_new = 'ERROR'.
      if io_messages->has_critical_messages( ) = abap_true.
        cs_log-error = io_messages->get_max_severity( ).
      endif.
      loop at io_messages->get_access( )->get_messages_as_bapiret( ) into
data(ls_message)
        where type = gc_error_type-error or
              type = gc_error_type-abend or
              type = gc_error_type-shortdump.
      add_message_to_log(
        exporting
          is_message = ls_message
        changing
          cs_log      = cs_log ).
    endloop.
    add_message_to_log(
      exporting
        iv_msg_text   = iv_comment
        iv_msg_type    = gc_error_type-error
        iv_is_upd_msg = abap_true
      changing
        cs_log = cs_log ).
    endif.
  endmethod.
```

Redefined Methods

Local Types

```
*** use this source file for any type of declarations (class
*** definitions, interfaces or type declarations) you need for
*** components in the private section
```

Local class definitions

```
*** use this source file for the definition and implementation of
*** local helper classes, interface definitions and type
*** declarations
```

Macros

```
*** use this source file for any macro definitions you need
*** in the implementation part of the class
```

Overview

Attributes	1
Attributes	1
Public attributes	1
Protected attribute	1
Private attribute	1
Intern. types	2
Private types	3
Methods	3
Public methods	3
GET_INSTANCE	3
SET_INSTANCE	4
Private methods	4
CONSTRUCTOR	4
ADD_BAL_MESSAGE_TO_LOG	4
ADD_COMPRESSION_MESSAGE	6
ADD_ISSUE_INFO	8
ADD_MESSAGE_TO_LOG	9
ADD_UP_ALL_RESULTS_STATS	11
ADD_UP_RESULTS_STATS_TOTALS	12
ADD_UP_RESULTS_STATS	13
ADD_SY_MESSAGE_TO_RESULTS	13
ADD_ITEM_INFO_FOR_LINE	13
ADD_ITEM_INFO_FOR_DOC	15
APPEND_LOG_ITEM_FOR_JV_LINE	18
CALC_RUNTIME	18
CALCULATE_GLOBAL_RUNTIMES	19
CHECK_COCODES_AND_AUTHORITIES	19
CHECK_FOR_REPOSTINGS_ACDOCA	20
CHECK_FOR_REPOSTINGS_JV	21
COMBINE_AND_ANALYZE_JV_LINES	22
COMPARE_AND_CORRECT_DOCUMENTS	24
COMPARE_BALANCES	31
COMPARE_FIELD_VALUES	36
COMPARE_LINES	47
COMPRESS_JV_DOC	54
CORRECT_ACDOCA_LINE	56
CORRECT_ACDOCA_LINES	58
CALCULATE_TASK_PACKAGES	61
COUNT_JV_DOCUMENTS_BY_PERIOD	62
COUNT_JV_DOCUMENTS_TOTAL	64
DETERMINE_CORRECTION_TYPE	65
DETERMINE_JVACTIVITY	67
DETERMINE_JV_AMNTS_IN_ACD_LINE	67
DETERMINE_JV_DOC_TYPE	68
DISPLAY_RESULTS	69
GET_ACDOCA_AMOUNT_FOR_JV	69
GET_ACDOCA_DOC_LINES	70
GET_ACDOCA_LINE_BY_ACC_DATA	73

GET_ACDOCA_LINE_BY_REF	75
GET_ACDOCA_LINE_FOR_JV	85
GET_CLASSIC_CURRENCIES	88
GET_COMPANY_CODE_SETTINGS	90
GET_CURRENCIES	91
GET_GLOBAL_SETTINGS	91
GET_LEDGERS	93
GET_LOG_ITEM	94
GET_SPLIT_CRITERIA	94
GET_TIMESTAMP	96
IS_CRITICAL_ERROR	96
JV_CURTP_EXISTS_AS_GL_CURTP	97
MAKE_WHERE_CLAUSE	97
MAKE_REFERENCE_CONDITION	101
MULTIPLE_ACD_LINE_PROCESSING	101
POST_MISSING_DOCUMENT	111
PREPARE_ACDOCA_BALANCES	113
PREPARE_ACDOCA_DOCUMENT	114
PREPARE_ACTIVITY_RANGE_FI_CO	115
PREPARE_ACTIVITY_RANGE_ME	116
PREPARE_ACTIVITY_RANGE_OP	117
PREPARE_JVSO_BALANCES	117
PROCESS_COMPANY_CODE	118
PREPARE_PERIOD_RANGE	119
PROCESS_DATA_PARALLELLY	119
PROCESS_DATA_SYNCHRONOUSLY	121
RETRIEVE_ACDOCA_LINES	121
RETRIEVE_AND_PROCESS_DATA	125
RETRIEVE_FI_DOC_HEADER_DATA	130
RETRIEVE_JV_DOCUMENTS	132
RETRIEVE_JV_DOC_LINES	135
RETRIEVE_LOG_ITEMS_DIFF_YEARS	136
RETURN_MULTIPLE_LINES_HIT	137
REV_AND_REP_ACDOCA_DOC	138
SAVE_RESULTS	141
SAVE_RESULTS_GLOBAL	141
UPDATE_TASK_INFO	142
SET_ERROR_TYPE	143
UPDATE_DOC_STATISTICS_ISSUES	144
UPDATE_DOC_STATISTICS_UPDATES	145
UPDATE_DOC_STATISTIC_PROCESSED	146
UPDATE_DOC_STATS_ALIGNM_FAILS	147
UPDATE_LINE_STATISTICS_ISSUES	147
UPDATE_LINE_STATISTICS_UPDATES	148
UPDATE_LINE_STATS_ALIGNM_FAILS	149
UPDATE_LINE_STATS_PROCESSED	150
UPDATE_LOG_DB_TABLE	150
UPDATE_LOG_DB_TABLE_PREP	150
UPDATE_LOG_WITH_POSTING_RESULT	151
Redefined Methods	152
Local Types	152

Local class definitions

152

Macros

152